

DWH

Content created from Kimball's book "Datawarehouse Toolkit"

Why do we need a
data warehouse?



Record Keeping – Operational System

One of the most important assets of any organization is its information.

This asset is almost always used for two purposes:

- operational record keeping
- analytical decision making.

Simply speaking, the operational systems are where you put the data in, and the DW/BI system is where you get the data out.

Record Keeping – Operational System

Users of an operational system turn the wheels of the organization – they take orders, sign up new customers, monitor the status of operational activities, and log complaints.

The operational systems are optimized to process transactions quickly.

These systems almost always deal with one transaction record at a time. They predictably perform the same operational tasks over and over, executing the organization's business processes.

Given this execution focus, operational systems typically do not maintain history, but rather update data to reflect the most current state.

Requirement – DW / BI System

Users of a DW/BI system, on the other hand, watch the wheels of the organization turn to evaluate performance - they count the new orders and compare them with last week's orders, and ask why the new customers signed up, and what the customers complained about.

Although they need detailed data to support their constantly changing questions, DW/BI users almost never deal with one transaction at a time.

These systems are optimized for high-performance queries as users' questions often require that hundreds or hundreds of thousands of transactions be searched and compressed into an answer set.

To further complicate matters, users of a DW/BI system typically demand that historical context be preserved to accurately evaluate the organization's performance over time.

Goals of Data Warehousing and Business Intelligence

“We collect tons of data, but we can’t access it.”

“We need to slice and dice the data every which way.”

“Business people need to get at the data easily.”

“Just show me what is important.”

“We spend entire meetings arguing about who has the right numbers rather than making decisions.”

“We want people to use information to support more fact-based decision making.”

Requirement of DW/BI System

The DW/BI system must make information easily accessible.

- The contents of the DW/BI system must be understandable and Data must be intuitive and obvious to the business user, not just the developer.
- Data structures and labels should reflect business users' thought processes and vocabulary.
- Business users should be able to separate and combine analytic data in endless combinations.
- Business intelligence tools and applications must be simple and easy to use.
- Tools must return query results with minimal wait times.
- In summary: the system should be **simple and fast**.

Requirement of DW/BI System

- The DW/BI system must present information consistently.
- The data in the DW/BI system must be credible.
- Data must be:
 - Carefully assembled from a variety of sources
 - Cleansed and quality assured
 - Released only when fit for user consumption
- Consistency requires using common labels and definitions across all data sources.
- If two performance measures have the same name, they must represent the same concept.
- If two measures differ in meaning, they should be labeled differently.

Requirement of DW/BI System

- The DW/BI system must adapt to change.
- User needs, business conditions, data, and technology are all subject to change.
- The system must handle change gracefully without invalidating existing data or applications.
- Existing data and applications should remain stable when:
 - New questions are asked by the business
 - New data is added to the warehouse
- If descriptive data must be modified:
 - Changes must be properly accounted for
 - Changes should be made transparent to users

Requirement of DW/BI System

- The DW/BI system must present information in a timely manner.
- As operational use increases, raw data may need to be transformed into actionable information within:
 - Hours
 - Minutes
 - Or even seconds
- The DW/BI team and business users must set realistic expectations for:
 - Data delivery speed
- The level of data cleaning or validation possible under tight timeframes

Requirement of DW/BI System

- The DW/BI system must be a secure bastion that protects information assets.
- The data warehouse contains the organization's informational crown jewels.
- It likely holds sensitive data such as:
 - What is being sold
 - To whom it is being sold
 - At what price
- This information could be harmful if accessed by the wrong people.
- The DW/BI system must effectively control access to confidential information.

Requirement of DW/BI System

- The DW/BI system must serve as the authoritative and trustworthy foundation for improved decision making.
- The data warehouse must contain the right data to support effective decisions.
- The most important outputs of a DW/BI system are the decisions driven by analytic evidence.
- These decisions create the business impact and value of the DW/BI system.
- The original term "**decision support system**" remains the best description of its purpose.

Requirement of DW/BI System

- The business community must accept the DW/BI system for it to be considered successful.
- Technical excellence alone (e.g., using best-of-breed products) is not enough.
- If business users do not actively use the DW/BI environment, it fails the acceptance test.
- Unlike operational systems, DW/BI system usage is often optional for users.
- Business users will embrace the DW/BI system if it becomes their “**simple and fast**” source of actionable information.

Two purposes

- What's the best category?
- How many sales compared to last month?
- What can be improved?

⇒ Evaluate performance
⇒ Decision-making



**Analytical
decision making**

OLAP=Online Analytical Processing



**Operational
data keeping**

- Receive orders
- React to complaints
- Fill up stock

⇒ Turn the wheel

OLTP=Online Transactional Processing

Why do we need a
data warehouse?

Two requirements

- Thousands of records at a time
- Fast query performance
- Historical context
- Usability



Analytical
decision making



Operational
data keeping

- One record at a time
- Data input
- No long history

Used for reporting and data analysis

DWH is there to address those analytical data needs!

Why do we need a
data warehouse?

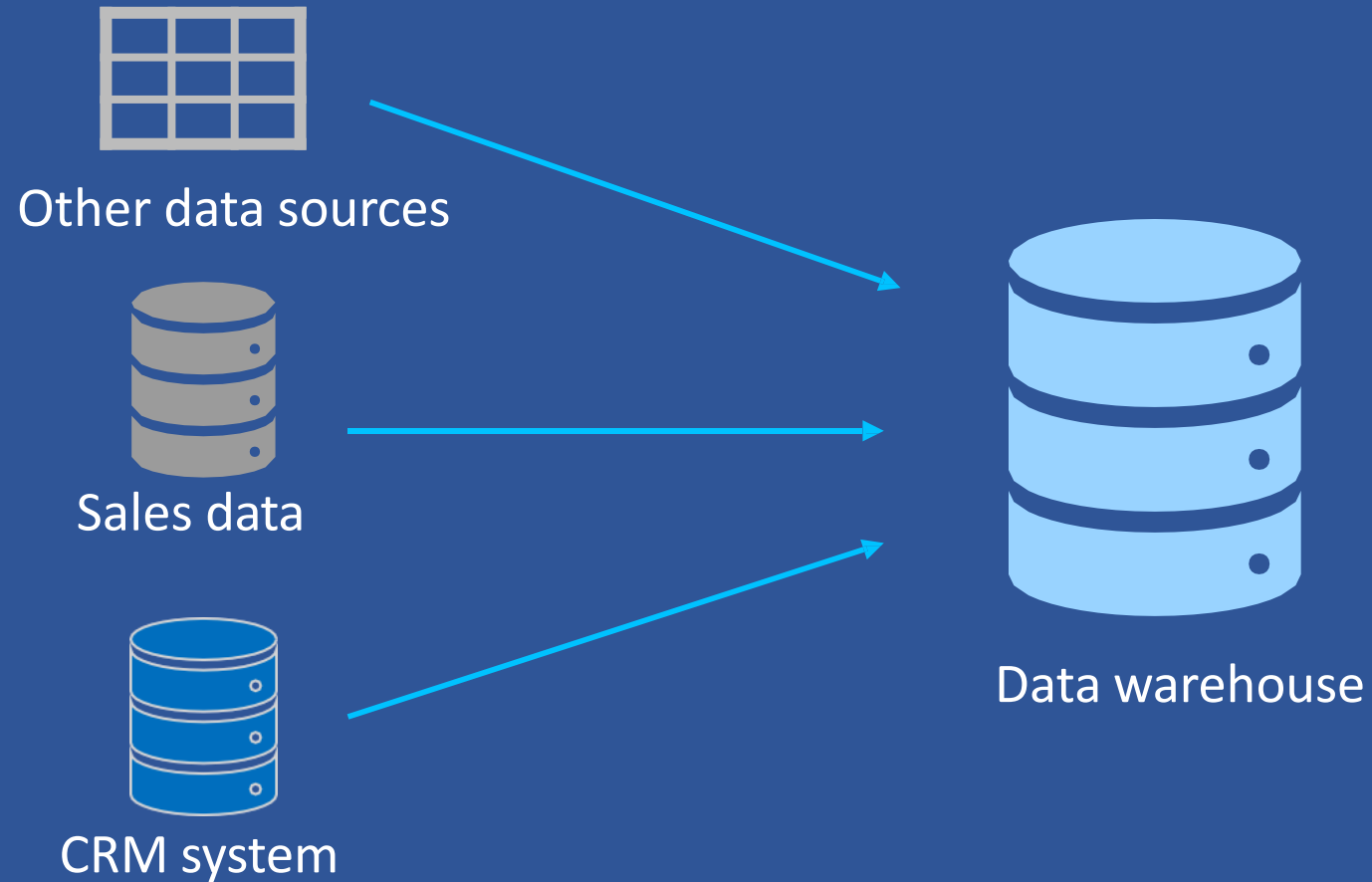
What is a Data Warehouse ?

Data warehouse:

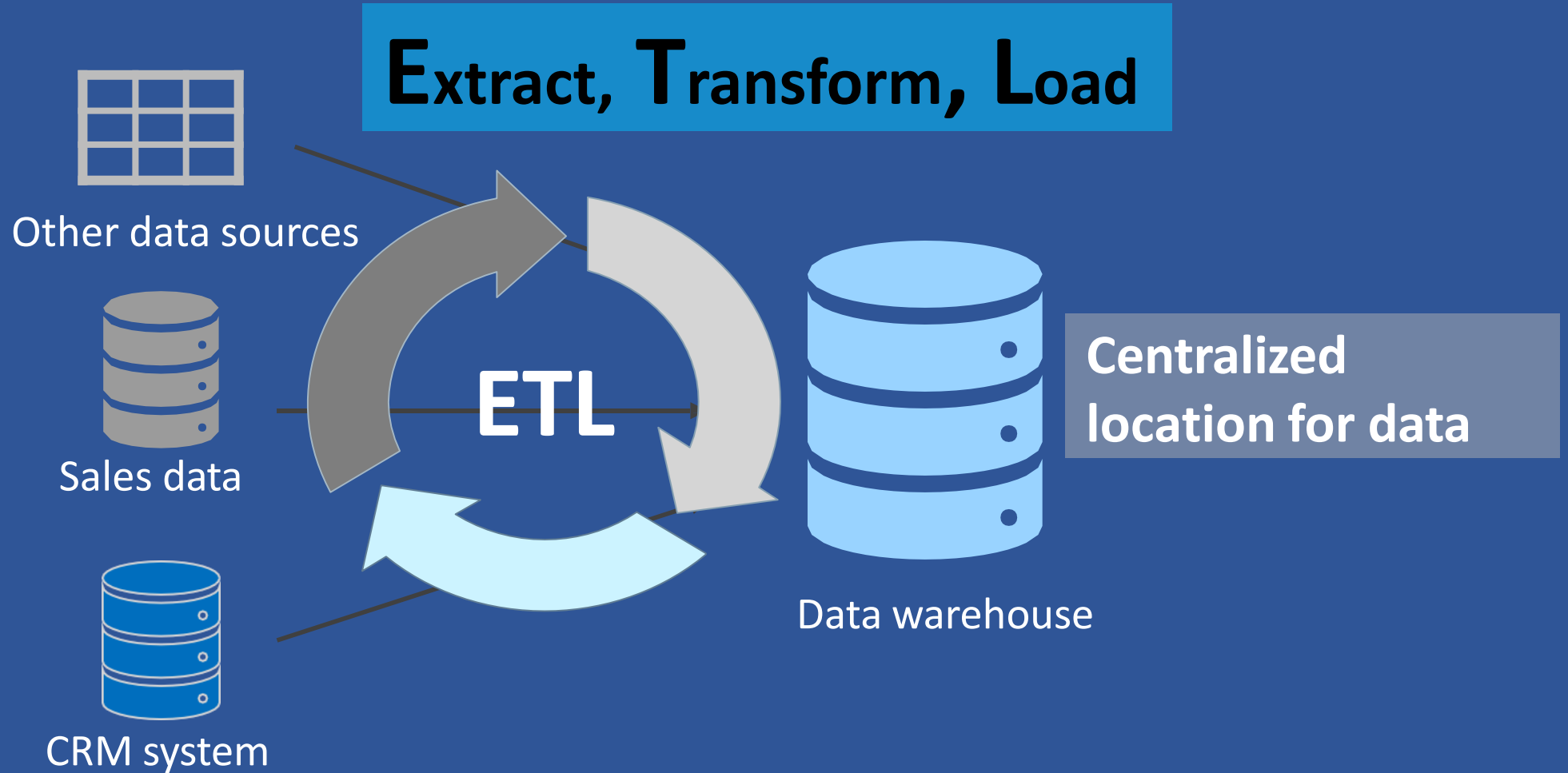
A Relational database used and optimized for analytical purposes.

- ✓ User friendly
- ✓ Fast query performance
- ✓ Enabling data analysis

Understanding a data warehouse



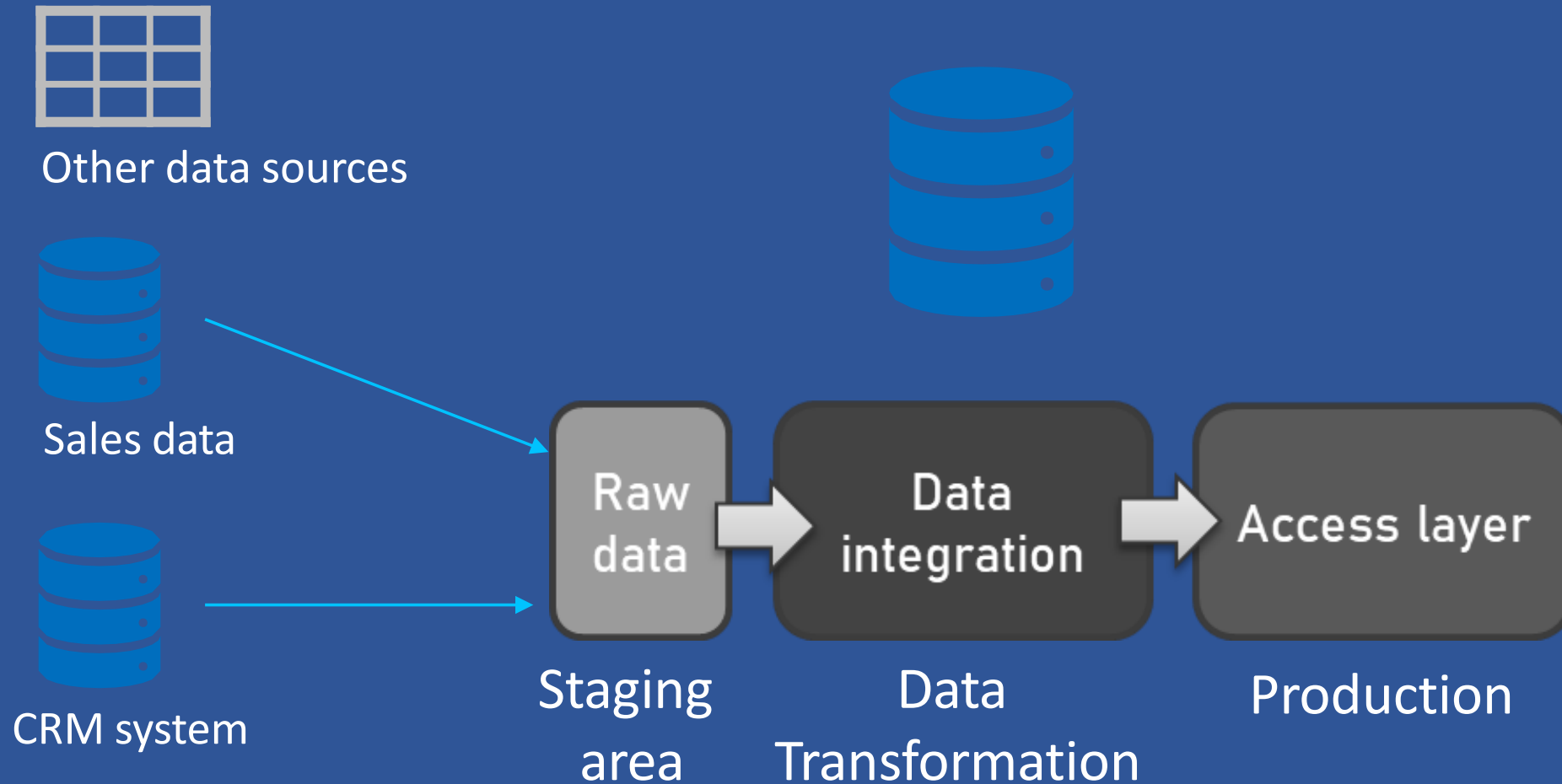
Understanding a data warehouse



Goals of a data warehouse

- ✓ Centralized and consistent location for data
- ✓ Data must be accessible fast (query performance)
- ✓ User-friendly (easy to understand)
- ✓ Must load data consistently and repeatedly (ETL)
- ✓ Reporting and data visualization built on top

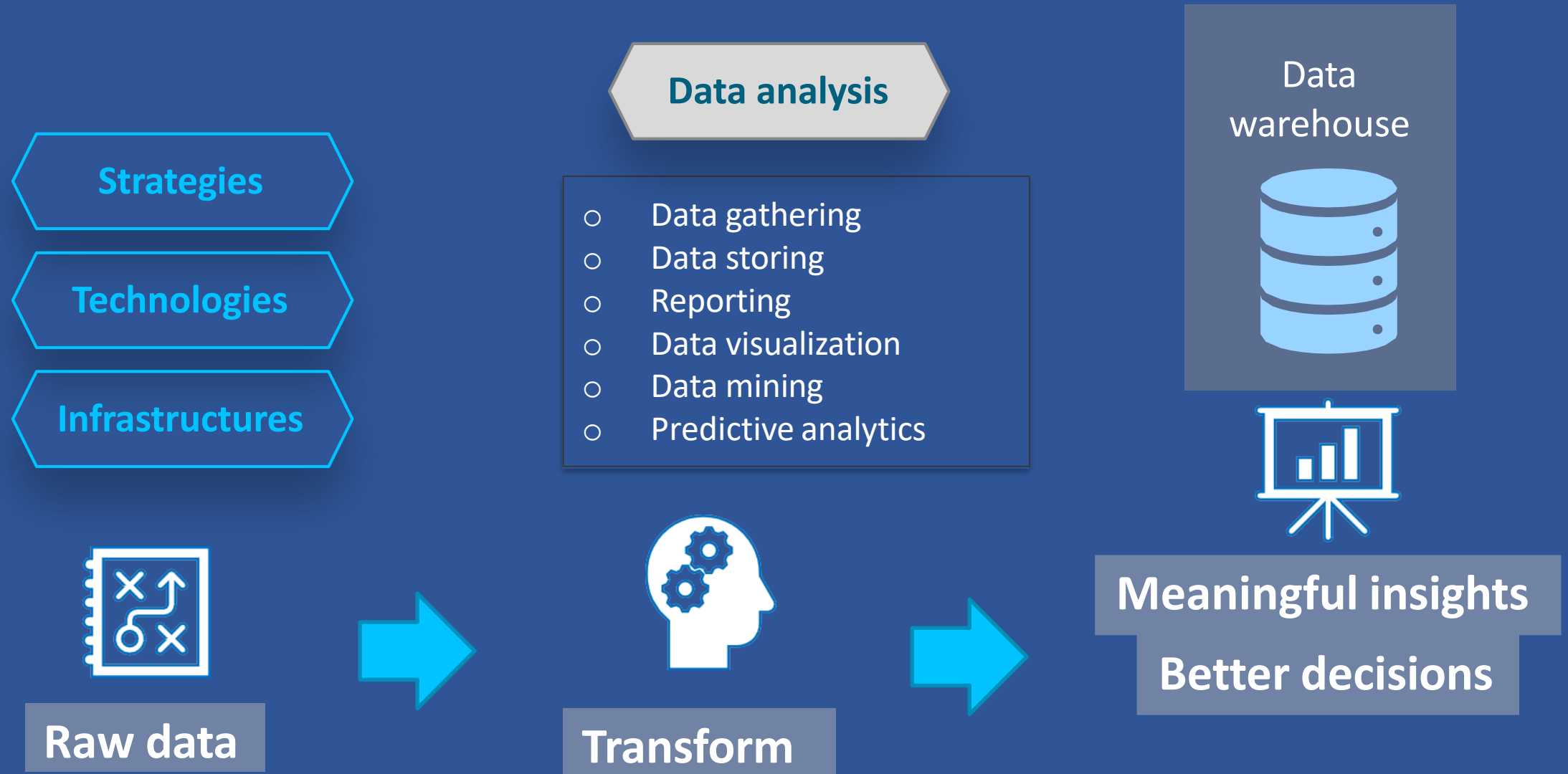
Understanding a data warehouse



We create a data warehouse for
Business Intelligence...

What is Business Intelligence?

What is Business Intelligence?



Data Lake & Data Warehouse Layers

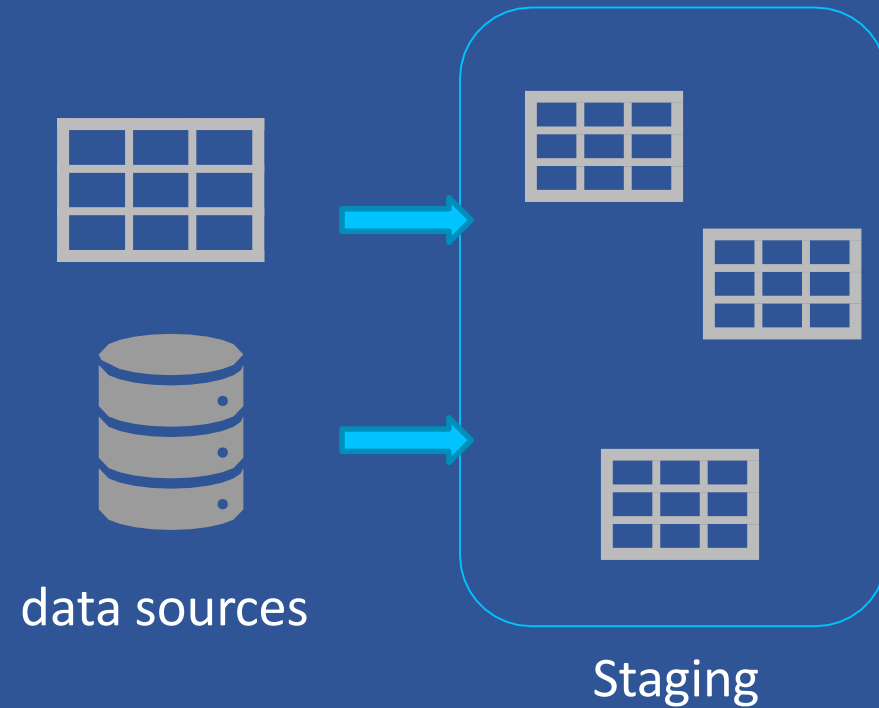
**Data lake & data warehouse are BOTH used as
centralized data storage**

Data Lake & Data Warehouse Layers

	Data Lake	Data Warehouse
Data	Raw	Processed
Technologies	Big data	Database
Structure	Unstructured	Structured
Usage	Not defined yet	Specific & ready to be used
Users	Data Scientists	Business users & IT

The layers of a Data Warehouse

Data Warehouse Layers



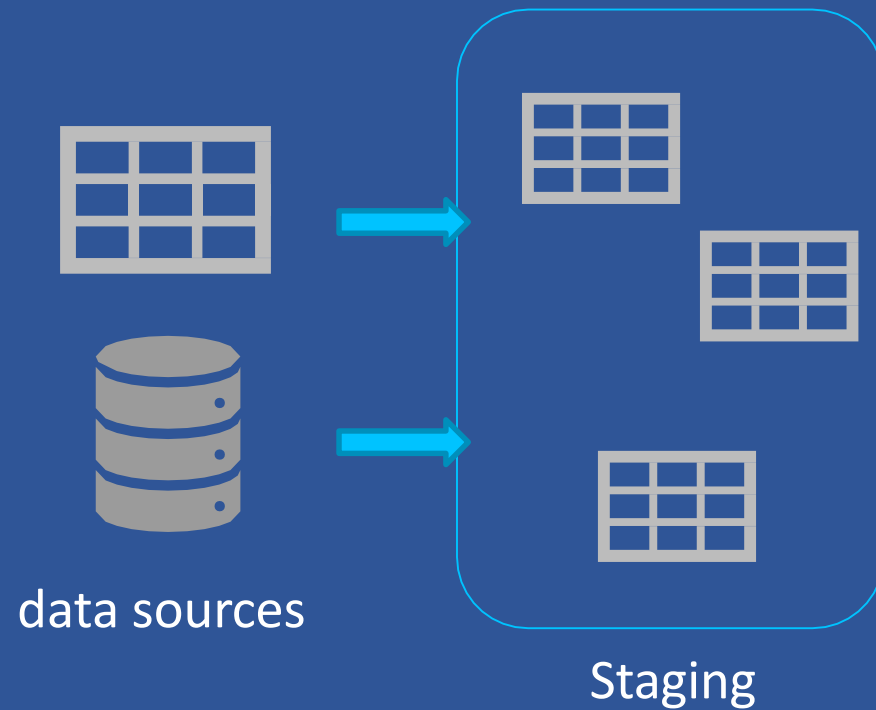
Department 1

employee_id	entry_date	position_level
1	1/2/2022	HR
2	1/6/2022	IT
3	1/4/2022	IT
4	1/7/2022	UM
5	1/6/2022	PM

Department 2

employee_id	entry_date	position_level
6	1/5/2022	HR
7	1/6/2022	HR
8	1/8/2022	PM
9	1/7/2022	UM
10	1/6/2022	PM

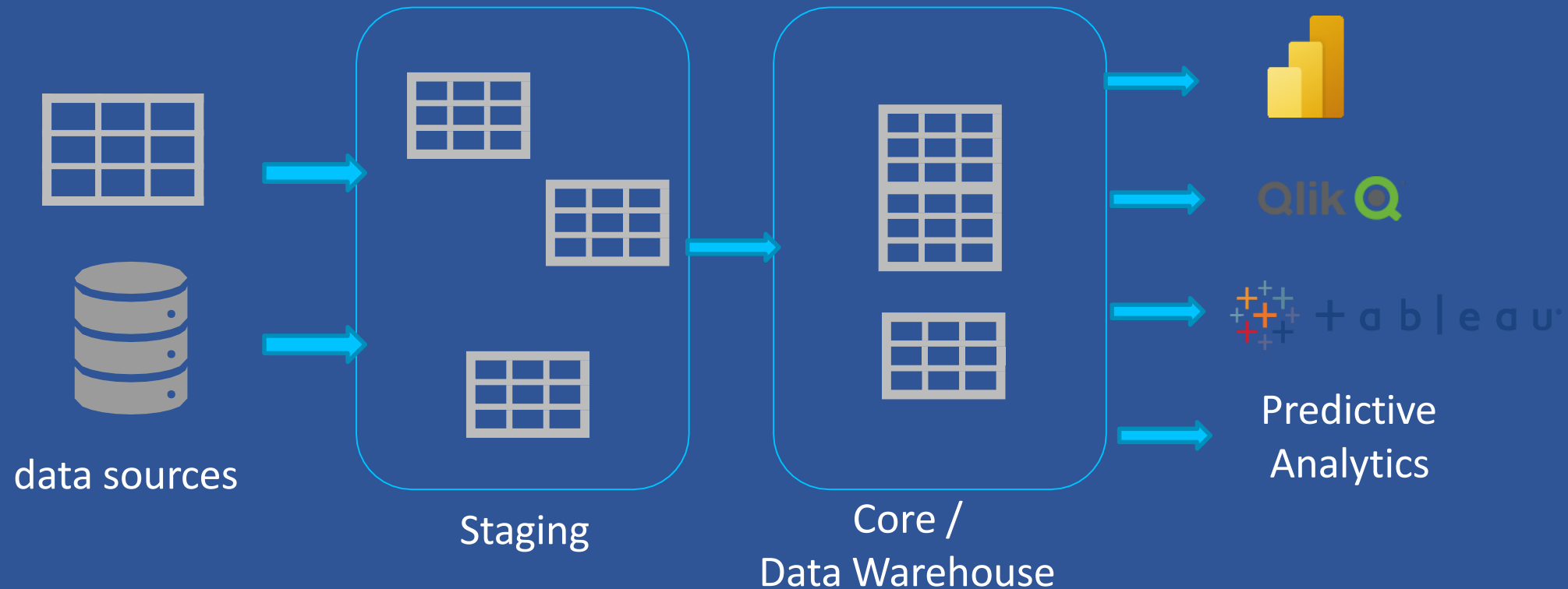
Data Warehouse Layers



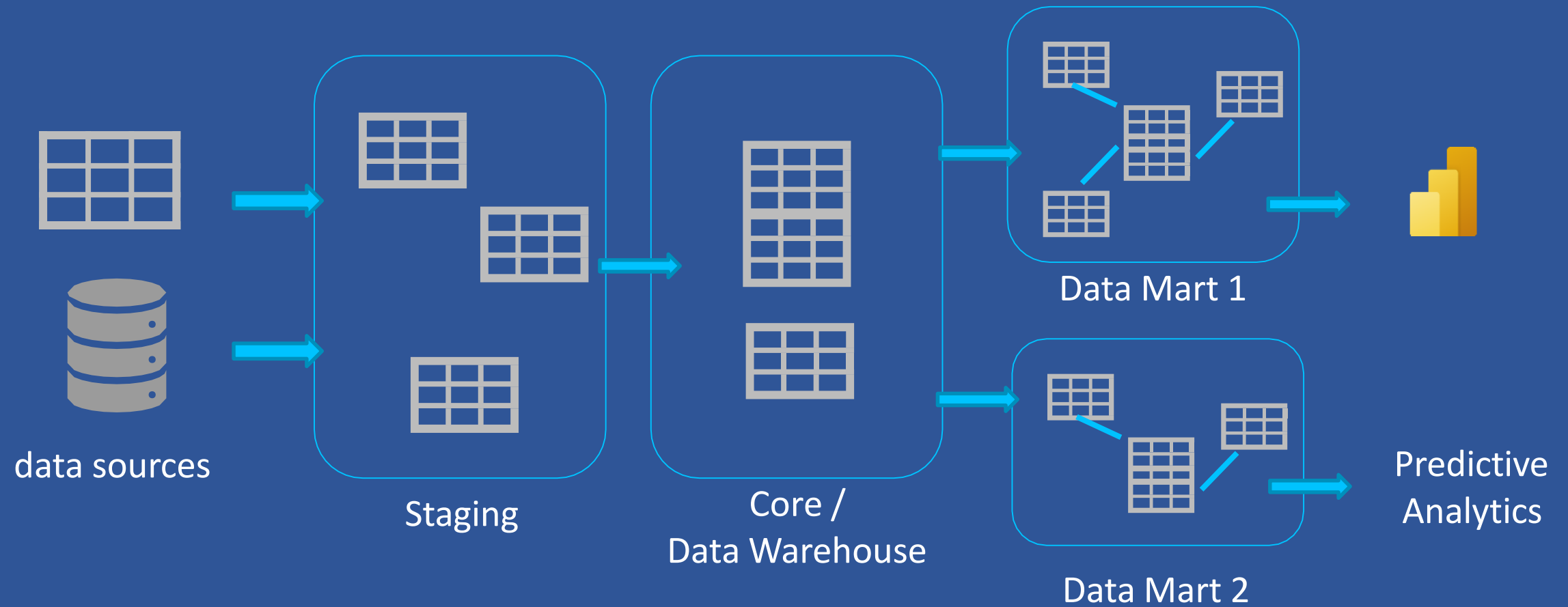
Departments

employee_id	entry_date	position_level
1	1/2/2022	HR
2	1/6/2022	IT
3	1/4/2022	IT
4	1/7/2022	UM
5	1/6/2022	PM
6	1/5/2022	HR
7	1/6/2022	HR
8	1/8/2022	PM
9	1/7/2022	UM
10	1/6/2022	PM

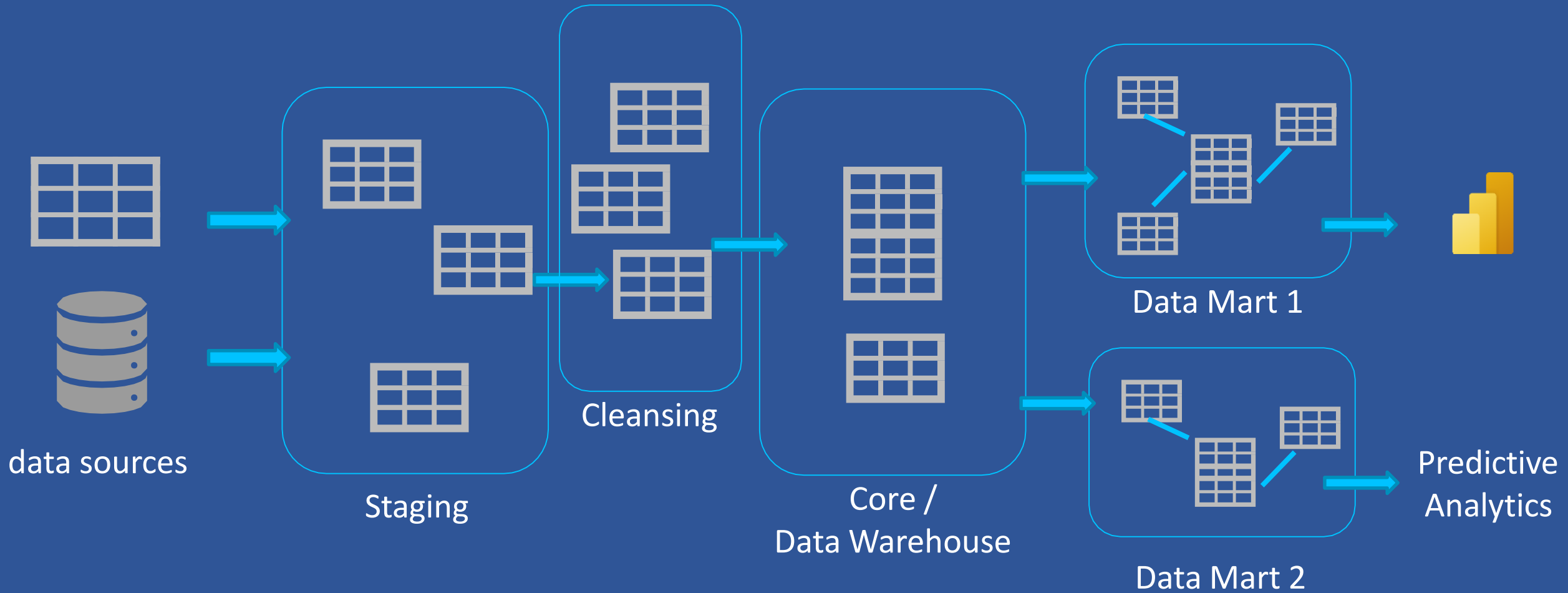
Data Warehouse Layers



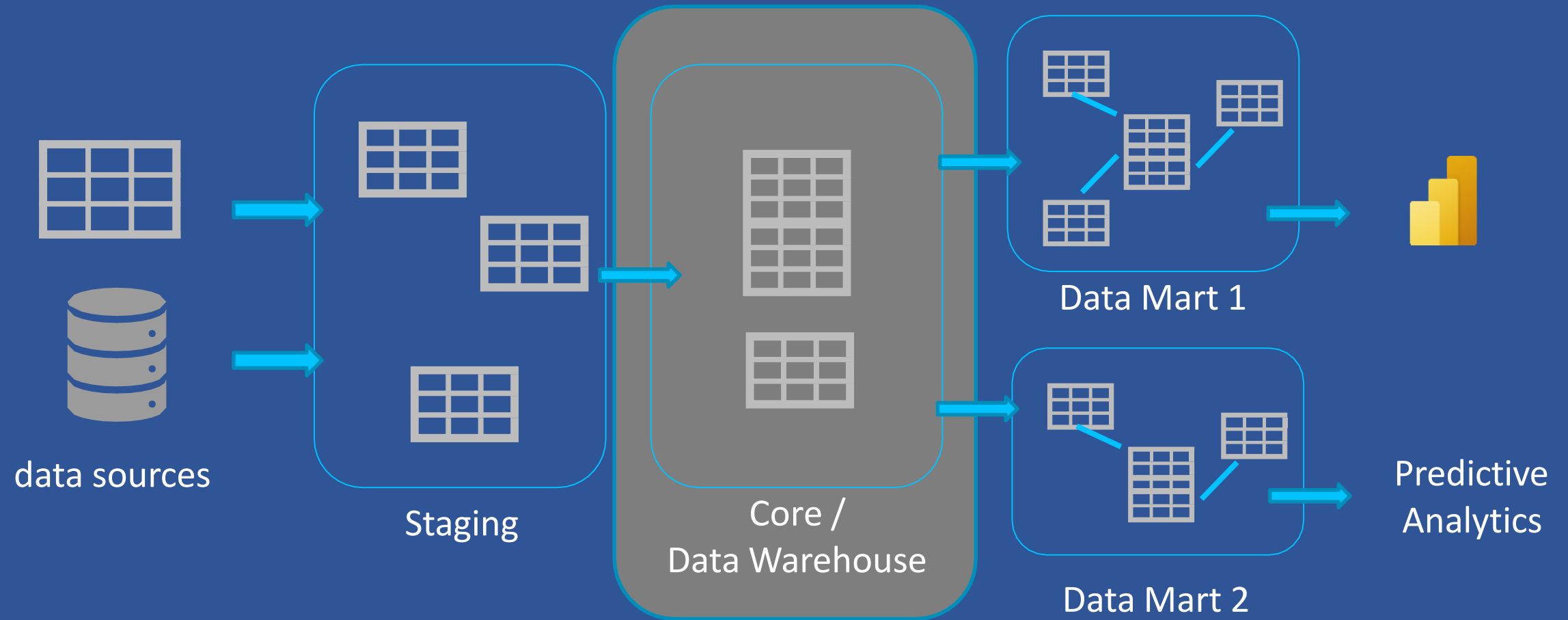
Data Warehouse Layers



Data Warehouse Layers

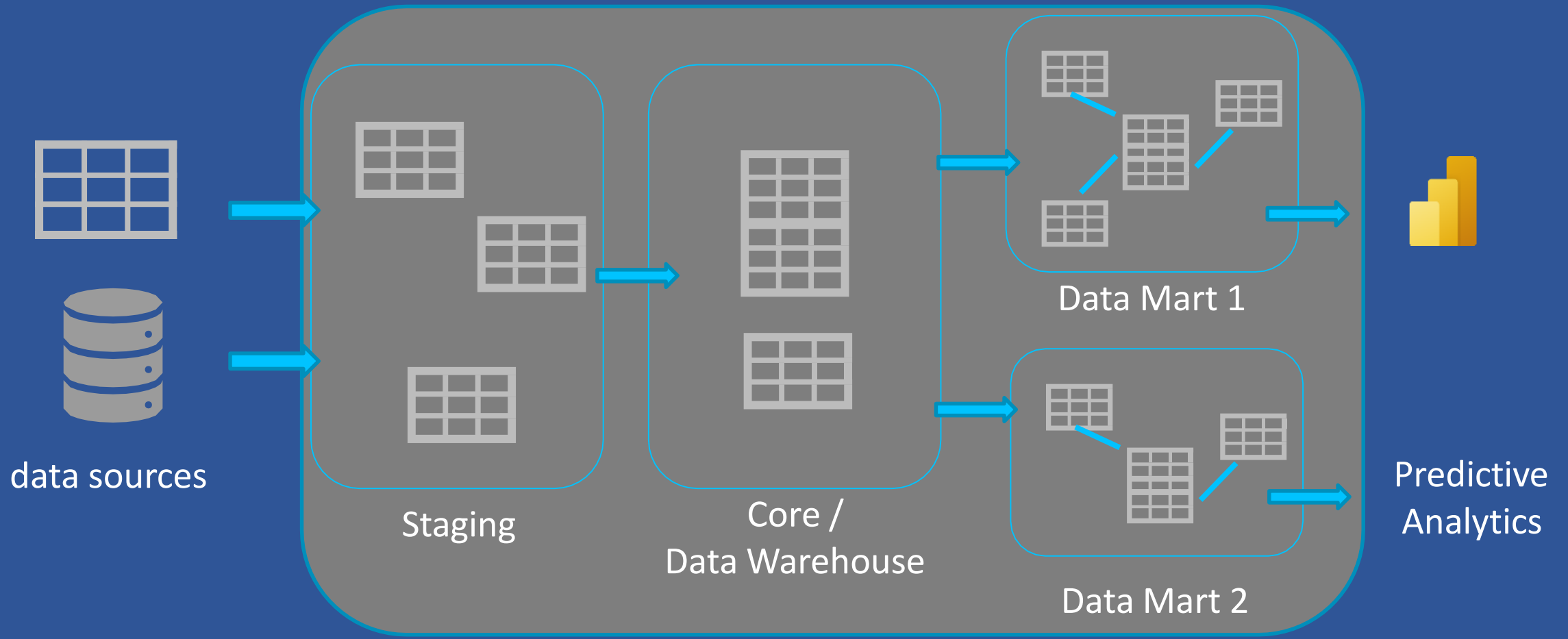


Data Warehouse Layers



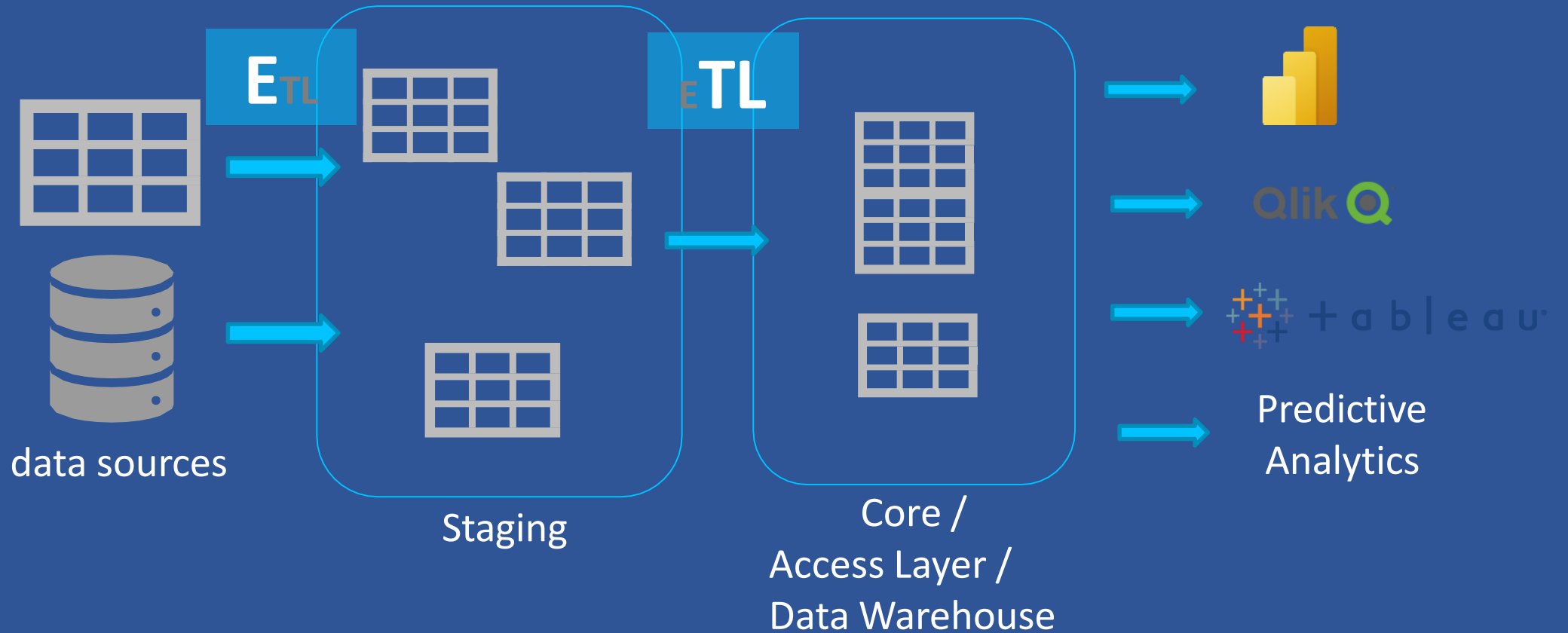
Data Warehouse Layers

Data Warehouse

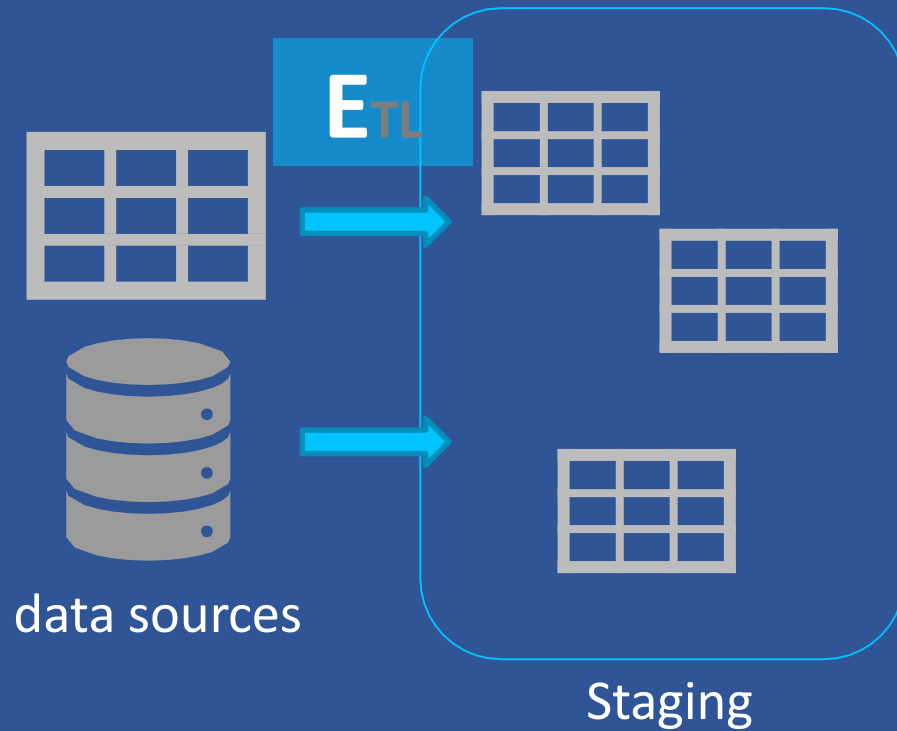


The Staging Area

Data Warehouse Layers

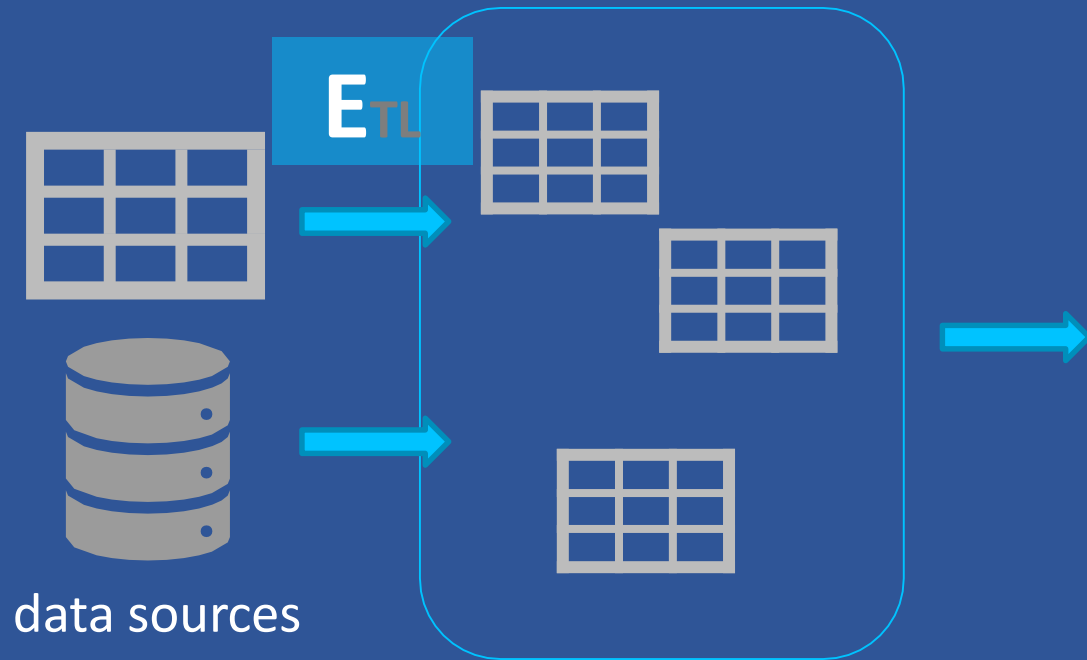


Data Warehouse Layers



- *"Short time on the source systems"*
- *"Quickly extract"*
- Move the data into relational database
- Start transformations from there

Data Warehouse Layers



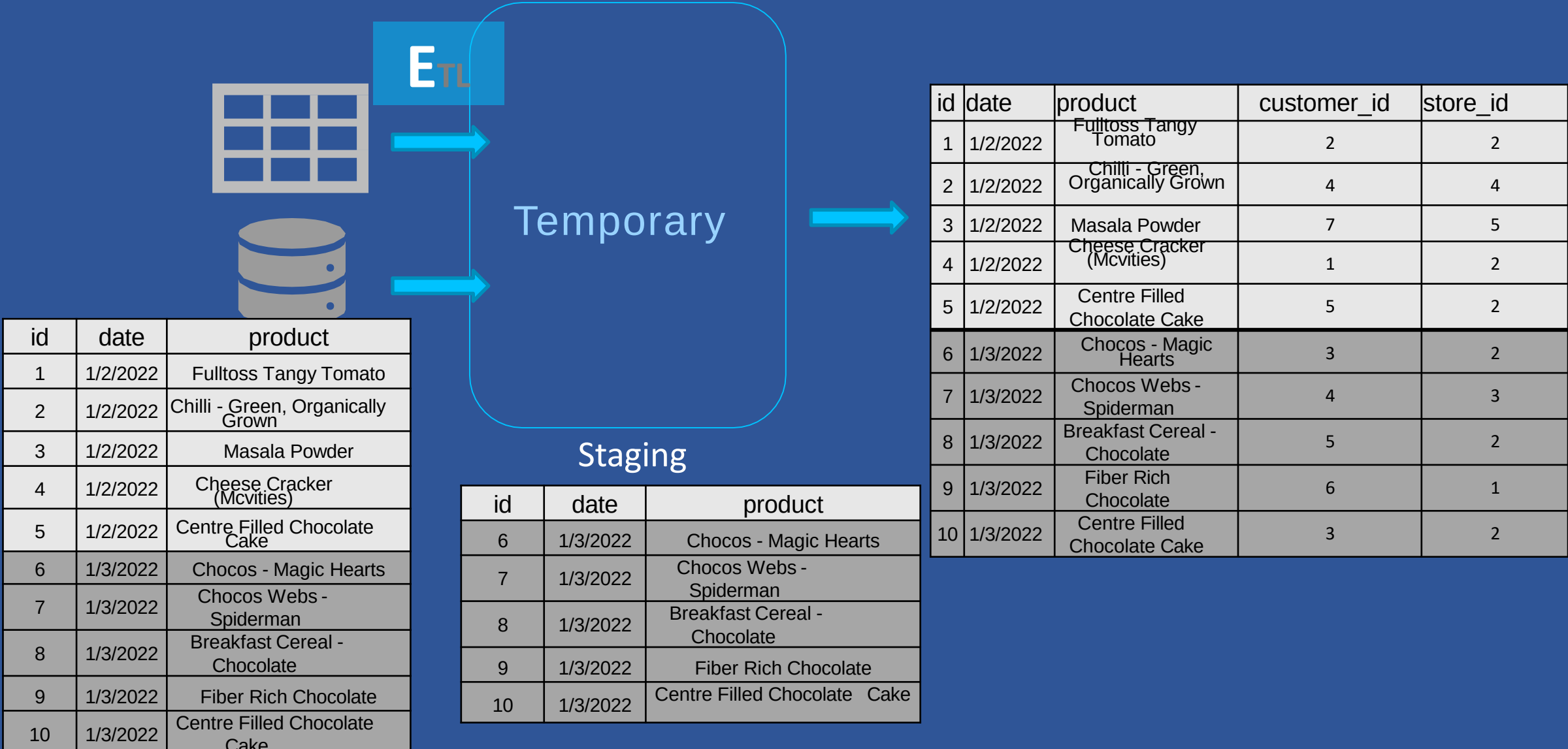
id	date	product	customer_id	store_id
1	1/2/2022	Fulltoss Tangy Tomato	2	2
2	1/2/2022	Chilli - Green, Organically Grown	4	4
3	1/2/2022	Masala Powder	7	5
4	1/2/2022	Cheese Cracker (Mcvities)	1	2
5	1/2/2022	Centre Filled Chocolate Cake	5	2

id	date	product
1	1/2/2022	Fulltoss Tangy Tomato
2	1/2/2022	Chilli - Green, Organically Grown
3	1/2/2022	Masala Powder
4	1/2/2022	Cheese Cracker (Mcvities)
5	1/2/2022	Centre Filled Chocolate Cake

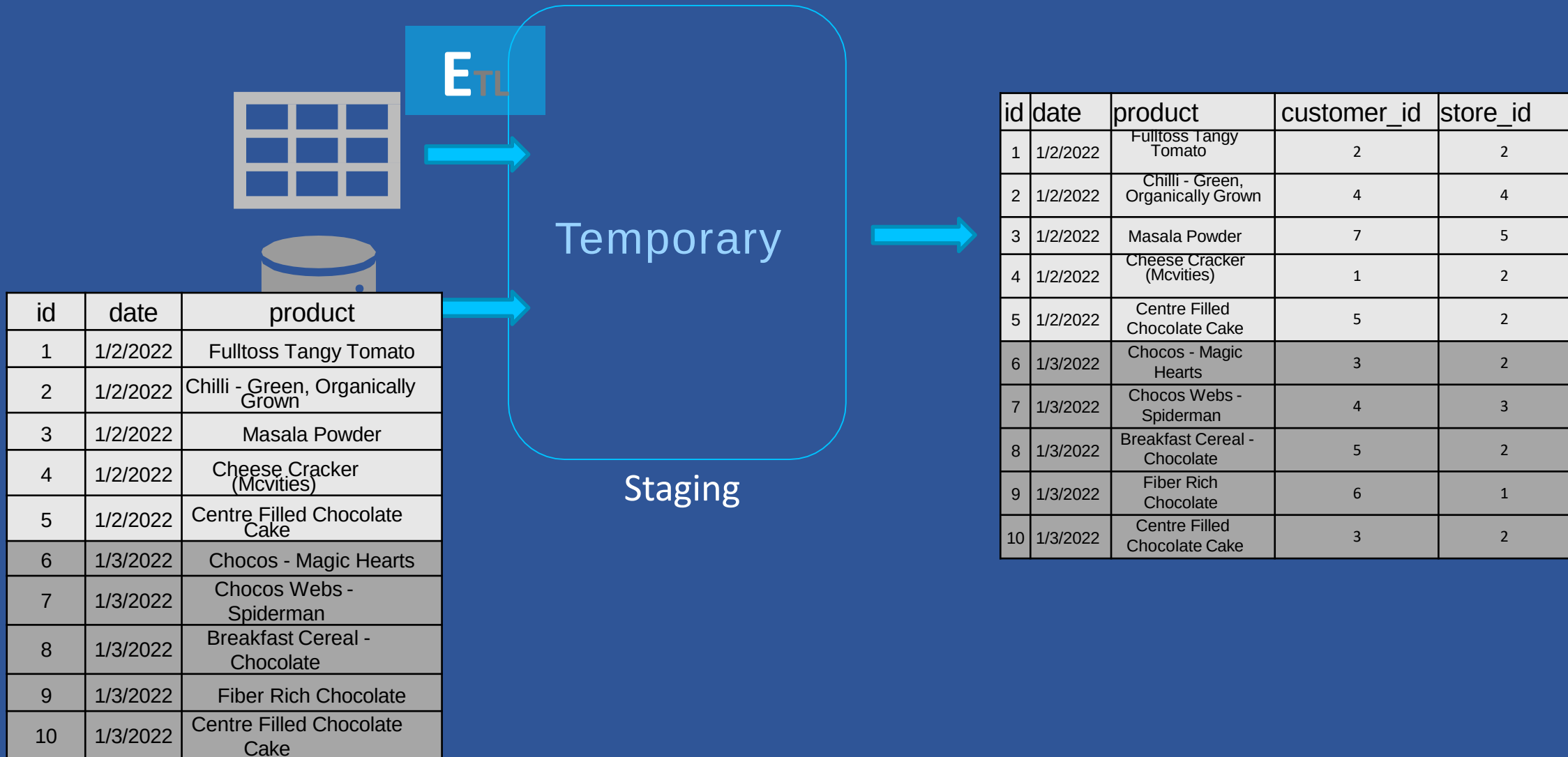
Staging

id	date	product
1	1/2/2022	Fulltoss Tangy Tomato
2	1/2/2022	Chilli - Green, Organically Grown
3	1/2/2022	Masala Powder
4	1/2/2022	Cheese Cracker (Mcvities)
5	1/2/2022	Centre Filled Chocolate Cake

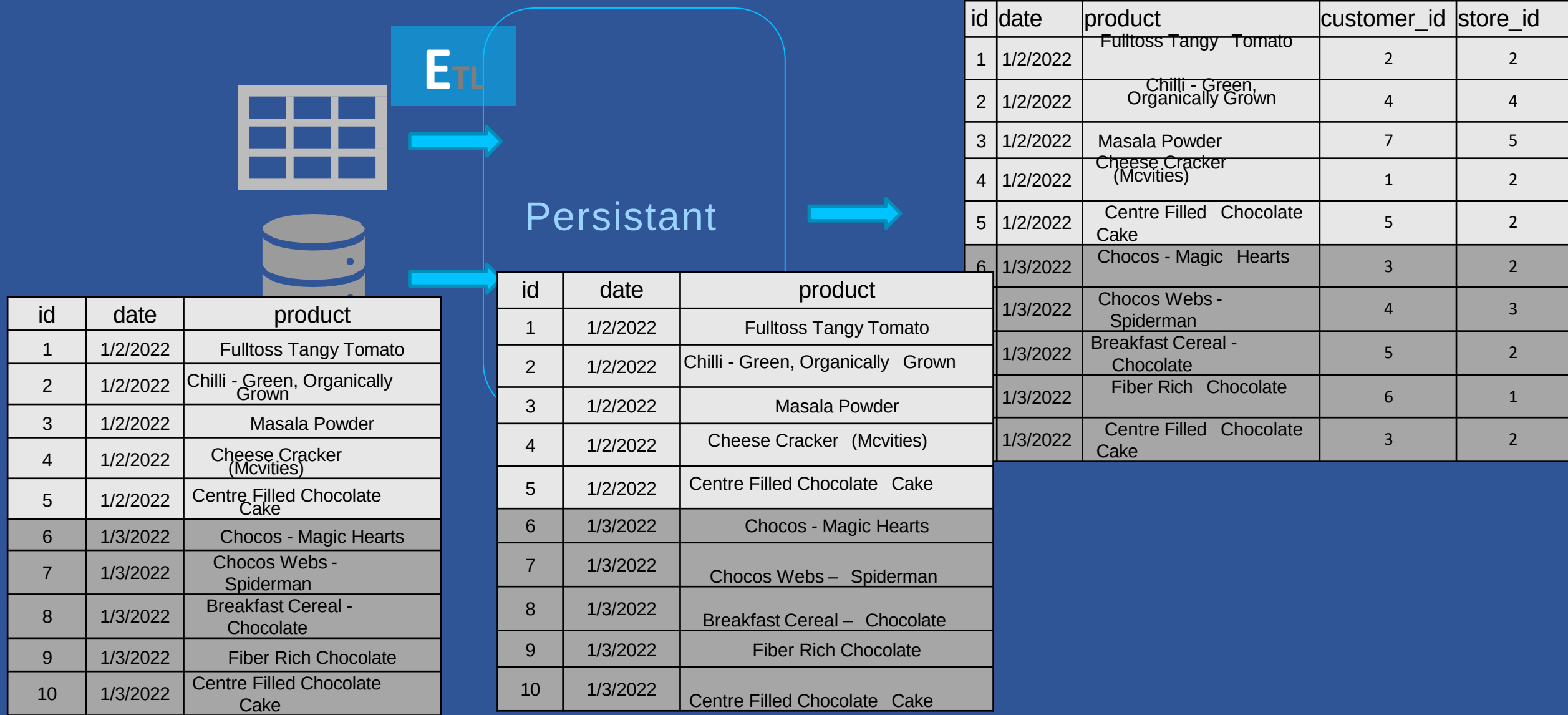
Data Warehouse Layers



Data Warehouse Layers



Data Warehouse Layers

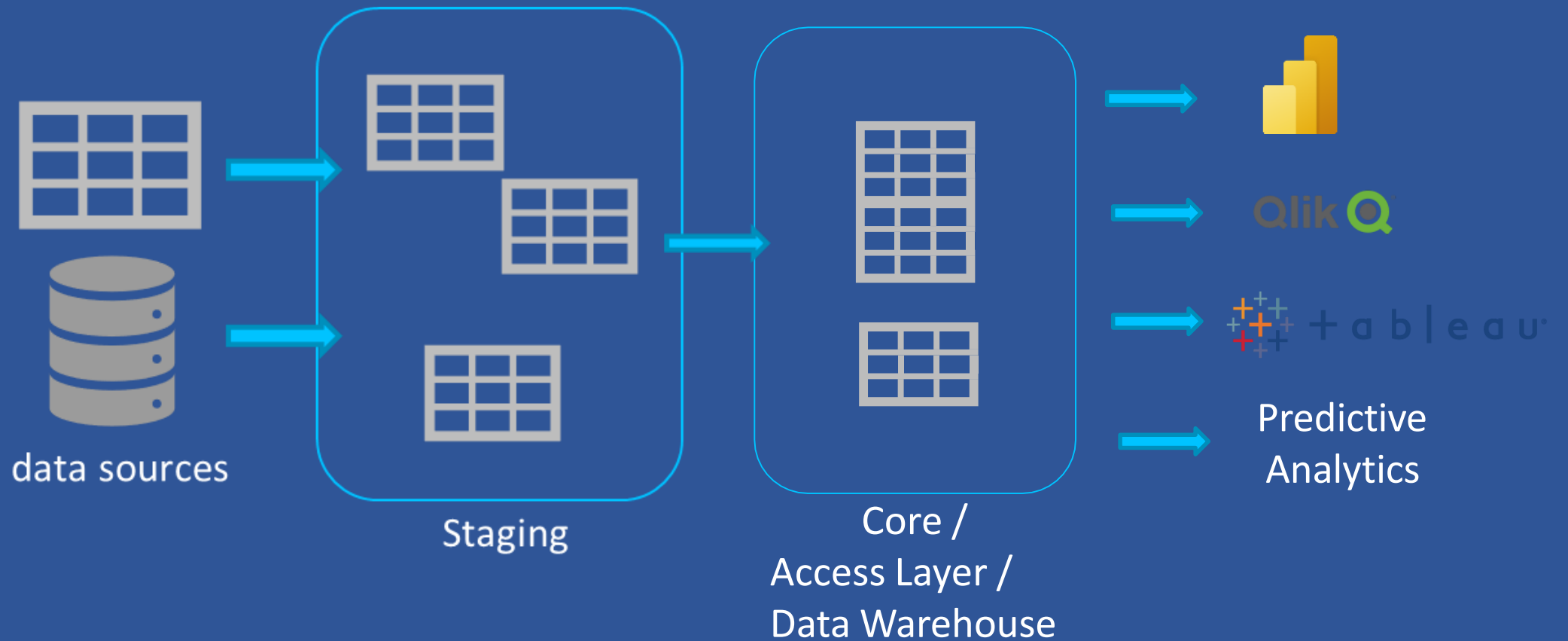


The Staging Layer

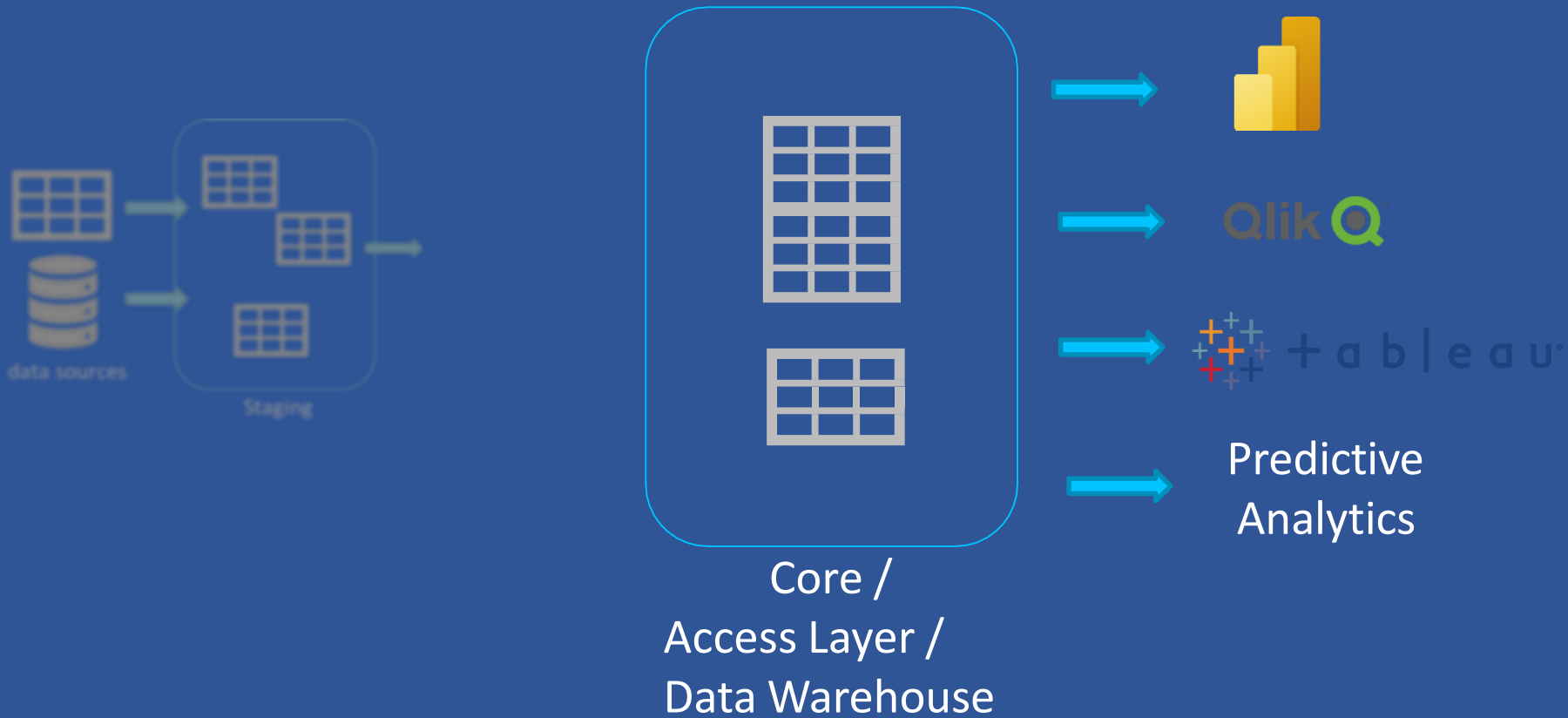
- ✓ Staging Layer is the landing zone extracted data
- ✓ Data in tables and on a separate database
- ✓ As little "touching" as possible
- ✓ We don't charge the source systems
- ✓ Temporary or Persistent Staging Layers

Data Marts

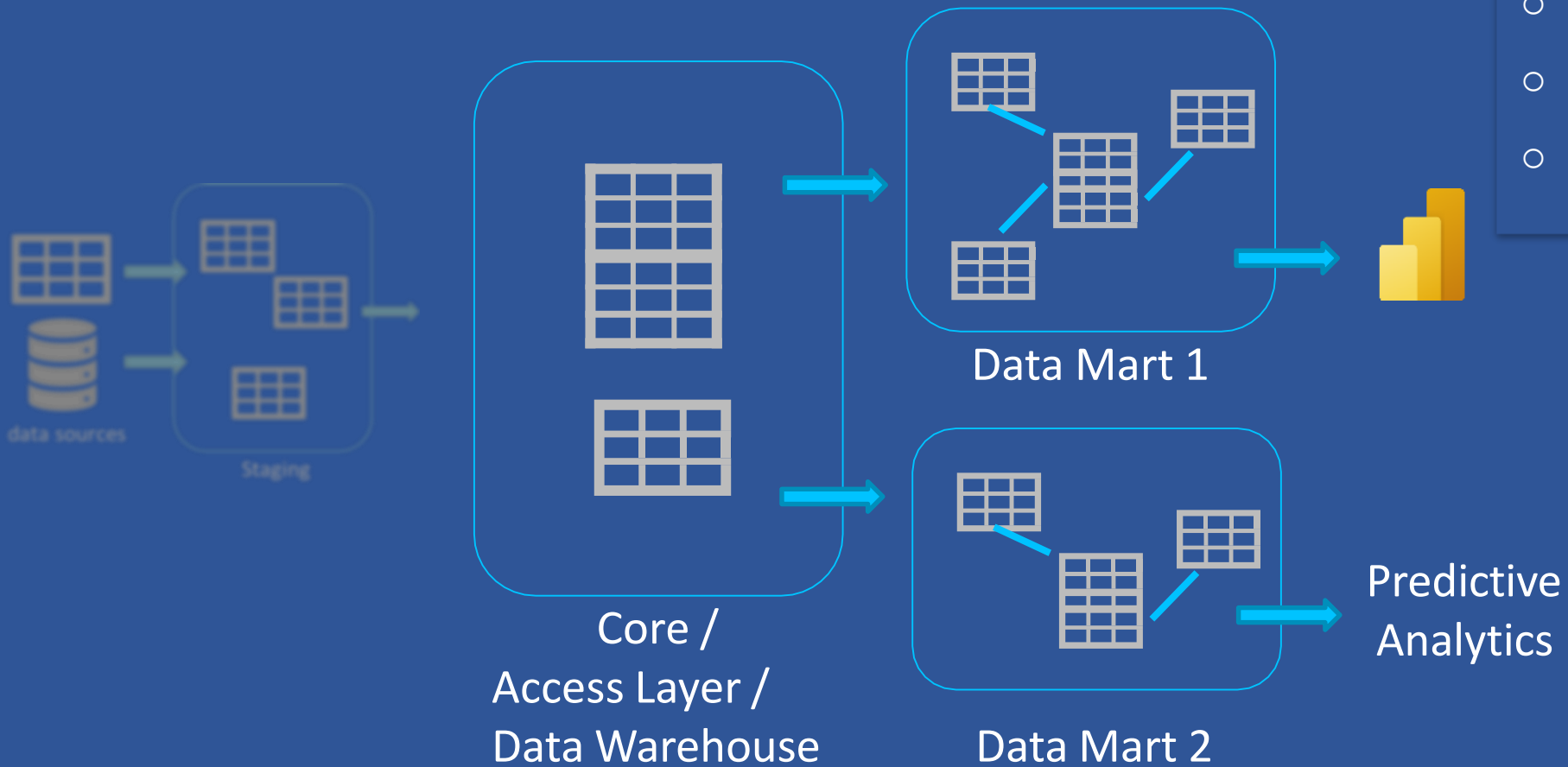
Data Warehouse Layers



Data Warehouse Layers

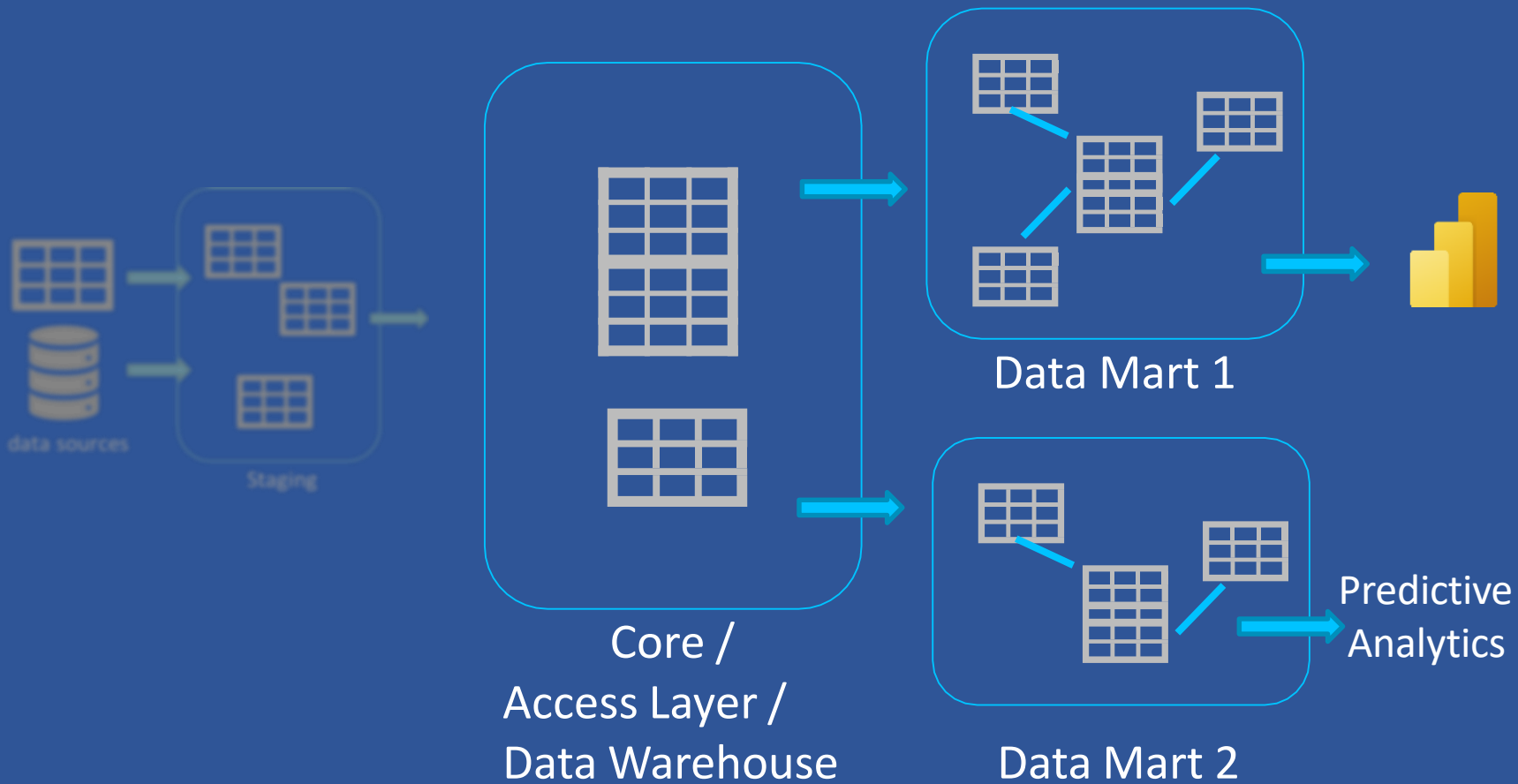


Data Warehouse Layers



- *Subset of a DWH*
- *Dimensional Model*
- *Can be further aggregated*

Data Warehouse Layers



- Subset of a DWH
- Dimensional Model
- Can be further aggregated

- *Usability + Acceptance*
- *Performance*

- *Tools*
- *Departments*
- *Regions*
- *Use-cases*

Data Marts

✓ Data Mart = Small scale DWH?

⇒ Focus on the business problem

✓ Should you use a Data Mart or not?

⇒ Focus on the business problem

Relational Database

Relational Database



Relational database



Tables (relations)

id	date	product	customer_id
1	1/2/2022	Fultoss Tangy Tomato	2
2	1/2/2022	Chilli - Green, Organically Grown	2
3	1/2/2022	Masala Powder	5
4	1/2/2022	Cheese Cracker (McVities)	1
5	1/2/2022	Centre Filled Chocolate Cake	5

```
SELECT      <column1>,  
            <column2>, ...  
FROM        <table_name>
```

Relational Database



Relational database



Tables (relations)

Primary key

Foreign key

id	date	product	customer_id
1	1/2/2022	Fulltoss Tangy Tomato	2
2	1/2/2022	Chilli - Green, Organically Grown	2
3	1/2/2022	Masala Powder	5
4	1/2/2022	Cheese Cracker (McVities)	1
5	1/2/2022	Centre Filled Chocolate Cake	5

id	name	city
1	Frank	New York
2	Sarah	Chicago
3	Sabrina	New Orleans
4	Maya	Los Angeles
5	Marc	Delas

Relational Database

```
SELECT    sales.id,  
          product,  
          customer_id,  
          name  
FROM      sales  
  
LEFT JOIN customer  
ON  customer_id = customer.id
```

Primary key

Foreign key

id	date	product	customer_id	name
1	/2/2022	Fulltoss Tangy Tomato	2	Sarah
2	/2/2022	Chilli - Green, Organically Grown	2	Sarah
3	/2/2022	Masala Powder	5	Marc
4	/2/2022	Cheese Cracker (McVities)	1	Frank
5	/2/2022	Centre Filled Chocolate Cake	5	Marc

id	name	city
1	Frank	New York
2	Sarah	Chicago
3	Sabrina	New Orleans
4	Maya	Los Angeles
5	Marc	Delas

Relational Database

- 70s to 90s building logic & improving performance
- Operational systems: 1 table
- OLAP / Analysis: multiple tables (context)

Rise of RD => rise of OLAP / DWH

Primary key

Foreign key

id	date	product	customer_id	name
1	/2/2022	Fulltoss Tangy Tomato	2	Sarah
2	/2/2022	Chilli - Green, Organically Grown	2	Sarah
3	/2/2022	Masala Powder	5	Marc
4	/2/2022	Cheese Cracker (McVities)	1	Frank
5	/2/2022	Centre Filled Chocolate Cake	5	Marc

id	name	city
1	Frank	New York
2	Sarah	Chicago
3	Sabrina	New Orleans
4	Maya	Los Angeles
5	Marc	Delas

Relational Database

- Relational database management system (RDMS)

Oracle

Microsoft SQL Server

PostgreSQL

MySQL

Amazon Relational Database Service (RDS)

Azure SQL databases

(Snowflake)

Primary key

Foreign key

id	date	product	customer_id	name
1	/2/2022	Fulltoss Tangy Tomato	2	Sarah
2	/2/2022	Chilli - Green, Organically Grown	2	Sarah
3	/2/2022	Masala Powder	5	Marc
4	/2/2022	Cheese Cracker (McVities)	1	Frank
5	/2/2022	Centre Filled Chocolate Cake	5	Marc

id	name	city
1	Frank	New York
2	Sarah	Chicago
3	Sabrina	New Orleans
4	Maya	Los Angeles
5	Marc	Delas

In Memory Database

In Memory Database

- ✓ Highly optimized for query performance
- ✓ Good for Analytics / High query volume
- ✓ Usually used for data marts
- ✓ Relational and non-relational

In Memory Database



Traditional database



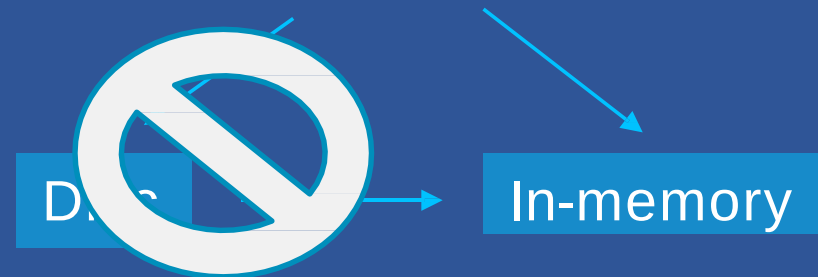
In Memory Database



Traditional database



In-memory database



- *columnar storage,*
- *parallel query plans,*
- *and other techniques*

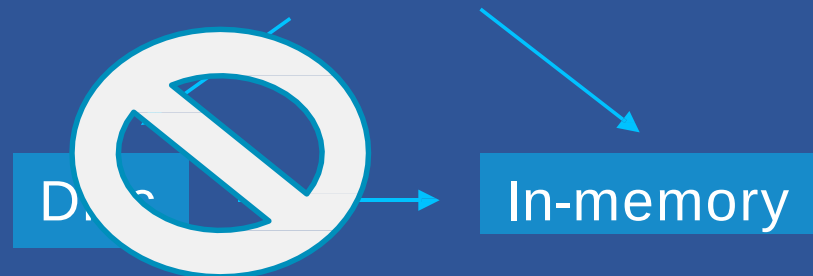
In Memory Database

- *Durability*: Lose all information when device loses power or is reset
- Durability added through snapshots / images
- Cost-factor
- Traditional DBs also trying reduce usage of disc



- *columnar storage,*
- *parallel query plans,*
- *and other techniques*

In-memory database



Products in RDMS context

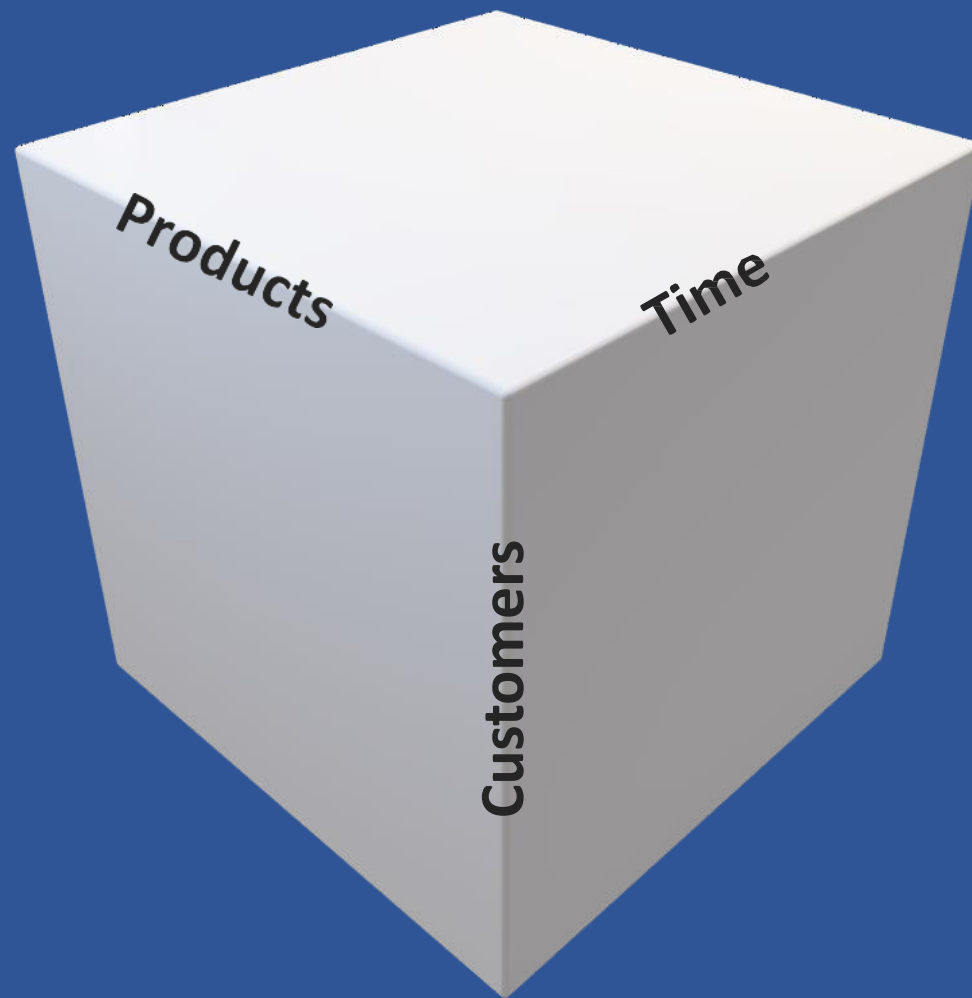
- ✓ SAP HANA
- ✓ MS SQL Server In-Memory Tables
- ✓ Oracle In-Memory
- ✓ Amazon MemoryDB

OLAP Cubes

OLAP Cube

- ✓ Traditional DWH based on relational DBMS (ROLAP)
- ✓ Data is organized non-relational in Cube (MOLAP)
Cube = Multidimensional dataset
- ✓ Arrays instead of tables
- ✓ Main reason to use: Fast query performance
- ✓ Works well with many BI solutions

OLAP Cube

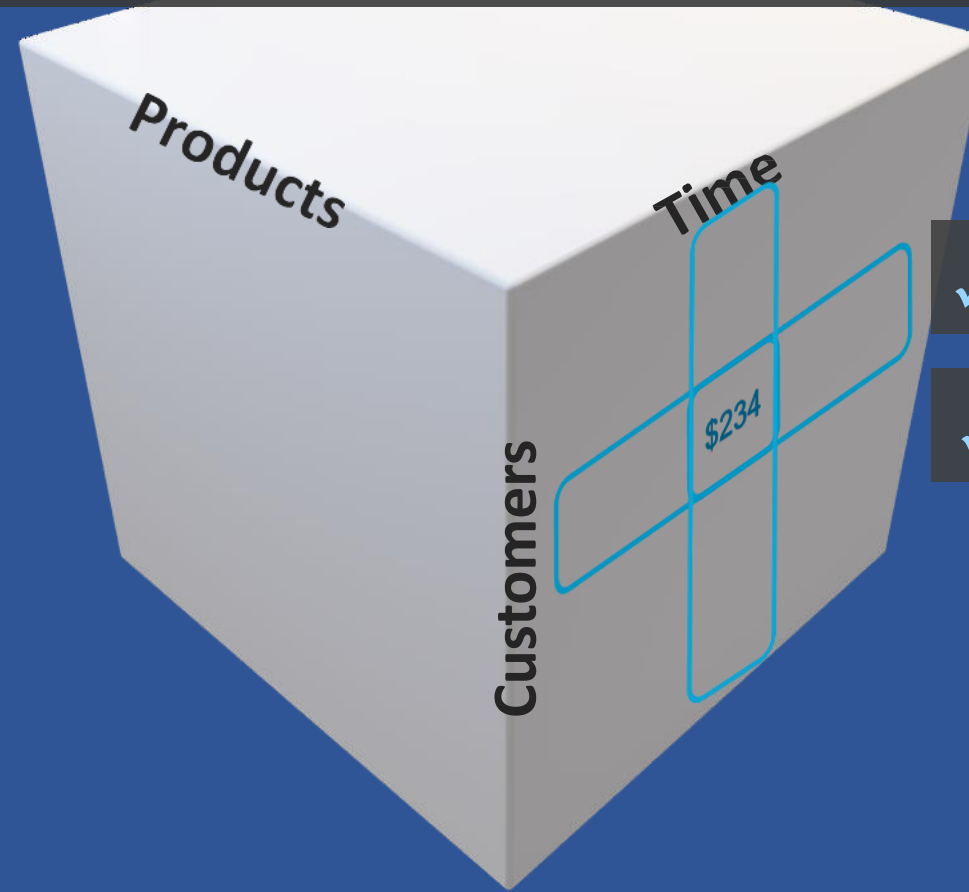


OLAP Cube

✓ Precalculated (aggregated values)

✓ High performance

✓ Interactive tools to
drill / slice & dice



✓ MDX

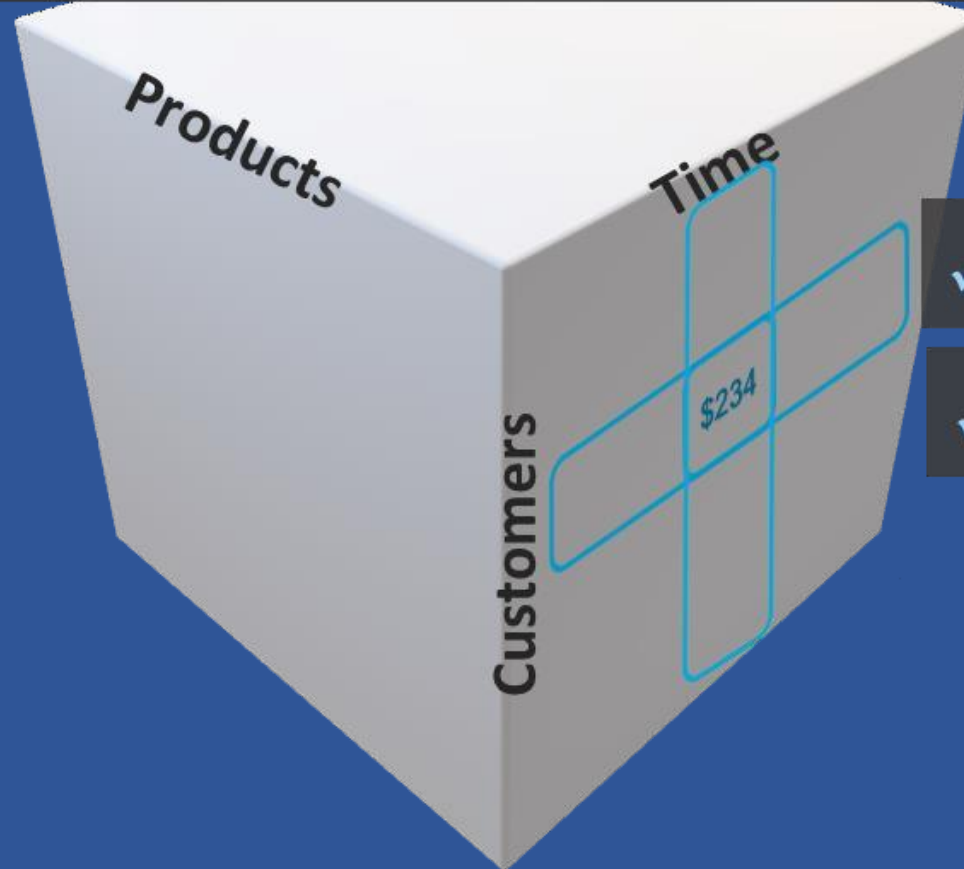
✓ Multidimensional DBs

Benefits

✓ Precalculated (aggregated values)

✓ High performance

✓ Interactive tools to
drill / slice & dice



✓ MDX

✓ Multidimensional DBs

Recommendation

- ✓ Built for a specific use-case (as data marts in general)
- ✓ More efficient & less complex with separate data marts
- ✓ Good for interactive queries with hierarchies
- ✓ Optional after star schema is built in relational DB

Alternatives

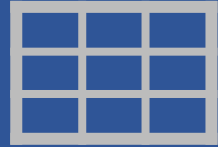
- ✓ Less important today with advancement of hardware
- ✓ Alternatives:
 - Tabular models (SSAS)
 - ROLAP
 - columnar storage

ODS (Operational Data Storage)

ODS

- ✓ Sometimes a little bit confusing
- ✓ Different understandings / definitions

ODS



Other data sources



Sales data



CRM system



Operational decision making

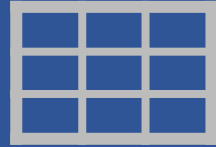


ODS

ODS

- ✓ No need for long history
- ✓ Needs to be very current or real-time

ODS



Other data sources

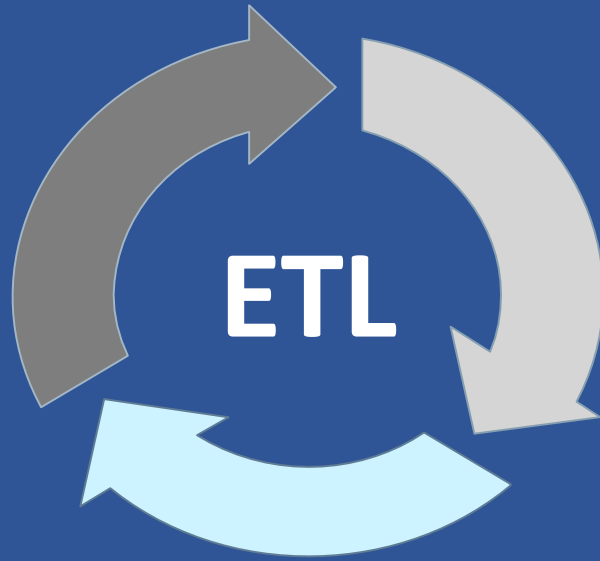


Sales data



CRM system

✓ (Near) real-time

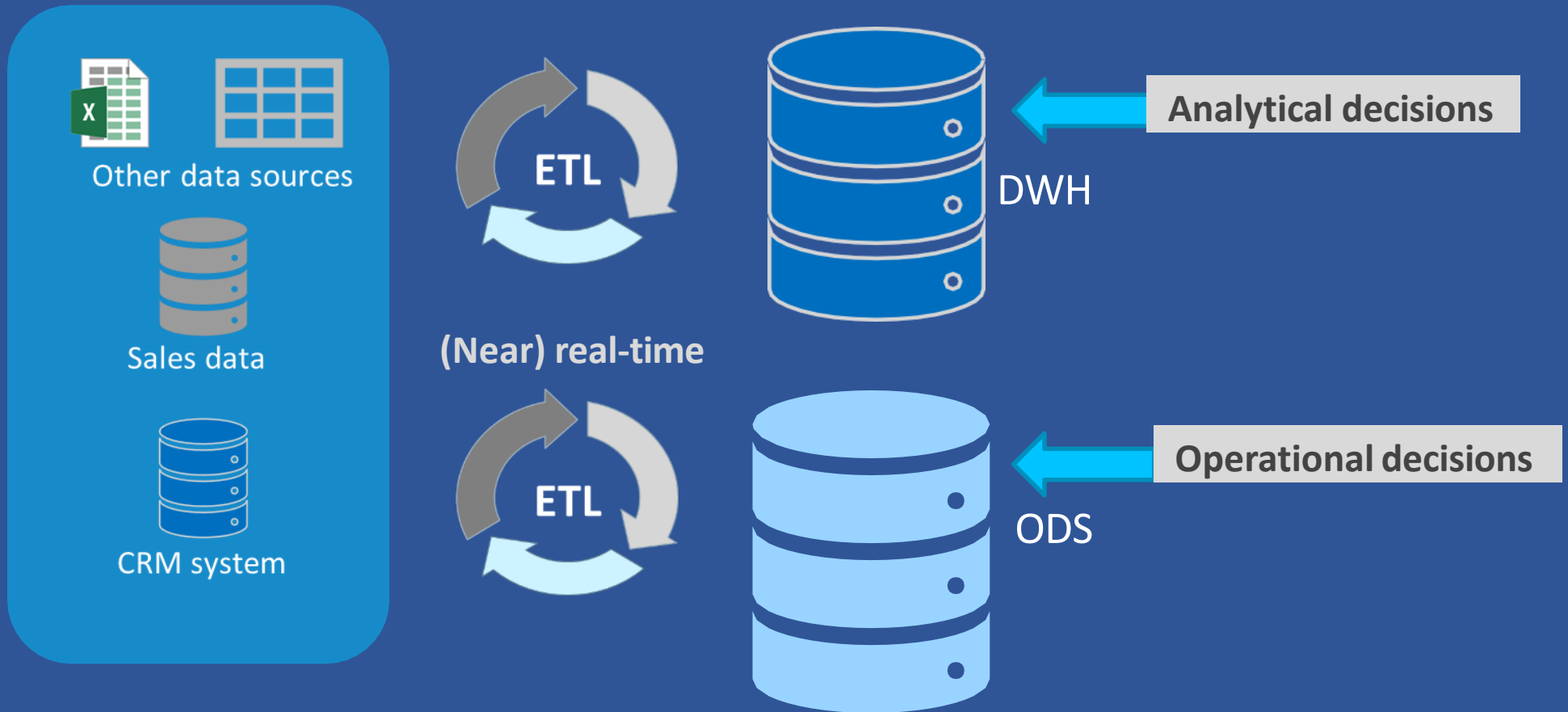


ODS

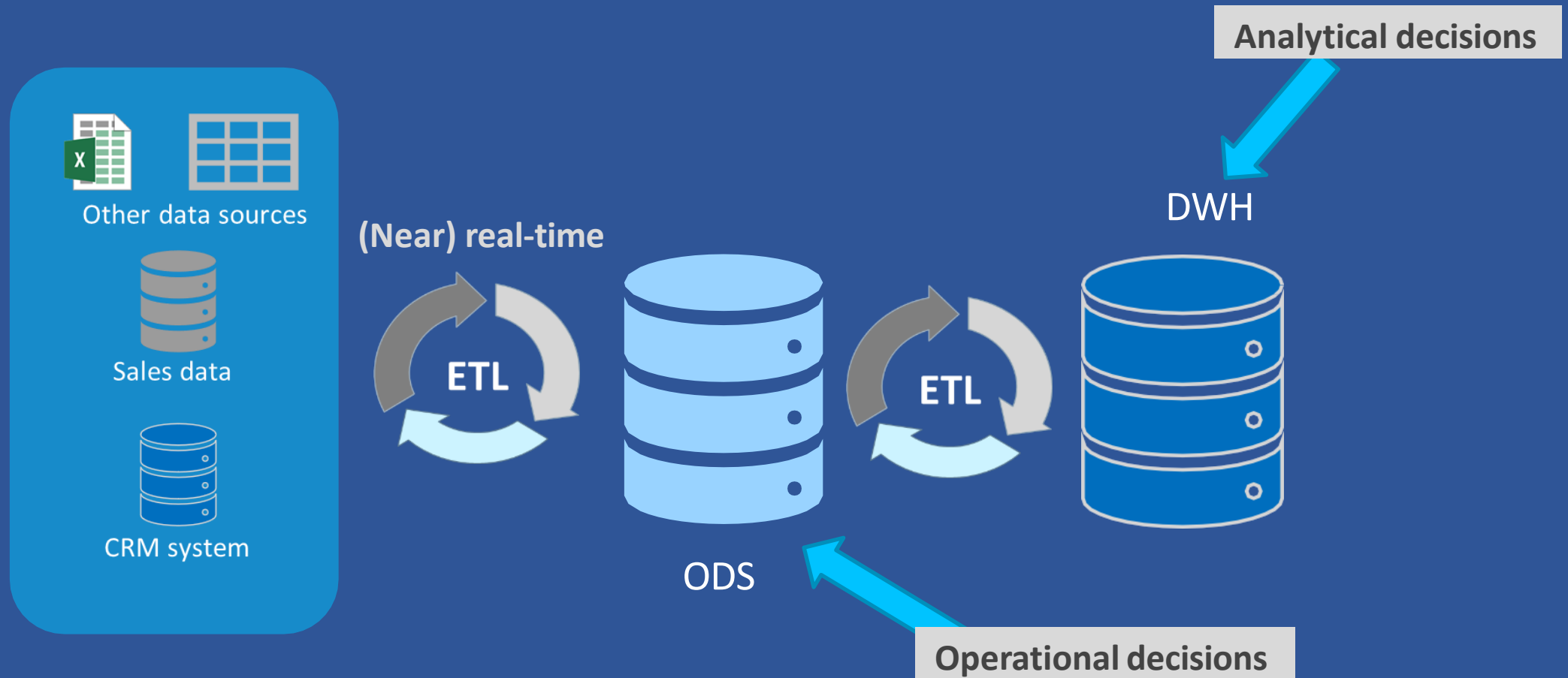
id	customer	total
1	Sarah	\$334
2	Frank	\$4234
3	Thomas	\$544
4	Angela	\$4332
5	Kate	\$460

✓ Current state (Update logic)

ODS - Parallel



ODS - Sequential

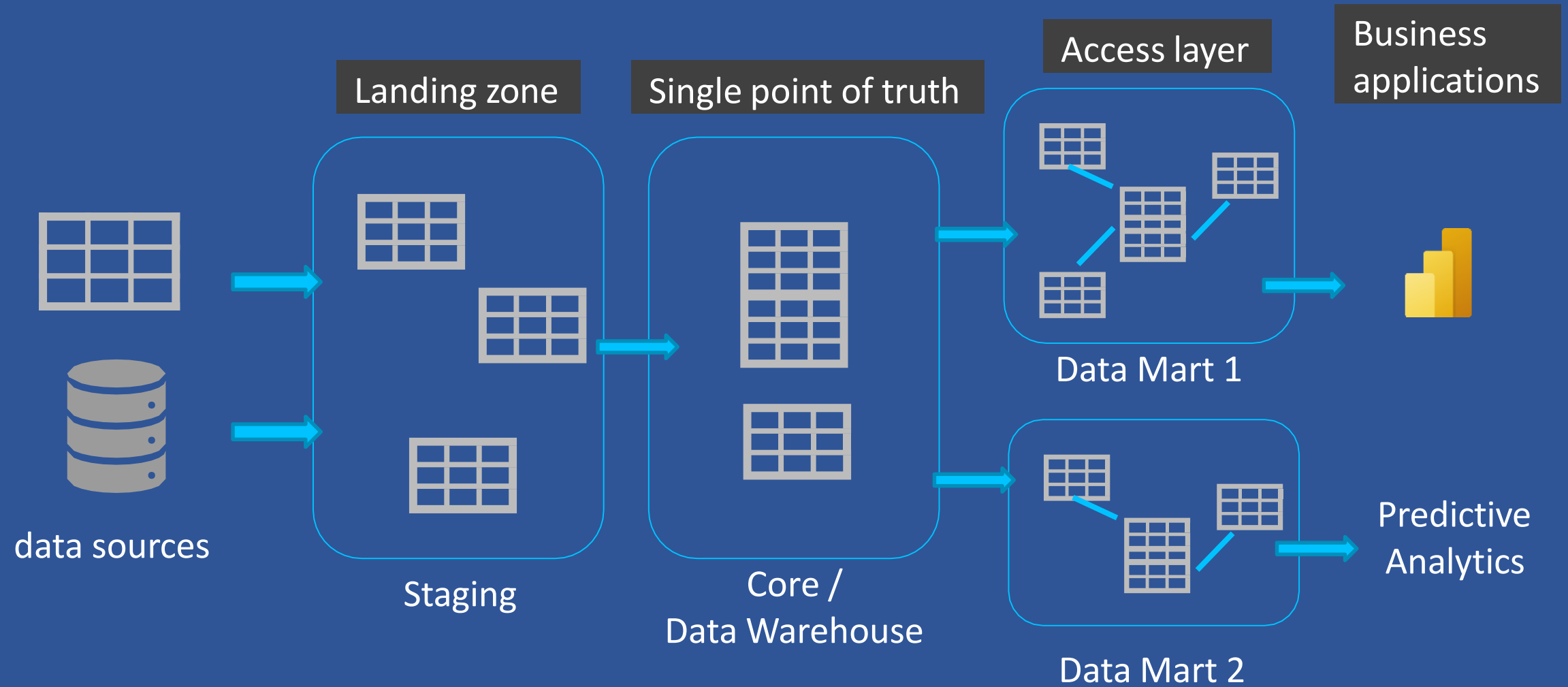


ODS

- ✓ **Getting less relevant**
 - ✓ Better performance (Faster ETL / DBs)
 - ✓ Big data technologies (very fast / real-time)
- ✓ **Don't get hung up with terminology!**

Let us Deliberate and Summarize

Data Warehouse Layers



The different layers

Staging

- ✓ Landing zone
- ✓ Minimal transformation
- ✓ "Stage" the data in tables

Core

- ✓ Always there
- ✓ Business Logic & Single Point of Truth
- ✓ Can be sometimes the access layer

Mart

- ✓ Access Layer
- ✓ Specific to one use-case
- ✓ Optimized for performance

What is dimensional modeling?

Dimensional Modelling

Dimensional modeling is widely accepted as the preferred technique for presenting analytic data because it addresses two simultaneous requirements:

- Deliver data that's understandable to the business users.
- Deliver fast query performance.

Dimensional Model simplifies database and it ensures that user can easily understand the data and allows software to navigate and deliver results quickly.

Dimensional Modelling

If someone describes business as “We sell products in various markets and measure our performance over time.”

A Dimensional Designer listens carefully and identifies product, market and time as dimensions.

Objective of a Dimensional Modeller should be to make initial design simple so that it remains simple. But if design is complex in the beginning then it will eventually be more complex.

Dimensional Modelling

- Dimensional model is instantiated in Relational database and they are difference than 3NF.
- 3NF is used to remove data redundancies and data is divided into many discrete entities.
- Dimensional model is different than 3NF based on degree of normalization.
- Both can be represented as E-R (Entity Relationship) Diagram.

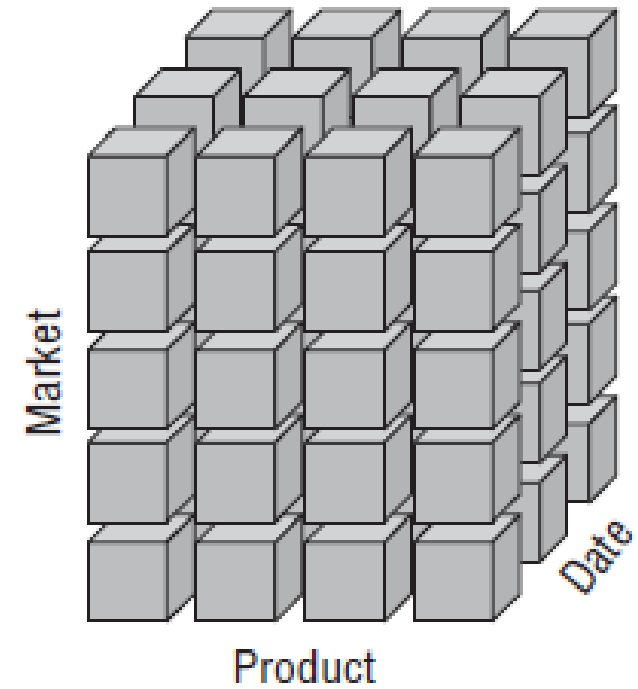
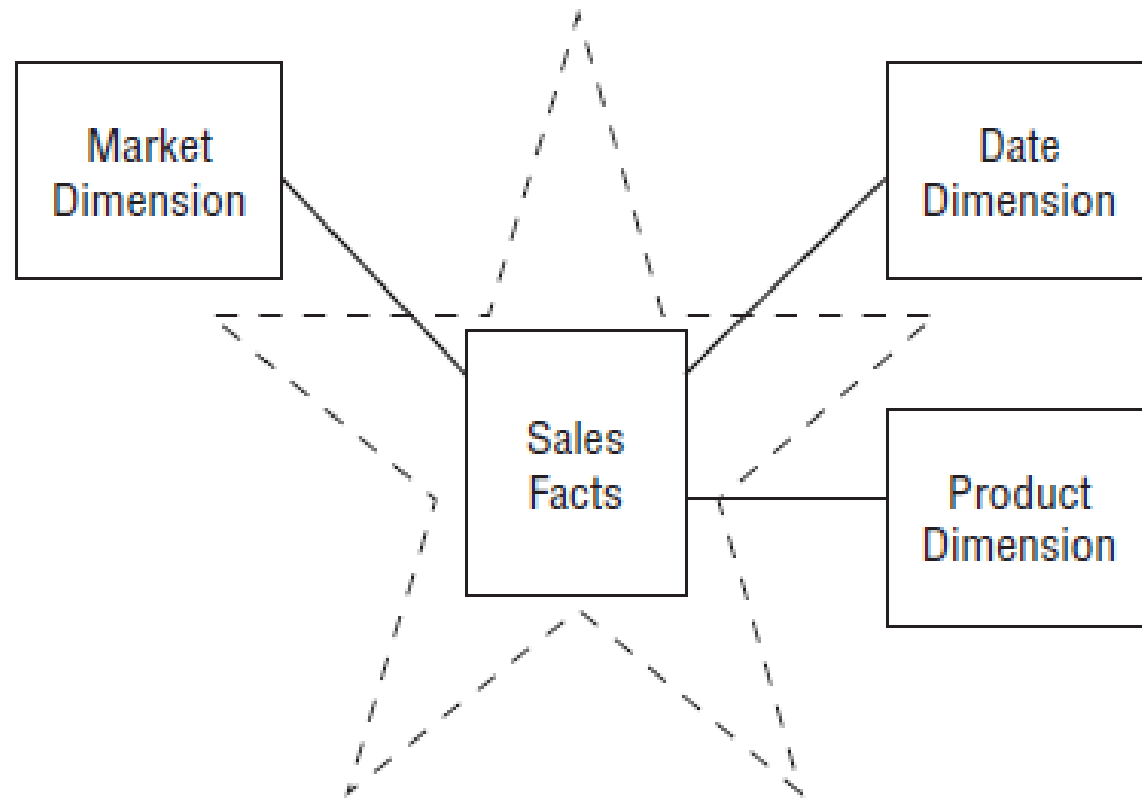
Dimensional Modelling

- Normalized 3NF structures are immensely useful in operational processing because an update or insert transaction touches the database in only one place.
- Normalized models are too complex for BI queries.
- The use of normalized modeling in the DW/BI presentation area defeats the intuitive and high-performance retrieval of data. Fortunately, dimensional modeling addresses the problem of overly complex schemas in the presentation area.

Star Schemas Versus OLAP Cubes

- Dimensional models implemented in relational database management systems are referred to as *star schemas* because of their resemblance to a star-like structure.
- Dimensional models implemented in multidimensional database environments are referred to as *online analytical processing (OLAP) cubes*,
- Both Star and Cube has common design with recognizable Dimensions.
- When data is loaded into an OLAP cube, it is stored and indexed using formats and techniques that are designed for dimensional data.
- Performance aggregations or precalculated summary tables are often created and managed by the OLAP cube engine. Consequently, cubes deliver superior query performance because of the precalculations, indexing strategies, and other optimizations.
- Business users can drill down or up by adding or removing attributes from their analyses with excellent performance without issuing new queries. OLAP cubes also provide more analytically robust functions that exceed those available with SQL.

Star Schemas Versus OLAP Cubes



Star Schemas Versus OLAP Cubes

A star schema hosted in a relational database is a good physical foundation for building an OLAP cube, and more stable basis for backup and recovery.

OLAP cubes are traditionally known been noted for extreme performance advantages over RDBMS, but that distinction has become less important with advances in computer hardware, like in-memory databases, and RDBMS software, such as columnar databases

OLAP cube data structures are more variable across different vendors than relational DBMSs - the final deployment details often depend on which OLAP software is chosen.

It is more difficult to port BI applications between different OLAP tools than to port BI applications across different relational databases.

OLAP cubes typically offer more sophisticated security options than RDBMSs, such as limiting access to detailed data but providing more open access to summary data.

OLAP cubes offer significantly richer analysis capabilities than RDBMSs, which are saddled by the constraints of SQL.

Star Schemas Versus OLAP Cubes

OLAP cubes typically offer more sophisticated security options than RDBMSs, such as limiting access to detailed data but providing more open access to summary data.

OLAP cubes offer significantly richer analysis capabilities than RDBMSs, which are saddled by the constraints of SQL. This may be the main justification for using an OLAP product.

OLAP cubes gracefully support slowly changing dimension type 2 changes, but cubes often need to be reprocessed partially or totally whenever data is overwritten using alternative slowly changing dimension techniques.

OLAP cubes gracefully support transaction and periodic snapshot fact tables, but do not handle accumulating snapshot fact tables because of the limitations on overwriting data described in the previous point.

Star Schemas Versus OLAP Cubes

OLAP cubes typically support complex ragged hierarchies of indeterminate depth, such as organization charts or bills of material, using native query syntax that is superior to the approaches required for RDBMSs.

OLAP cubes may impose detailed constraints on the structure of dimension keys that implement drill-down hierarchies compared to relational databases.

Some OLAP products do not enable dimensional roles or aliases, thus requiring separate physical dimensions to be defined.

Dimensional modeling

✓ Method of organizing data (in a data warehouse)

Dimensional modelling is a longstanding technique for making databases simple.

✓ Facts

- Measurement like profit

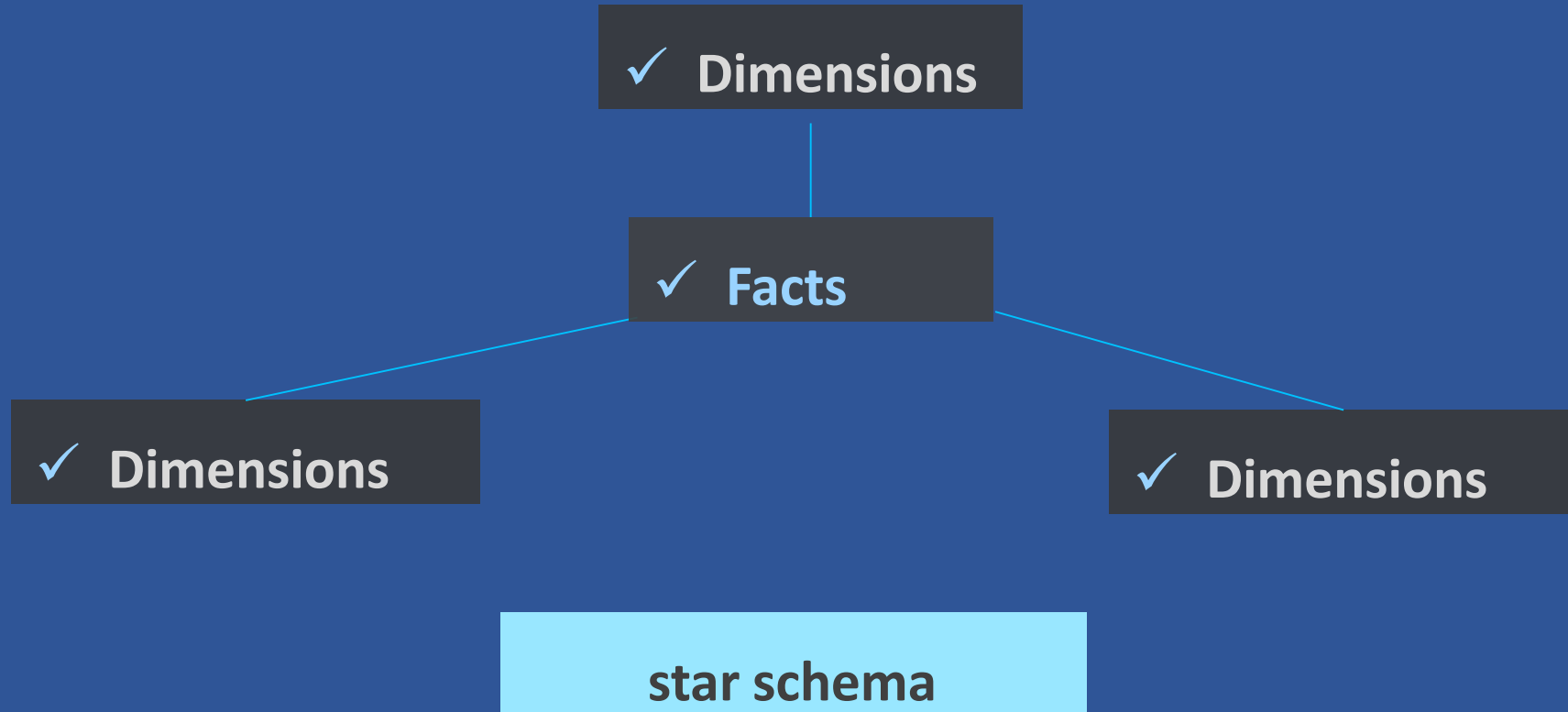
Profit by year

Profit by category

✓ Dimensions

- Context like category or period

Dimensional modeling



Dimensional modeling

- ✓ Unique technique of structuring data
- ✓ Commonly used in DWH
- ✓ Optimized for faster data retrieval
- ✓ Oriented around performance & usability
- ✓ Designed Reporting / OLAP

Why dimensional modeling?

Dimensional modeling

✓ Goal: Fast data retrieval

✓ Oriented around performance & usability

id	date	product	category	customer_id	name	profit
1	1/2/2022	Fulltoss Tangy Tomato	Vegetables	2	Sarah	\$23
2	1/2/2022	Chilli - Green, Organically Grown	Snacks	2	Sarah	\$12
3	1/2/2022	Masala Powder	Herbs	5	Marc	\$93
4	1/2/2022	Cheese Cracker (McVities)	Snacks	1	Frank	\$23
5	1/2/2022	Centre Filled Chocolate Cake	Snacks	5	Marc	\$21

Dimensional modeling

- ✓ Goal: Fast data retrieval
- ✓ Oriented around performance & usability

id	date	product	category	customer_id	name	profit
1	1/2/2022	Fultoss Tanga Tomato	Vegetables	2	Sarah	\$23
2	1/2/2022	Chilli - Green, Organically Grown	Snacks	2	Sarah	\$12
3	1/2/2022	Masala Powder	Herbs	5	Marc	\$93
4	1/2/2022	Cheese Cracker (McVities)	Snacks	1	Frank	\$23
5	1/2/2022	Centre Filled Chocolate Cake	Snacks	5	Marc	\$21

Dimensional modeling

- ✓ Goal: Fast data retrieval
- ✓ Oriented around performance & usability

id	date	product	category	customer_id	profit
1	1/2/2022	Fulltoss Tangy Tomato	Vegetables	2	\$23
2	1/2/2022	Chilli.- Green, Organically Grown	Snacks	2	\$12
3	1/2/2022	Masala Powder	Herbs	5	\$93
4	1/2/2022	Cheese Cracker (McVities)	Snacks	1	\$23
5	1/2/2022	Centre Filled Chocolate Cake	Snacks	5	\$21

Dimensional modeling

- ✓ Goal: Fast data retrieval
- ✓ Oriented around performance & usability

id	date	product	category	customer_id	profit
1	1/2/2022	Fulitoss Tangy Tomato	Vegetables	2	\$23
2	1/2/2022	Chilli - Green, Organically Grown	Snacks	2	\$12
3	1/2/2022	Masala Powder	Herbs	5	\$93
4	1/2/2022	Cheese Cracker (Mcvities)	Snacks	1	\$23
5	1/2/2022	Centre Filled Chocolate Cake	Snacks	5	\$21

Dimensional modeling

- ✓ Goal: Fast data retrieval
- ✓ Oriented around performance & usability

FK

id	date	product_id	customer_id	profit
1	1/2/2022	2	2	\$23
2	1/2/2022	5	2	\$12
3	1/2/2022	6	5	\$93
4	1/2/2022	23	1	\$23
5	1/2/2022	16	5	\$21

Profit Fact Table

PK

product_id	product	category
1	product 1	Vegetables
2	product 2	Snacks
3	product 3	Herbs
4	product 4	Snacks
5	product 5	Snacks

Product Dim

Dimensional modeling

- ✓ **Goal: Fast data retrieval**
- ✓ **Oriented around performance & usability**

id	date_id	product_id	customer_id	profit
1	20220102	2	2	\$23
2	20220102	5	2	\$12
3	20220102	6	5	\$93
4	20220102	23	1	\$23
5	20220102	16	5	\$21

Dimensional modeling

- ✓ Goal: Fast data retrieval
- ✓ Oriented around performance & usability

id	date_id	product_id	customer_id	profit
1	20220102	2	2	\$23
2	20220102	5	2	\$12
3	20220102	6	5	\$93
4	20220102	23	1	\$23
5	20220102	16	5	\$21

Profit Fact Table

date_id	weekday	month
20220102	Monday	January
20220103	Tuesday	January
20220104	Wednesday	January
20220105	Friday	January
20220106	Saturday	January

Date Dim

Dimensional modeling

- ✓ Goal: Fast data retrieval
- ✓ Oriented around performance & usability



Performance

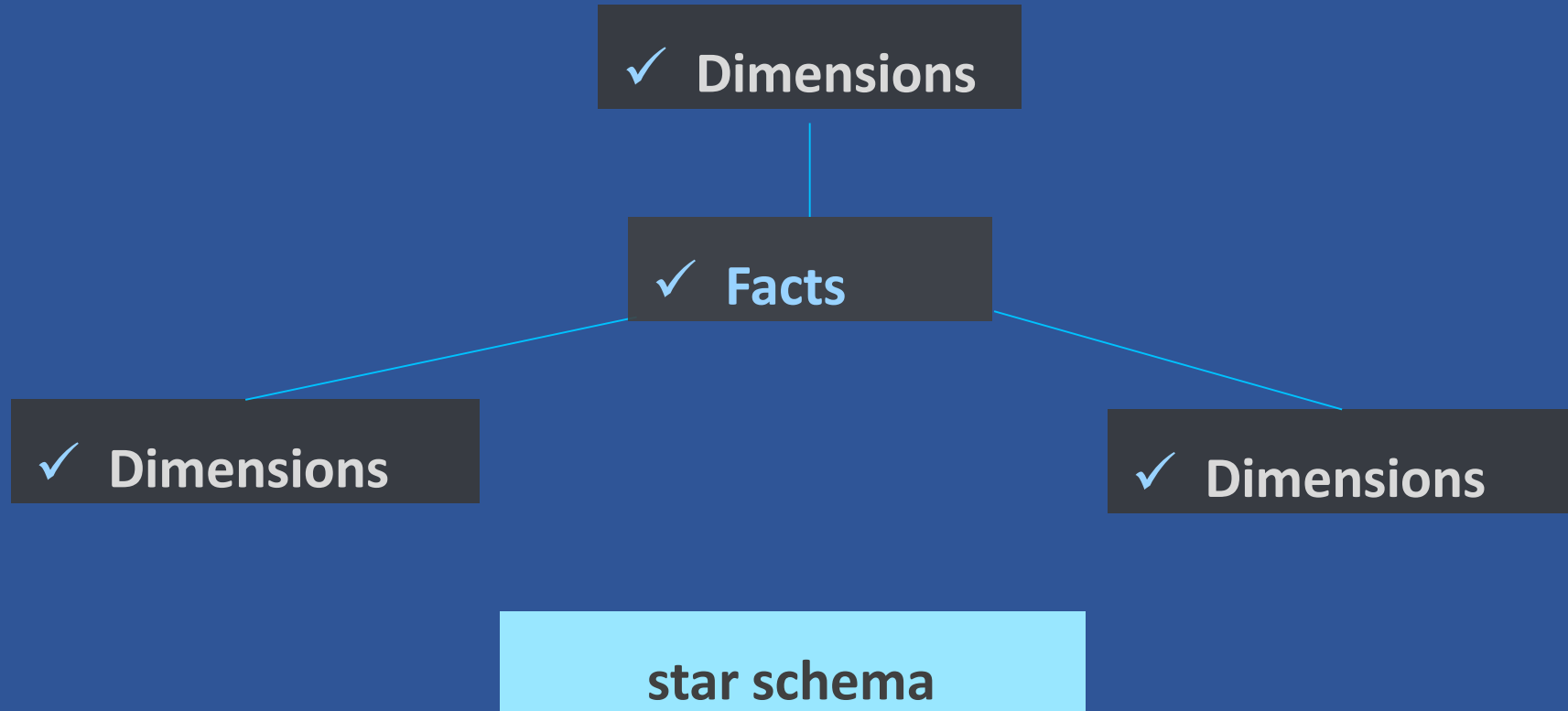
Usability



Preferred technique for data warehouse!

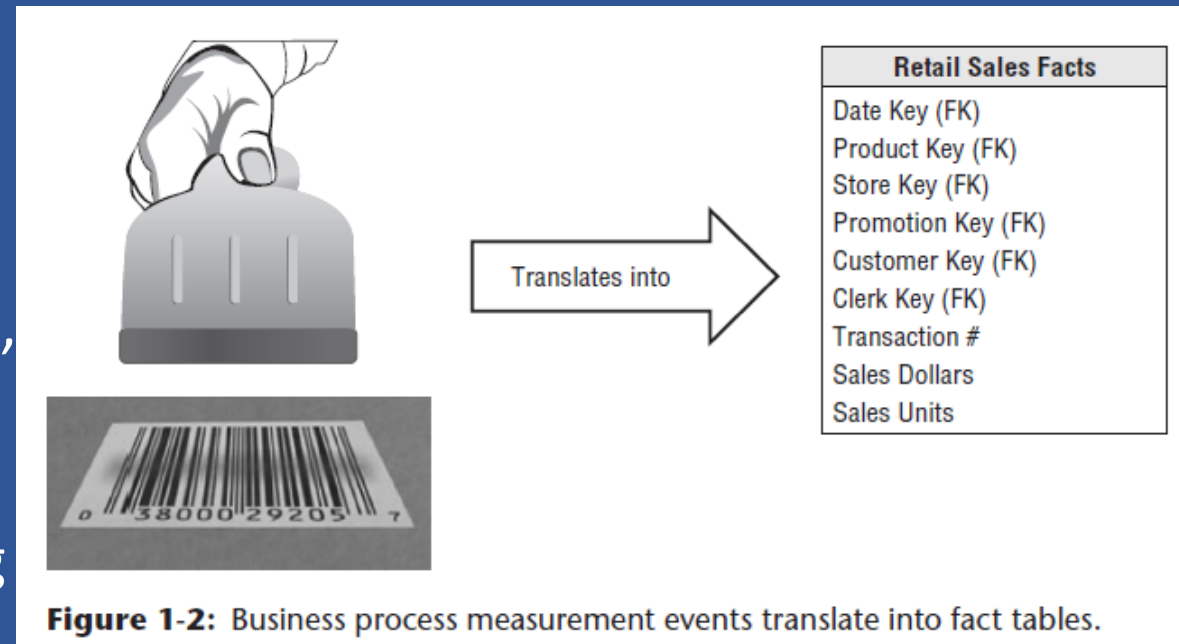
Facts

Facts



Fact

- The term *fact* represents a business measure. Imagine standing in the marketplace watching products being sold and writing down the unit quantity and dollar sales amount for each product in each sales transaction.
- These measurements are captured as products are scanned at the register
- Each row in a fact table corresponds to a measurement event. The data on each row is at a specific level of detail, referred to as the *grain*, such as one row per product sold on a sales transaction.
- One of the core tenets of dimensional modeling is that all the measurement rows in a fact table must be at the same grain.



Fact

- Fact table grains fall into one of three categories: transaction, periodic snapshot, and accumulating snapshot.
- All fact tables have two or more foreign keys that connect to the dimension tables' primary keys.

- *Foundation of DWH*
- *Key measurements*
- *Aggregated and analyzed*

Dim_Customer

customer_id
first name
last name
sex
city

✓ Dimensions

Usually...

- *Aggregatable (numerical values)*
- *Measureable vs. descriptive*
- *Event- or transactional data*
- *Date/time in a fact table*

✓ Facts

Sales

sales_id
product_id
customer_id
units
price

✓ Dimensions

Dim_Product

product_id
name
category
subcategory
dimensions

✓ Dimensions

Dim_Date

date_id
year
quarter
month
week
day
weekday
holiday_flag

Facts

- ✓ **Fact table: PK, FK & Facts**
- ✓ **Grain: Most atomic level facts are defined**

id	date_id	region_id	profit
1	20220102	1	\$23
2	20220102	2	\$12
3	20220102	2	\$93
4	20220102	3	\$23
5	20220102	16	\$21

- ✓ **Different types of facts**

Dimensions

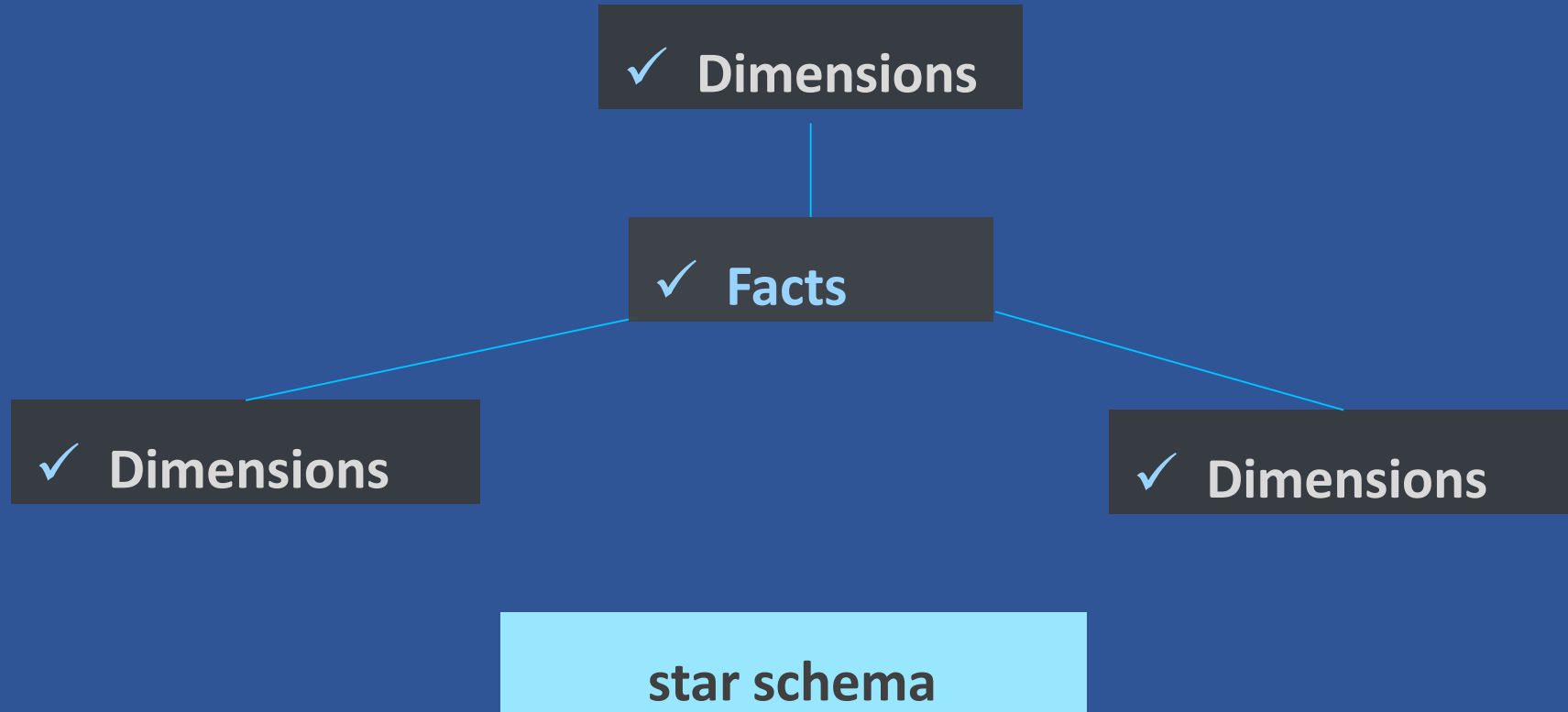
Dimensions

- The analytic power of the DW/BI environment is directly proportional to the quality and depth of the dimension attributes.
- Dimensions provide the entry points to the data, and the final labels and groupings on all DW/BI analyses.
- dimension tables often represent hierarchical relationships.
- For example, products roll up into brands and then into categories. For each
- row in the product dimension, you should store the associated brand and category description. The hierarchical descriptive information is stored redundantly in the spirit of ease of use and query performance.
- The hierarchical descriptive information is stored redundantly in the spirit of ease of use and query performance

Sample Dimensions

Product Key	Product Description	Brand Name	Category Name
1	PowerAll 20 oz	PowerClean	All Purpose Cleaner
2	PowerAll 32 oz	PowerClean	All Purpose Cleaner
3	PowerAll 48 oz	PowerClean	All Purpose Cleaner
4	PowerAll 64 oz	PowerClean	All Purpose Cleaner
5	ZipAll 20 oz	Zippy	All Purpose Cleaner
6	ZipAll 32 oz	Zippy	All Purpose Cleaner
7	ZipAll 48 oz	Zippy	All Purpose Cleaner
8	Shiny 20 oz	Clean Fast	Glass Cleaner
9	Shiny 32 oz	Clean Fast	Glass Cleaner
10	ZipGlass 20 oz	Zippy	Glass Cleaner
11	ZipGlass 32 oz	Zippy	Glass Cleaner

Dimensions



Dimensions

- *Categorizes facts*
- *Supportive & descriptive*
- *Filtering, Grouping & Labeling*

✓ **Dimensions**

Usually...

- *Non-Aggregatable*
- *Measureable vs. descriptive*
- *(More) static*

Dim_Product

product_id
name
category
subcategory
dimensions

S

✓ Facts

Sales

sales_id
product_id
customer_id
units
price

Dim_Date

date_id
year
quarter
month
week
day
weekday
holiday_flag

✓ Di

Dimensions

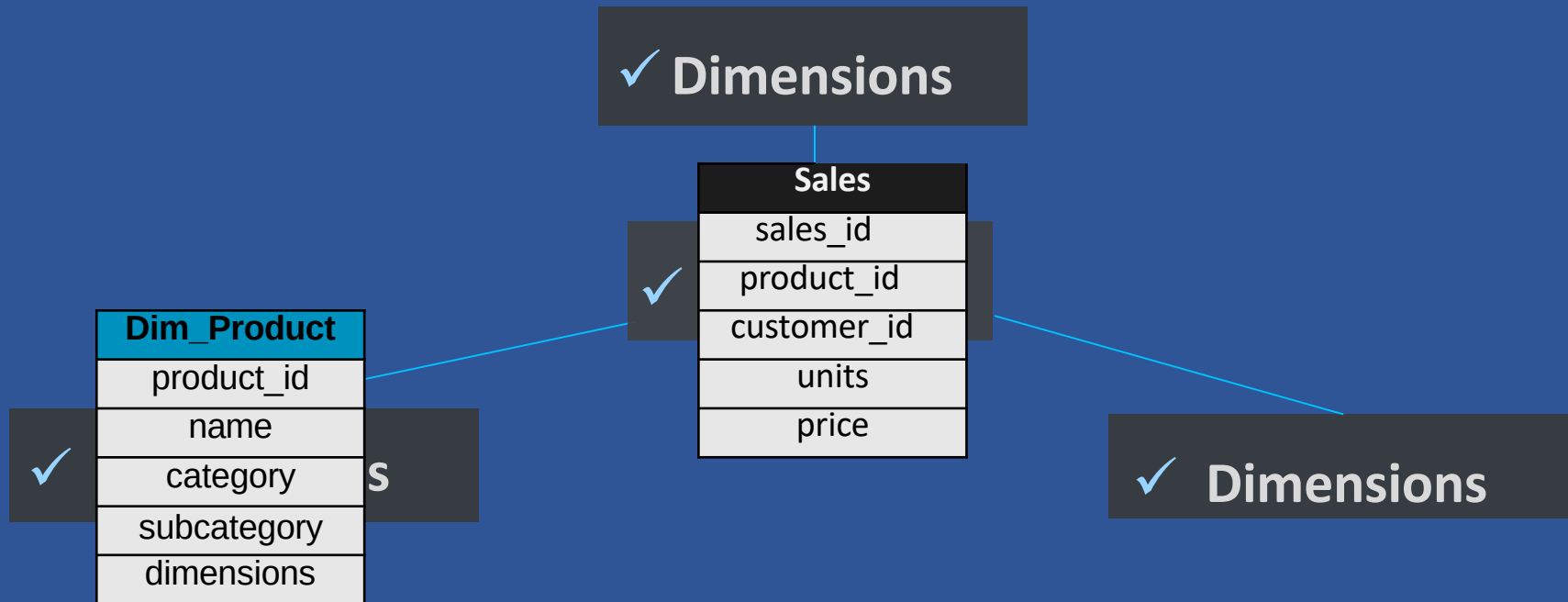
- ✓ Dimension table: PK, Dimension, (FK)
- ✓ People, products, places, time

customer_id	first_name	first_name	email
1	Mike	Miller	mike323@gmail.com
2	Sofia	Snider	snider_sof@gmail.com
2	Marco	Steadman	mstread23@gmail.com
3	Sarah	Griffith	sarah.griff@gmail.com
4	Jennifer	Lovell	jlovell@gmail.com

- ✓ Different types of dimension

Star schema

Star schema



Star schema

Normalized

- *Technique to avoid redundancy*
- *Minimizes storage*
- *Performance (write / update)*
- *Many tables*
- *Many joins necessary*

FK

sales_id	product_id	customer_id	units	price
1	3	23	1	2.99
2	5	13	1	1.99
3	2	7	2	3.49
4	3	16	1	2.29
5	3	13	5	1.49

✓ Facts

1:n

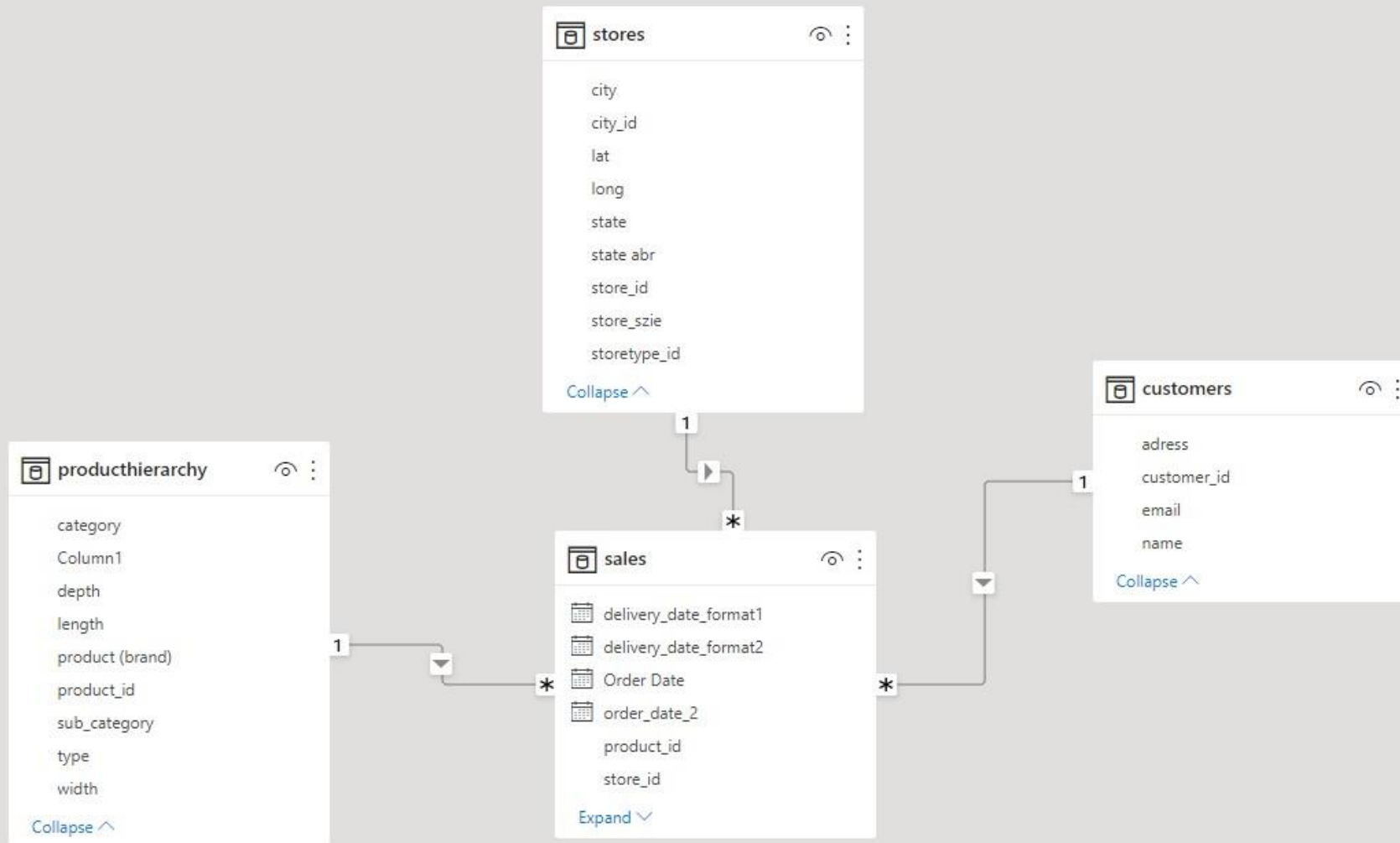
PK

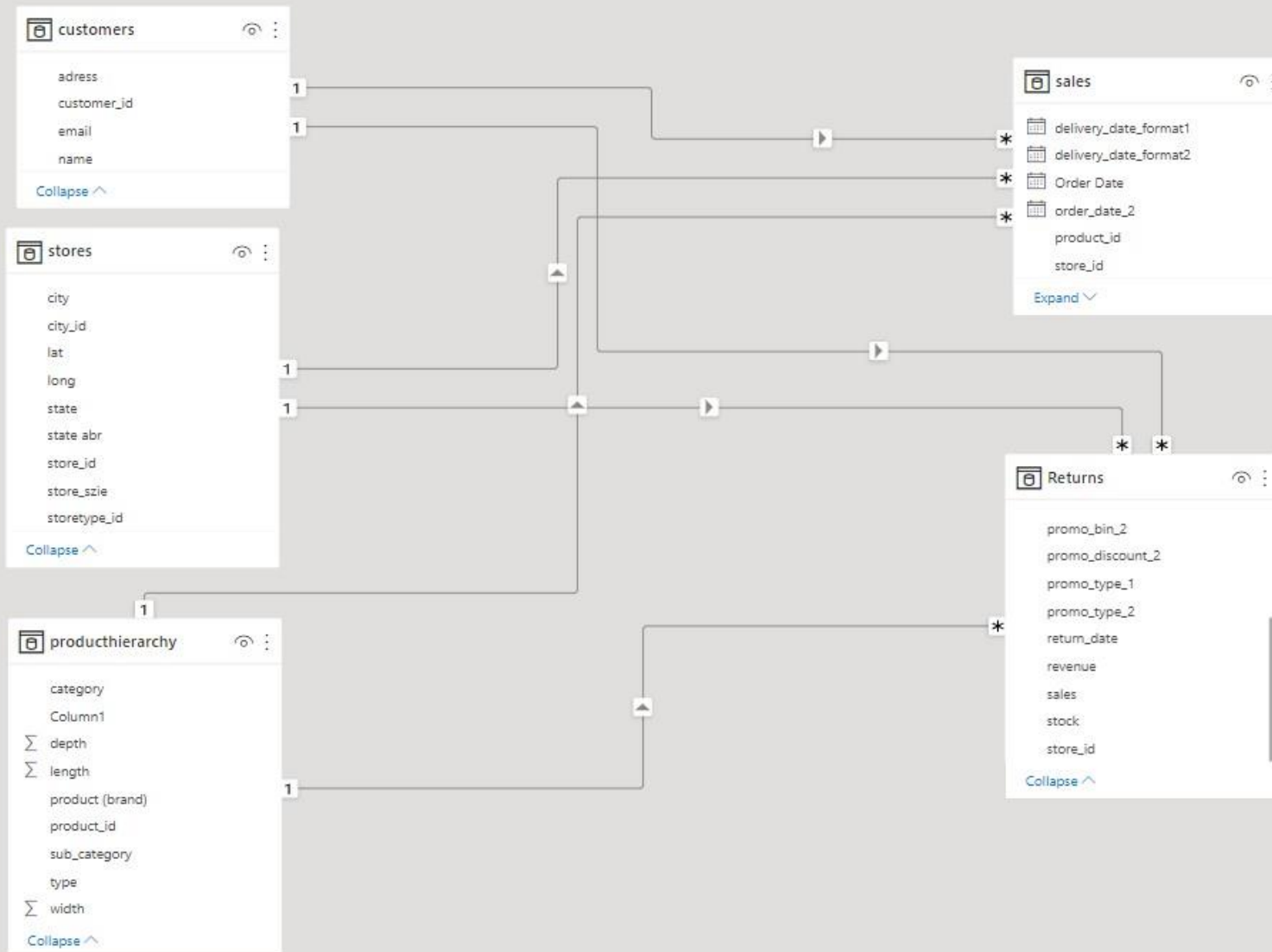
✓ Dimensions

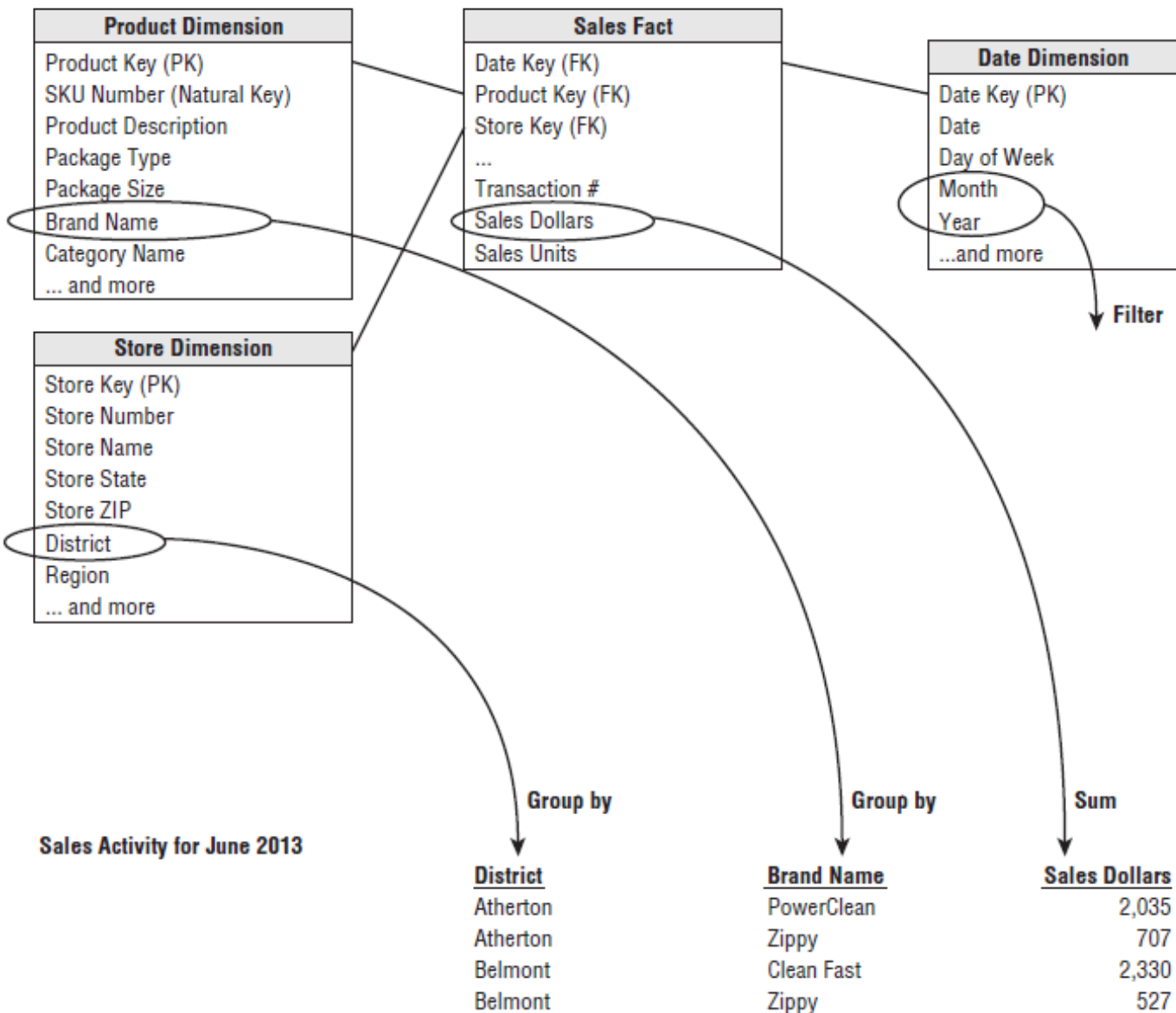
product_id	name	category	sub_category
1	Chili	Herbs	Spices
2	Garlic	Fruits & Vegetables	Vegetable
3	Banana	Fruits & Vegetables	Fruits
4	Chocolate	Sweets & Snacks	Sweets
5	Chips	Sweets & Snacks	Snacks

Denormalized

- *There is data redundancy!*
- *Optimized to get data out*
- *Query performance (read)*
- *User experience*







```

SELECT
store.district_name,
product.brand,
sum(sales_facts.sales_dollars) AS "Sales Dollars"
FROM
store,
product,
date,
sales_facts
WHERE
date.month_name="January" AND
date.year=2013 AND
store.store_key = sales_facts.store_key AND
product.product_key = sales_facts.product_key AND
date.date_key = sales_facts.date_key
GROUP BY
store.district_name,
product.brand

```

Star schema

- ✓ **Most common schema in Data Mart**
- ✓ **Simplest form (vs. snowflake schema)**
- ✓ **Work best for specific needs**
(simple set of queries vs complex queries)
- ✓ **Usablity + Performance for specific (read) use-case**

Kimball's DW/BI Architecture

Four separate and distinct components in the DW/BI environment:

- Operational source systems,
- ETL system,
- Data presentation area, and
- Business intelligence applications.

Kimball's DW/BI Architecture – Operational Source Systems

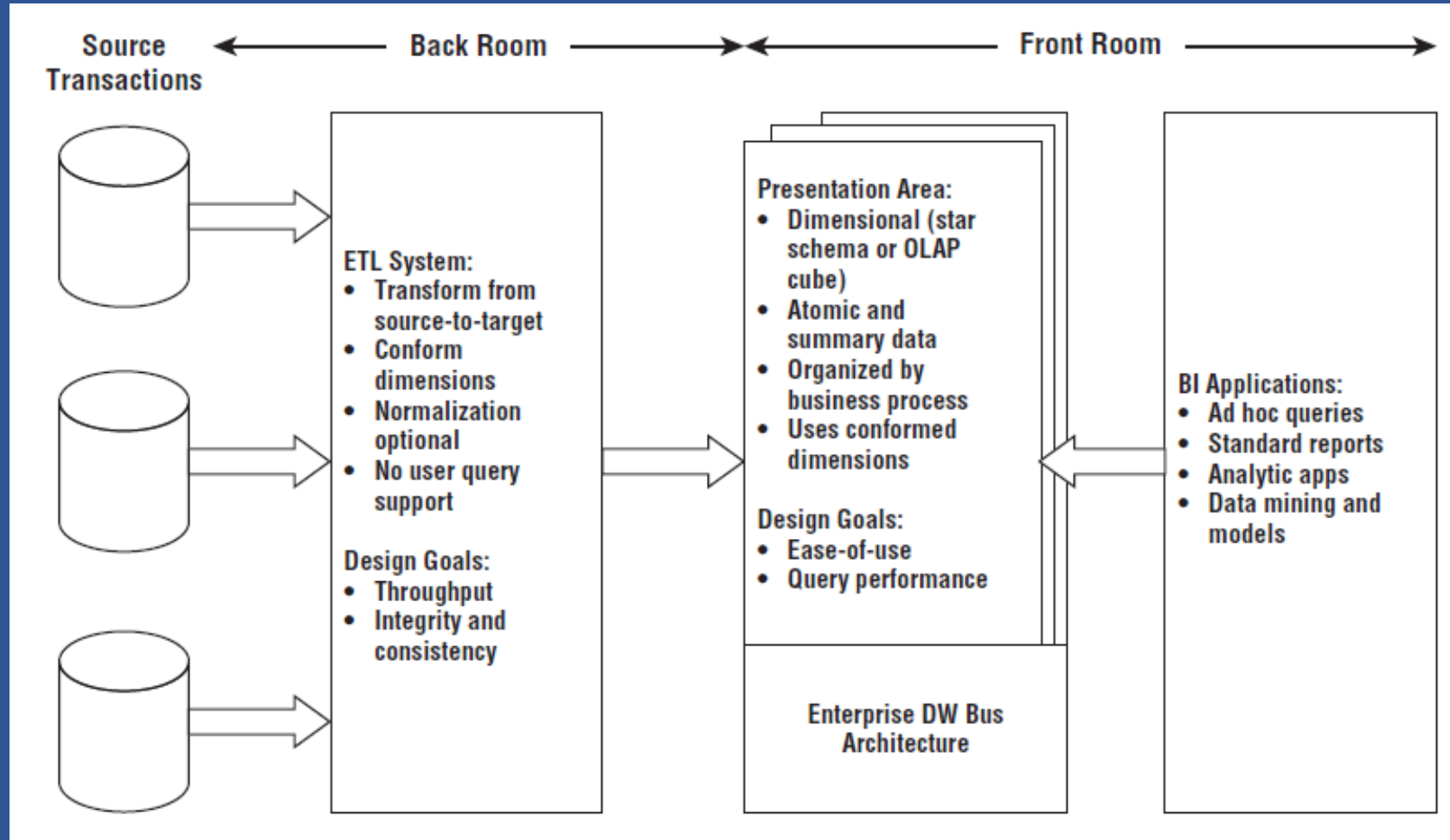
Operational source systems captures business transactions

Focus of source systems are processing performance and availability

Operational queries are narrow – one records at a time unlike DW / BI systems

Source systems maintains very little history

Kimball's DW/BI Architecture – Core elements



Kimball's DW/BI Architecture – ETL

The *extract, transformation, and load (ETL) system* of the DW/BI environment consists of a work area, instantiated data structures, and a set of processes.

Extracting means reading and understanding the source data and copying the data needed into the ETL system for further manipulation.

After the data is extracted to the ETL system, there are numerous potential transformations, such as cleansing the data (correcting misspellings, resolving domain conflicts, dealing with missing elements, or parsing into standard formats), combining data from multiple sources, and de-duplicating data.

The final step of the ETL process is the physical structuring and loading of data into the presentation area's target dimensional models.

The creation of both normalized structures for the ETL and dimensional structures for presentation means that the data is potentially extracted, transformed, and loaded twice—once into the normalized database and then again when you load the dimensional model.

Kimball's DW/BI Architecture – Presentation Area

The DW/BI presentation area is where data is organized, stored, and made available for direct querying by users, report writers, and other analytical BI applications.

Being ETL is the backroom for data preparation, Business users sees data at Front end after it is cooked at Backend.

Presentation area can be made off Star Schema or OLAP.

Presentation area contains granular data (atomic data) ; although presentation area may contain aggregated data but granular data in dimensional form is needed to cater to users' ad-hoc query needs.

Data in the queryable presentation area of the DW/BI system must be dimensional, atomic (complemented by performance-enhancing aggregates), business process-centric, and adhere to the enterprise data warehouse bus architecture.

Snowflake schema

Star schema

sales_id	product_id	customer_id	units	price
1	3	23	1	2.99
2	5	13	1	1.99
3	2	7	2	3.49
4	3	16	1	2.29
5	3	13	5	1.49

✓ Facts

✓ Dimensions

product_id	name	category	sub_category
1	Chili	Herbs	Spices
2	Garlic	Fruits & Vegetables	Vegetable
3	Banana	Fruits & Vegetables	Fruits
4	Chocolate	Sweets & Snacks	Sweets
5	Chips	Sweets & Snacks	Snacks

Snowflake schema

✓ Facts

sales_id	product_id	customer_id	units	price
1	3	23	1	2.99
2	5	13	1	1.99
3	2	7	2	3.49
4	3	16	1	2.29
5	3	13	5	1.49

product_id	name	category_id	sub_category
1	Chili	1	Spices
2	Garlic	2	Vegetable
3	Banana	2	Fruits
4	Chocolate	3	Sweets
5	Chips	3	Snacks

Snowflake schema

(More) normalized

category_id	category
1	Herbs
2	Fruits & Vegetables
3	Sweets & Snacks

Snowflake schema

Advantage

- ✓ Less space (storage cost)
- ✓ No (less) redundant data
(easier to maintain/update,
less risk of corrupted data)
- ✓ Solves write slow downs

Disadvantage

- ✓ More complex
- ✓ More joins
(more complex SQL queries)
- ✓ Less performance Data Marts
/ Cubes

Snowflake schema

Data Mart

✓ **Star schema**

Core

✓ **Star schema**

✓ **Maybe snowflake schema**

Additivity in facts

Additivity

Additive

- ✓ Can be added across all dimensions
- ✓ Most flexible & useful

Semi-additive

- ✓ Can be added across a few dimensions

Non-additive

- ✓ Cannot be added across any dimension

Additive facts

sales_id	product_id	date_id	units	amount
1	3	20220101	1	2.99
2	5	20220102	1	1.99
3	2	20220102	2	3.49
4	3	20220103	1	2.29
5	3	20220104	5	1.49

product_id	name	category	sub_category
1	Chili	Herbs	Spices
2	Garlic	Fruits & Vegetables	Vegetable
3	Banana	Fruits & Vegetables	Fruits
4	Chocolate	Sweets & Snacks	Sweets
5	Chips	Sweets & Snacks	Snacks

category	units
Herbs	0
Fruits & Vegetables	9
Sweets & Snacks	1

name	Units
Chili	0
Garlic	2
Banana	7
Chocolate	0
Chips	1

Additive facts

date_id	Date	Day	Month
20220101	01/01/2022	1	1
20220102	02/01/2022	2	1
20220103	03/01/2022	3	1
20220104	04/01/2022	4	1
20220105	05/01/2022	5	1

Date	units
01/01/2022	2
02/01/2022	3
03/01/2022	1
04/01/2022	5

sales_id	product_id	date_id	units	price
1	3	20220101	1	2.99
2	5	20220102	1	1.99
3	2	20220102	2	3.49
4	3	20220103	1	2.29
5	3	20220104	5	1.49

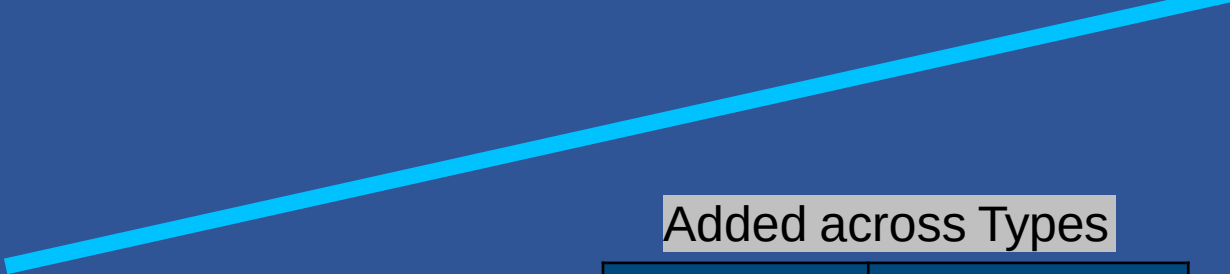
category	units
Herbs	0
Fruits & Vegetables	9
Sweets & Snacks	1

product_id	name	category	sub_category
1	Chili	Herbs	Spices
2	Garlic	Fruits & Vegetables	Vegetable
3	Banana	Fruits & Vegetables	Fruits
4	Chocolate	Sweets & Snacks	Sweets
5	Chips	Sweets & Snacks	Snacks

name	Units
Chili	0
Garlic	2
Banana	7
Chocolate	0
Chips	1

Semi-additive facts

balance_id	portfolio_id	date_id	balance
1	1	20220101	\$50
2	1	20220102	\$100
3	1	20220103	\$100
4	2	20220101	\$120
5	2	20220102	\$170
6	2	20220103	\$60



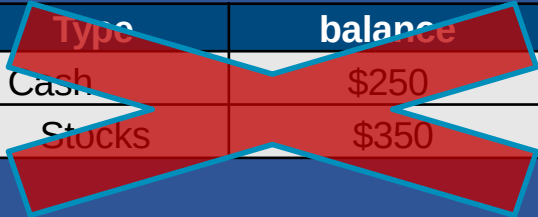
Portfolio_id	Type
1	Cash
2	Stocks

Added across Types

Date_id	balance
20220101	\$170
20220102	\$270
20220103	\$160

Added across Date

Type	balance
Cash	\$250
Stocks	\$350



Semi-additive facts

balance_id	portfolio_id	date_id	balance
1	1	20220101	\$50
2	1	20220102	\$100
3	1	20220103	\$100
4	2	20220101	\$120
5	2	20220102	\$170
6	2	20220103	\$60

Portfolio_id	Type
1	USD Cash
2	Stocks

Added across Types

Date_id	balance
20220101	\$170
20220102	\$270
20220103	\$160

Average across Date

Type	balance
USD Cash	\$83.33
Stocks	\$116.67

Additivity

Additive

- ✓ Can be added across all dimensions
- ✓ Most flexible & useful
- ✓ Most facts are fully additive

Semi-additive

- ✓ Can be added across a few dimensions
- ✓ Used carefully & less flexible
- ✓ Averaging might be an alternative
- ✓ Example: Balance

Non-additive

- ✓ Cannot be added across any dimension

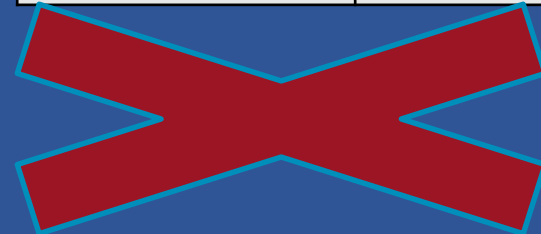
Non-additive facts

- *Price*
- *Percentages*
- *Ratios*

sales_id	product_id	date_id	units	price
1	3	20220101	1	2.99
2	5	20220102	1	1.99
3	2	20220102	2	3.49
4	3	20220103	1	2.29
5	3	20220104	5	1.49

product_id	name	category	sub_category
1	Chili	Herbs	Spices
2	Garlic	Fruits & Vegetables	Vegetable
3	Banana	Fruits & Vegetables	Fruits
4	Chocolate	Sweets & Snacks	Sweets
5	Chips	Sweets & Snacks	Snacks

category	price
Herbs	\$0
Fruits & Vegetables	\$10.26
Sweets & Snacks	\$1.99



Additivity

Additive

- ✓ Can be added across all dimensions
- ✓ Most flexible & useful
- ✓ Most facts are fully additive

Semi-additive

- ✓ Can be added across a few dimensions
- ✓ Used carefully & less flexible
- ✓ Averaging might be an alternative
- ✓ Example: Balance

Non-additive

- ✓ Cannot be added across any dimension
- ✓ Limited analytical value
- ✓ Store underlying value
- ✓ Ratio, price etc.

Year-to-Date facts

Year-to-Date facts

- ✓ Often requested by business users
- ✓ Tempted to store them in columns
- ✓ Month-to-Date, Quarter-to-Date, Fiscal-Year-to-Date etc.
- ✓ Better store the underlying values in defined grain (!)
- ✓ Instead calculate all the to-Date variations in BI tool

Transactional fact table

Transactional fact table

- ✓ 1 row = measurement of 1 event / transaction
- ✓ Taken place at a specific time
- ✓ One transaction defines the lowest grain

	FK	FK	Measure
sales_id	product_id	date_id	units
1	3	20220101	1
2	5	20220102	1
3	2	20220102	2
4	3	20220103	1
5	3	20220104	5

Sales transactions

	FK	FK	FK	Measure
call_id	emp_id	date_id	customer_id	duration
1	3	20220101	1	43
2	5	20220102	1	12
3	2	20220102	2	134
4	3	20220103	1	62
5	3	20220104	5	22

Calls

Characteristics

- ✓ Most common and very flexible
- ✓ Typically additive
- ✓ Tend to have a lot of dimensions associated
- ✓ Can be enormous in size

	FK	FK	Measure
sales_id	product_id	date_id	units
1	3	20220101	1
2	5	20220102	1
3	2	20220102	2
4	3	20220103	1
5	3	20220104	5

Sales transactions

	FK	FK	FK	Measure
call_id	emp_id	date_id	customer_id	duration
1	3	20220101	1	43
2	5	20220102	1	12
3	2	20220102	2	134
4	3	20220103	1	62
5	3	20220104	5	22

Calls

Periodic snapshot fact table

Periodic snapshot fact table

- ✓ 1 row = summarizes measure of many events / transactions
- ✓ Summarized of standard period (e.g. 1 day, 1 week etc.)
- ✓ Lowest period defines the grain

	Measure	Measure	Measure
week_id	revenue	sales	cost
1	323	123	12
2	541	322	31
3	242	108	12
4	352	212	51
5	312	198	25

Sales transactions

	Measure	Measure	Measure
day_id	no. calls	missed calles	duration
1	31	3	432
2	25	4	142
3	52	2	134
4	23	6	562
5	53	4	122

Calls

Characteristics

- ✓ Tend to be not as enormous in size
- ✓ Typically additive
- ✓ Tend to have a lot of facts and fewer dimensions associated
- ✓ No events = null or 0

	Measure	Measure	Measure
week_id	revenue	sales	cost
1	323	123	12
2	541	322	31
3	242	108	12
4	352	212	51
5	312	198	25

Sales transactions

	Measure	Measure	Measure
day_id	no. calls	missed calles	duration
1	31	3	432
2	25	4	142
3	52	2	134
4	23	6	562
5	53	4	122

Calls

Accumulation snapshot fact table

Accumulation snapshot fact table

- ✓ 1 row = summarizes measure of many events / transactions
- ✓ Summarized of lifespan of 1 process (e.g. order fulfillment)
- ✓ Definite beginning & definite ending (& steps in between)

	Date FK	Measure	Product FK	Date FK	Date FK	Date FK	Date FK	Measure
order_id	Order Date FK	No. Products	Product_FK	Production Start FK	Production End FK	Inspection Date FK	Shipping Date FK	Damaged products
1	20220102	100	32	20220103	20220110	20220112	20220113	3
2	20220103	100	32	20220104	20220112	20220113	20220113	4
3	20220103	100	32	20220103	20220112	20220113	20220114	1
4	20220104	100	32	20220106	20220110	20220112	20220113	0
5	20220104	100	32	20220108	20220117	20220119	20220120	6

Order production

Characteristics

- ✓ Least common
- ✓ Workflow or process analysis
- ✓ Multiple Date/Time foreign keys (for each process step)
- ✓ Date/Time keys associated with role-playing dimension

	Date FK	Measure	Product FK	Date FK	Date FK	Date FK	Date FK	Measure
order_id	Order Date FK	No. Products	Product_FK	Production Start FK	Production End FK	Inspection Date FK	Shipping Date FK	Damaged products
1	20220102	100	32	20220103	20220110	20220112	20220113	3
2	20220103	100	32	20220104	20220112	20220113	20220113	4
3	20220103	100	32	20220103	20220112	20220113	20220114	1
4	20220104	100	32	20220106	20220110	20220112	20220113	0
5	20220104	100	32	20220108	20220117	20220119	20220120	6

Order production

Types of fact tables

Types of fact tables

Type	Transactional	Periodic Snapshot	Accumulating Snapshot
Grain			
Date Dimensions			
No. Of dimensions			
Facts			
Size			
Performance			

Types of fact tables

Type	Transactional	Periodic Snapshot	Accumulating Snapshot
Grain	1 row = 1 transaction	1 row = 1 defined period (plus other dimensions)	1 row = lifetime of process /event
Date Dimensions			
No. Of dimensions			
Facts			
Size			
Performance			

Types of fact tables

Type	Transactional	Periodic Snapshot	Accumulating Snapshot
Grain	1 row = 1 transaction	1 row = 1 defined period (plus other dimensions)	1 row = lifetime of process /event
Date Dimensions	1 Transaction date	Snapshot date (end of period)	Multiple snapshot dates
No. Of dimensions			
Facts			
Size			
Performance			

Types of fact tables

Type	Transactional	Periodic Snapshot	Accumulating Snapshot
Grain	1 row = 1 transaction	1 row = 1 defined period (plus other dimensions)	1 row = lifetime of process /event
Date Dimensions	1 Transaction date	Snapshot date (end of period)	Multiple snapshot dates
No. Of dimensions	High	Lower	Very high
Facts			
Size			
Performance			

Types of fact tables

Type	Transactional	Periodic Snapshot	Accumulating Snapshot
Grain	1 row = 1 transaction	1 row = 1 defined period (plus other dimensions)	1 row = lifetime of process /event
Date Dimensions	1 Transaction date	Snapshot date (end of period)	Multiple snapshot dates
No. Of dimensions	High	Lower	Very high
Facts	Measures of transactions	Cumulative measures of transactions in period	Measures of process in lifespan
Size			
Performance			

Types of fact tables

Type	Transactional	Periodic Snapshot	Accumulating Snapshot
Grain	1 row = 1 transaction	1 row = 1 defined period (plus other dimensions)	1 row = lifetime of process /event
Date Dimensions	1 Transaction date	Snapshot date (end of period)	Multiple snapshot dates
No. Of dimensions	High	Lower	Very high
Facts	Measures of transactions	Cumulative measures of transactions in period	Measures of process in lifespan
Size	Largest (most detailed grain)	Middle (less detailed grain)	Lowest (highest aggregation)
Performance			

Types of fact tables

Type	Transactional	Periodic Snapshot	Accumulating Snapshot
Grain	1 row = 1 transaction	1 row = 1 defined period (plus other dimensions)	1 row = lifetime of process /event
Date Dimensions	1 Transaction date	Snapshot date (end of period)	Multiple snapshot dates
No. Of dimensions	High	Lower	Very high
Facts	Measures of transactions	Cumulative measures of transactions in period	Measures of process in lifespan
Size	Largest (most detailed grain)	Middle (less detailed grain)	Lowest (highest aggregation)
Performance	Can be improved with aggregation	Better (less detailed)	Good performance

Steps to create a fact table

Steps to create a fact table

What are the key decisions we need to take during the design?

4 key decisions

**Considering the
business needs**

Tables & columns

Steps to create a fact table

1) Identify business process for analysis

Example:
Sales,
Order processing

sales_id	date	Sales amount
1	2022-01-01	\$41
2	2022-01-02	\$15
3	2022-01-02	\$24
4	2022-01-03	\$13
5	2022-01-04	\$52

2) Declare the grain

Example: Transaction, Order, Order lines, Daily, Daily + location

3) Identify dimensions that are relevant

What, when, where, how and why

Example: Time, locations, products, customers,...

Filtering & grouping

"Soul" for analysis

4) Identify facts for measurement

Defined by the grain & not by specific use-case

Steps to create a fact table

1) Identify business process for analysis

sales_id	date	Sales amount
1	2022-01-01	\$41
2	2022-01-02	\$15
3	2022-01-02	\$24
4	2022-01-03	\$13
5	2022-01-04	\$52

2) Declare the grain

Example: Transaction

3) Identify dimensions that are relevant

Example: Time, locations, products

sales_id	date	Sales amount	prod_id	loc_id
1	20220101	\$41	3	1
2	20220102	\$15	4	5
3	20220102	\$24	6	4
4	20220103	\$13	1	3
5	20220104	\$52	23	4

4) Identify facts for measurement

Example: Sales amount & order quantity

Factless fact table



Factless fact table

- ✓ Facts are usually numeric
- ✓ Sometimes only dimensionals aspects of an event are recorded
- ✓ Example new employee is registered

reg_id	Entry Date FK	dep_id	region_id	manager_id	Pos_id
1	20220102	1	2	3	10
2	20220103	3	3	4	112
3	20220103	4	6	3	202
4	20220104	4	8	6	110
5	20220104	3	4	8	17

Employee registration

Events

No metrics

Factless fact table

- ✓ How many employees have been registered last month?
- ✓ How many employees have been registered in a certain region?
- ✓ Example new employee is registered

reg_id	Entry Date FK	dep_id	region_id	manager_id	Pos_id
1	20220102	1	2	3	10
2	20220103	3	3	4	112
3	20220103	4	6	3	202
4	20220104	4	8	6	110
5	20220104	3	4	8	17

Events

No metrics

Employee registration

Factless fact table

promo_id	Promo date_id	prod_id	channel_id	campaign_id
1	20220102	5	2	3
2	20220103	3	3	4
3	20220103	4	6	3
4	20220104	4	8	6
5	20220104	3	4	8

Employee registration

Events

No metrics

Occurrence of events

Natural vs. Surrogate key

Natural vs. Surrogate key

Natural keys

A grey arrow-shaped box with the text 'Natural keys' is positioned at the top center. Two blue arrows point downwards from the bottom corners of this box to the 'product_id' column of the 'Products' table and the 'sales_id' column of the 'Sales' table, respectively.

product_id	name	category
PX30	Chili	Herbs
PT32	Garlic	Fruits & Vegetables
AX42	Banana	Fruits & Vegetables
DA24	Chocolate	Sweets & Snacks
PO20	Chips	Sweets & Snacks

Products

sales_id	date	Sales amount
GXF-EFS	2022-01-01	\$41
DOS-FWA	2022-01-02	\$15
DSF-GWS	2022-01-02	\$24
PTG-DWD	2022-01-03	\$13
ERW-DWD	2022-01-04	\$52

Sales

Natural vs. Surrogate key

Natural keys

- ✓ Come out of the source system

Product_PK	product_id	name	category
1	PX30	Chili	Herbs
2	PT32	Garlic	Fruits & Vegetables
3	AX42	Banana	Fruits & Vegetables
4	DA24	Chocolate	Sweets & Snacks

Surrogate key

Artificial keys

- ✓ Integer number
- ✓ _PK or _FK suffix
- ✓ Created by the database / ETL tool

Benefits

Surrogate key

- ✓ Improve performance (less storage/better joins)
- ✓ Handle dummy values (nulls / missing values) e.g. 999 or -1
- ✓ Integrate multiple source systems
- ✓ Easier administrate / update
- ✓ Sometimes there are even no natural keys available

Practical guidelines

Surrogate key

- ✓ **Always use surrogate keys in tables as main PK and FK**
- ✓ **Both for Facts & Dimensions (except date dimension)**
- ✓ **Optionally keep the natural keys**

Case study: E-Commerce

Case study: E-Commerce

Corporate IT

E-Commerce company



✓ 3 websites



✓ Each website operated independently by multiple departments



✓ ~ 1000 individual products



✓ Groceries, kitchen products, household products etc.



Case study: E-Commerce

Data collection

✓ Shopping cart check out

✓ Warehouse data



Sales data

Customer_id

Customer_name

Order_id

Order_line_name

Order_line_id

Quantity

Unit_price

Discounted_price

sales amount

promo_id

product_cost

DateTime

Customer_id	Customer name	Order_id	Order_line_name	Order_line_id	Quantity	Unit_price	Discounted_price	promo_id	sales amount	product_cost	DateTime
312	Franklin Miller	2314	Sunglasses SU-6	34	2	22.99	22.99	null	45.98	14.84	23/4/2022 13:34
312	Franklin Miller	2314	Beach towel red	156	3	8.99	8.99	null	26.97	4.87	23/4/2022 13:35
312	Franklin Miller	2314	Swimsuit blue	643	1	16.99	14.99		14.99	12.53	23/4/2022 13:36

Case study: E-Commerce

Goals

- ✓ Logistics in warehouse
- ✓ Maximizing profits
 - ❖ Profit margine, sales volume, product cost, promotions, discounts



Sales data

Customer_id

Customer_name

Order_id

Order_line_name

Order_line_id

Quantity

Unit_price

Discounted_price

sales amount

promo_id

product_cost

DateTime

Customer_id	Customer name	Order_id	Order_line_name	Order_line_id	Quantity	Unit_price	Discounted_price	promo_id	sales amount	product_cost	DateTime
312	Franklin Miller	2314	Sunglasses SU-6	34	2	22.99	22.99	null	45.98	14.84	23/4/2022 13:34
312	Franklin Miller	2314	Beach towel red	156	3	8.99	8.99	null	26.97	4.87	23/4/2022 13:35
312	Franklin Miller	2314	Swimsuit blue	643	1	16.99	14.99		3	14.99	12.53 23/4/2022 13:36

Case study: E-Commerce

Step 1

Identify Business process

✓ Business process for first DWH?

- ❖ Most critical for business
- ❖ Data availability, data quality

Sales transactions

- ❖ Which products sold
- ❖ What is sales profit
- ❖ Sales of each website
- ❖ Performance on different days
- ❖ Sales over time

Case study: E-Commerce

Step 2

Declare the grain

✓ What level of detail?

- ❖ Most analytical value with atomic grain
- ❖ Highest dimensionality

Order + Order line

Customer_id	Customer name	Order_id	Order_line_name	Order_line_id	Quantity	Unit_price	Discounted_price	promo_id	sales amount	product_cost	DateTime
312	Franklin Miller	2314	Sunglases SU-6	34	2	22.99	22.99	null	45.98	14.84	23/4/2022 13:34
312	Franklin Miller	2314	Beach towel red	156	3	8.99	8.99	null	26.97	4.87	23/4/2022 13:35
312	Franklin Miller	2314	Swimsuit blue	643	1	16.99	14.99	3	14.99	12.53	23/4/2022 13:36

Customer_id	Customer name	Order_id	No. order lines	Total quantity	sales amount	DateTime
312	Franklin Miller	2314	3	6	87.94	23/4/2022 13:34

Case study: E-Commerce

Step 3

Identify dimensions

- ✓ **Descriptive aspects of measures**
- ❖ **Naturally derived after grain defined**

Dimensions

Customer_id	Customer name	Order_id	Order_line_name	Order_line_id	Quantity	Unit_price	Discounted_price	promo_id	sales amount	product_cost	DateTime
312	Franklin Miller	2314	Sunglasses SU-6	34	2	22.99	22.99	null	45.98	14.84	23/4/2022 13:34
312	Franklin Miller	2314	Beach towel red	156	3	8.99	8.99	null	26.97	4.87	23/4/2022 13:35
312	Franklin Miller	2314	Swimsuit blue	643	1	16.99	14.99	3	14.99	12.53	23/4/2022 13:36

- ❖ Customer
- ❖ Products
- ❖ Promotions
- ❖ Time/date
- ❖ Website

Case study: E-Commerce

Step 3

Identify dimensions

- ✓ **Descriptive aspects of measures**
- ❖ **Naturally derived after grain defined**

Dimensions

Sales_PK	Website_FK	Customer_FK	Order_id	Order_line_FK	Quantity	Unit_price	Discounted_price	promo_FK	sales_amt	product_cost	DateTime_FK
1001	2	312	2314	34	2	22.99	22.99	-1	45.98	14.84	202204231300
1002	2	312	2314	156	3	8.99	8.99	-1	26.97	4.87	202204231300
1003	2	312	2314	643	1	16.99	14.99	3	14.99	12.53	202204231300

- ❖ Customer
- ❖ Products
- ❖ Promotions
- ❖ Time/date
- ❖ Website

Case study: E-Commerce

Step 4

Identify facts for measurement

- ✓ What facts are in the fact table?
- ❖ Must comply with the grain

Facts

Sales_PK	Website_FK	Customer_FK	Order_id	Order_line_FK	Quantity	Unit_price	Discounted_price	promo_FK	Discount	sales amount	product_cost	profit	DateTime_FK
1001	2	312	2314	34	2	22.99	22.99	-1	0	45.98	14.84	31.14	202204231300
1002	2	312	2314	156	3	8.99	8.99	-1	0	26.97	4.87	22.1	202204231300
1003	2	312	2314	643	1	16.99	14.99	3	2.00	14.99	12.53	2.46	202204231300

- ✓ Additive
- ❖ Discount absolut (yes?)
- ❖ Discount percentage (no?)
- ❖ Profit

Case study: E-Commerce

Result

					Fact	Fact	Fact		Fact	Fact	Fact	Fact	
Sales_PK	Website_FK	Customer_FK	Order_id	Order_line_FK	Quantity	Unit_price	Discounted_price	promo_FK	Discount	sales amount	product_cost	profit	DateTime_FK
1001	2	312	2314	34	2	22.99	22.99	-1	0	45.98	14.84	31.14	202204231300
1002	2	312	2314	156	3	8.99	8.99	-1	0	26.97	4.87	22.10	202204231300
1003	2	312	2314	643	1	16.99	14.99	3	2.00	14.99	12.53	2.46	202204231300

Website

Customer

Products

Date/time

Dimension tables

Dimensions tables

✓ Always has a Primary Key (PK)

Sales_PK	Website_FK	Customer_FK	Order_id	Order_line_ID
1001	2	312	2314	P034
1002	2	312	2314	P156
1003	2	312	2314	P643

Product_ID	Name	Category
P001	Sunglases TR-7	Assecoirs
P002	Chocolate bar 70% cacao	Sweets
P003	Oat meal biscuits	Sweets

✓ Use surrogate key

Product_PK	Name	Category
1	Sunglases TR-7	Assecoirs
2	Chocolate bar 70% cacao	Sweets
3	Oat meal biscuits	Sweets

Product_PK	Product_ID
1	P001
2	P002
3	P003

✓ Lookup table

Dimensions tables

Sales_PK	Website_FK	Customer_FK	Order_id	Order_line_ID
1001	2	312	2314	P034
1002	2	312	2314	P156
1003	2	312	2314	P643

Product_ID	Name	Category
P001	Sunglases TR-7	Assecoirs
P002	Chocolate bar 70% cacao	Sweets
P003	Oat meal biscuits	Sweets

Product_PK	Name	Category
1	Sunglases TR-7	Assecoirs
2	Chocolate bar 70% cacao	Sweets
3	Oat meal biscuits	Sweets

Product_PK	Product_ID
1	P001
2	P002
3	P003

```

SELECT
    S.*,
    P.Product_PK
FROM Sales_Fact S
LEFT JOIN Product_Dim as P
ON P.Product_ID = S.Order_line_ID
    
```

Sales_PK	Website_FK	Customer_FK	Order_id	Order_line_ID	Product_FK	Unit_price
1001	2	312	2314	P034	34	22.99
1002	2	312	2314	P156	156	8.99
1003	2	312	2314	P643	643	16.99

Dimensions tables

- ✓ Always has a Primary Key (PK)

Product_PK	Name	Category
1	Sunglasses TR-7	Assecoirs
2	Chocolate bar 70% cacao	Sweets
3	Oat meal biscuits	Sweets

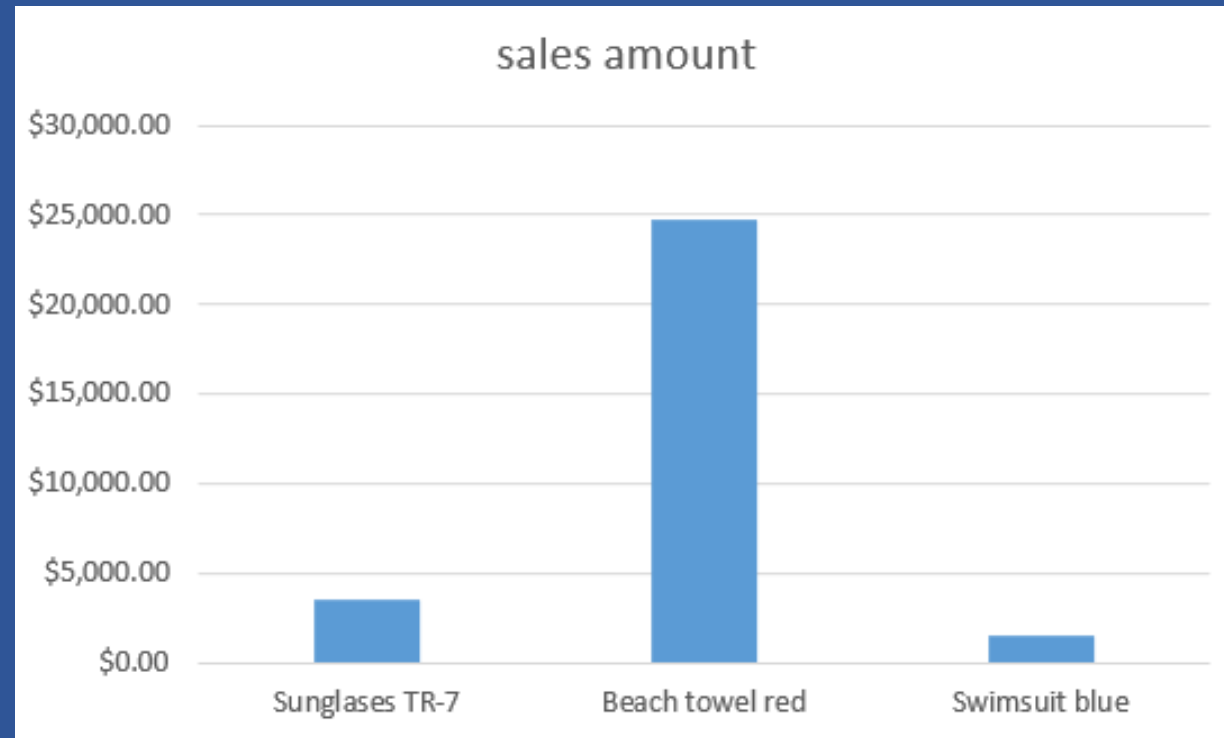
Customer_id	Customer name	Order_id	Order_line_name	Order_line_FK	Quantity	Unit_price	Discounted_price
312	Franklin Miller	2345	Sunglasses TR-7	1	2	23.99	23.99
312	Franklin Miller	2314	Beach towel red	156	3	8.99	8.99
312	Franklin Miller	2314	Swimsuit blue	643	1	16.99	14.99

- ✓ Relatively few rows / many columns with descriptive attributes

Dimensions tables

✓ Group & Filter ("slice & dice")

Order_line_name	sales amount
Sunglasses TR-7	\$3,543.00
Beach towel red	\$24,678.00
Swimsuit blue	\$1,456.00



Date Dimension

Date Dimension

- ✓ **One of the most common & most important dimensions**
- ✓ **Contains date related features**
 - ❖ Year, Month (name & number), Day, Quarter, Week, Weekday (name & number), ...
- ✓ **Meaningful surrogate key** `YYYYMMDD`

For example `2022-04-02` ⇔ `20220402`
- ✓ **Extra row for** `no date/null (source)` ⇔ `1900-01-01 (dim)`

Date Dimension

- ✓ Time is usually a separate dimension
- ✓ Can be populated in advance (e.g. for next 5 or 10 years)

Date features

- Numbers & Text (e.g. January, 1)
- Long & Abbreviated (Jan, January – Mon, Monday)
- Combinations of attributes (Q1, 2022-Q1)
- Fiscal dates (Fiscal Year etc.)
- Flags (Weekend, company holidays etc.)

Date_PK	Date	Month	Short Month	Year-Quarter	Year	Weekday	Is_Weekend
20220101	2022-01-01	January	Jan	2022-Q1	2022	Saturday	1
20220102	2022-01-02	January	Jan	2022-Q1	2022	Sunday	1
20220103	2022-01-03	January	Jan	2022-Q1	2022	Monday	0

Date Dimension

- ✓ Time is usually a separate dimension
- ✓ Can be populated in advance (e.g. for next 5 or 10 years)

Date features

- Numbers & Text (e.g. January, 1)
- Long & Abbreviated (Jan, January – Mon, Monday)
- Combinations of attributes (Q1, 2022-Q1)
- Fiscal dates (Fiscal Year etc.)
- Flags (Weekend, company holidays etc.)

Nulls in dimensions

Nulls in dimensions

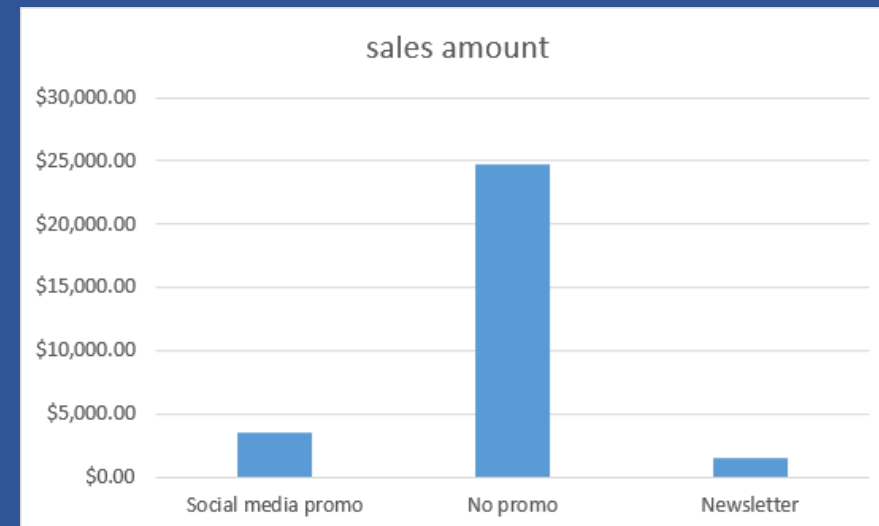
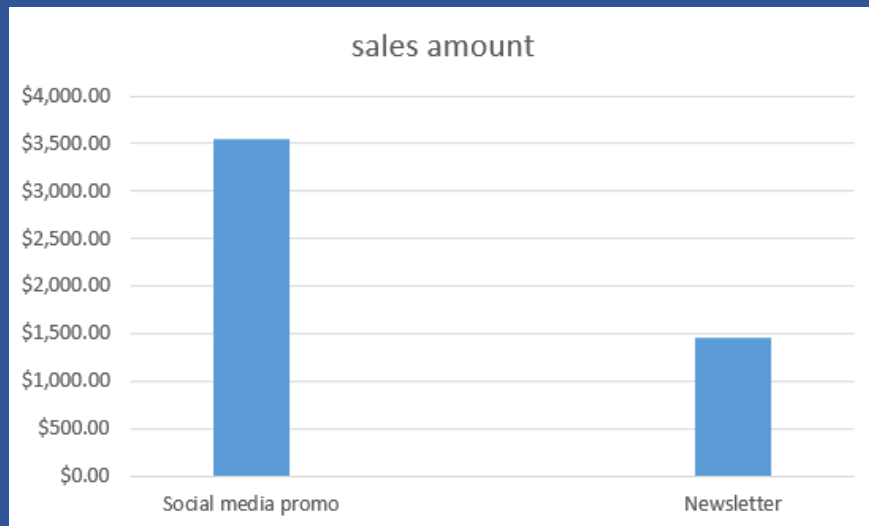
What we've learnt

✓ Nulls must be avoided in FKs

Customer_id	Customer name	Order_id	Order_line_name	Order_line_id	Quantity	Unit_price	Discounted_price	promo_id	sales amount	product_cost	DateTime
312	Franklin Miller	2314	Sunglasses SU-6	34	2	22.99	22.99	null	45.98	14.84	23/4/2022 13:34
312	Franklin Miller	2314	Beach towel red	156	3	8.99	8.99	null	26.97	4.87	23/4/2022 13:35
312	Franklin Miller	2314	Swimsuit blue	643	1	16.99	14.99	3	14.99	12.53	23/4/2022 13:36

❖ Nulls in FKs break referential integrity!

❖ They don't appear in Joins



Nulls in dimensions

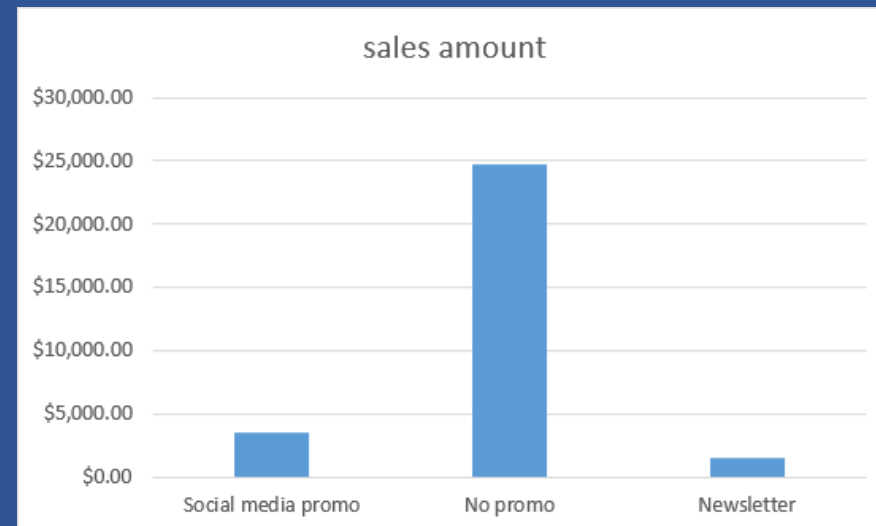
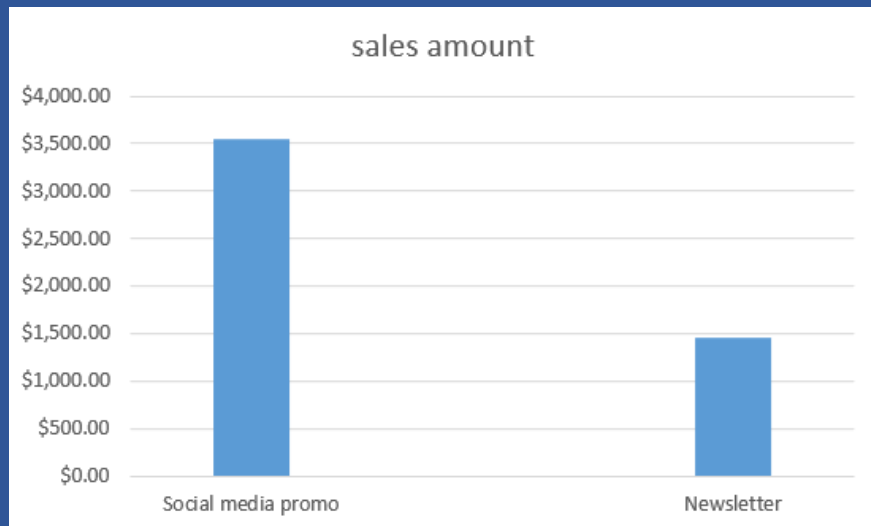
What we've learnt

✓ Nulls must be avoided in FKs

Sales_PK	Website_FK	Customer_FK	Order_id	Order_line_FK	Quantity	Unit_price	Discounted_pi	promo_FK	sales amc	product_cost	DateTime_FK
1001	2	312	2314	34	2	22.99	22.99	-1	45.98	14.84	202204231300
1002	2	312	2314	156	3	8.99	8.99	-1	26.97	4.87	202204231300
1003	2	312	2314	643	1	16.99	14.99	3	14.99	12.53	202204231300

❖ Nulls in FKs break referential integrity!

❖ They don't appear in Joins



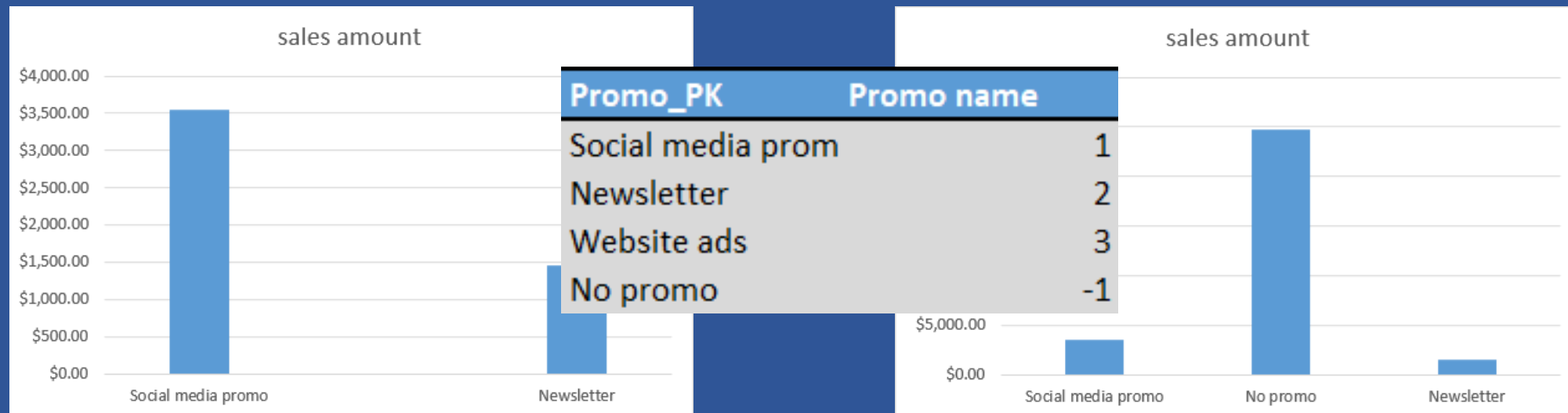
Nulls in dimensions

What we've learnt

- ✓ Nulls must be avoided in FKs

Sales_PK	Website_FK	Customer_FK	Order_id	Order_line_FK	Quantity	Unit_price	Discounted_p	promo_FK	sales amc	product_cost	DateTime_FK
1001	2	312	2314	34	2	22.99	22.99	-1	45.98	14.84	202204231300
1002	2	312	2314	156	3	8.99	8.99	-1	26.97	4.87	202204231300
1003	2	312	2314	643	1	16.99	14.99	3	14.99	12.53	202204231300

- ❖ Nulls in FKs break referential integrity!
- ❖ They don't appear in Joins



Nulls in dimensions

What we've learnt

- ✓ Nulls must be avoided in FKs

Sales_PK	Website_FK	Customer_FK	Order_id	Order_line_FK	Quantity	Unit_price	Discounted_pi	promo_FK	sales amc	product_cost	DateTime_FK
1001	2	312	2314	34	2	22.99	22.99	-1	45.98	14.84	202204231300
1002	2	312	2314	156	3	8.99	8.99	-1	26.97	4.87	202204231300
1003	2	312	2314	643	1	16.99	14.99	3	14.99	12.53	202204231300

- ❖ Nulls in FKs break referential integrity!
- ❖ They don't appear in Joins

Promo_PK	Promo name
Social media prom	1
Newsletter	2
Website ads	3
No promo	-1

Date_PK	Date
20220101	1/1/2022
20220102	1/2/2022
20220103	1/3/2022
19000101	1/1/1900

Nulls in dimensions

What we've learnt

✓ Nulls must be avoided in FKs

Sales_PK	Website_FK	Customer_FK	Order_id	Order_line_FK	Quantity	Unit_price	Discounted_price	promo_FK	sales amt	product_cost	DateTime_FK
1001	2	312	2314	34	2	22.99	22.99	-1	45.98	14.84	202204231300
1002	2	312	2314	156	3	8.99	8.99	-1	26.97	4.87	202204231300
1003	2	312	2314	643	1	16.99	14.99	3	14.99	12.53	202204231300

✓ Nulls can be present in Facts

Promo_PK	Promo name
Social media prom	1
Newsletter	2
Website ads	3
No promo	-1

Date_PK	Date
20220101	1/1/2022
20220102	1/2/2022
20220103	1/3/2022
19000101	1/1/1900

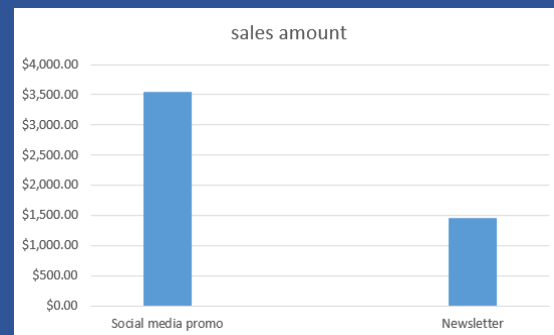
Nulls in dimensions

Dimensions

Promo_PK	Promo name
Social media prom	1
Newsletter	2
Website ads	3
No promo	-1

Date_PK	Date
20220101	1/1/2022
20220102	1/2/2022
20220103	1/3/2022
19000101	1/1/1900

- ✓ Replace nulls with descriptive values
 - ✓ More understandable for business users
 - ✓ Values appear in aggregations in BI tools



Hierarchies in dimensions

Hierarchies in dimensions

Source data

✓ Often normalized

Product name	Category_id
Milk	1
Printer	2
Red Towel	3
Green Towel	3
Blue Towel	3

Category_id	Category
1	Groceries
2	Electronics
3	Houshold

Hierarchies in dimensions

Source data

✓ Often normalized

Product name	Category_id
Milk	1
Printer	2
Red Towel	3
Green Towel	3
Blue Towel	3

Category_id	Category
1	Groceries
2	Electronics
3	Houshold

Customer_id	Customer name	Order_id	Order_line_id	Product_FK	Category_FK
312	Franklin Miller	2314	Sunglasses SU	34	2

Hierarchies in dimensions

Source data

✓ Often normalized

Product name	Category_id
Milk	1
Printer	2
Red Towel	3
Green Towel	3
Blue Towel	3

Category_id	Category
1	Groceries
2	Electronics
3	Houshold

Customer_id	Customer name	Order_id	Order_line_id	Product_FK	Category_FK
312	Franklin Miller	2314	Sunglasses SU	34	2

✓ Snowflaked schema (should be avoided)

Hierarchies in dimensions

Source data

✓ Often normalized

Product name	Category_id
Milk	1
Printer	2
Red Towel	3
Green Towel	3
Blue Towel	3

Category_id	Category
1	Groceries
2	Electronics
3	Houshold

Some professionals have the habit to normalize data

⇒ Bad for usability & performance!

⇒ We should not do that!

Hierarchies in dimensions

What we should do

✓ Denormalize / flattened

Product name	Category_id
Milk	1
Printer	2
Red Towel	3
Green Towel	3
Blue Towel	3

Category_id	Category
1	Groceries
2	Electronics
3	Houshold



Product_ID	Product name	Category
1	Milk	Groceries
2	Printer	Electronics
3	Red Towel	Houshold
4	Green Towel	Houshold
5	Blue Towel	Houshold

Flattened dimension

Hierarchies in dimensions

What we should do

✓ Consider combinations if helpful

Year-Month	Year-Month	Year-Quarter
01-01-2022	Jan-2022	2022-Q1
02-01-2022	Jan-2022	2022-Q1
03-01-2022	Jan-2022	2022-Q1

Location_PK	City	State	City-State
1	Nashville	Tennessee	Nashville, Tennessee
2	Nashville	Indiana	Nashville, Indiana
3	Kansas City	Kansas	Kansas City, Kansas

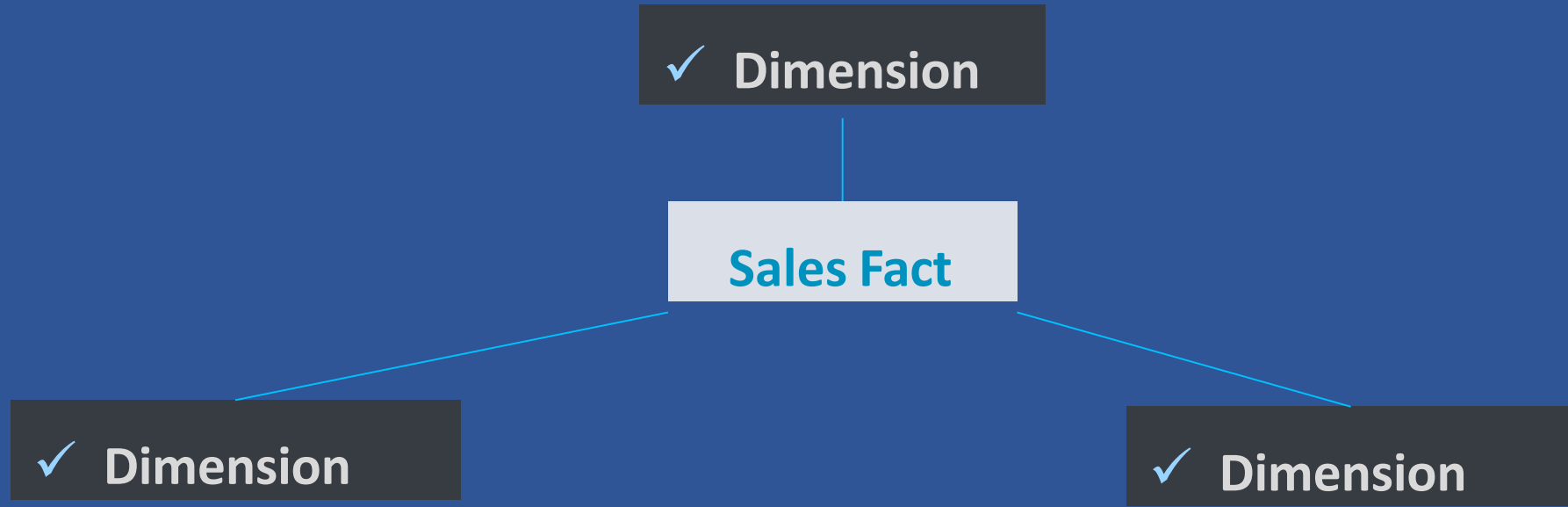
Conformed dimensions

Conformed dimensions

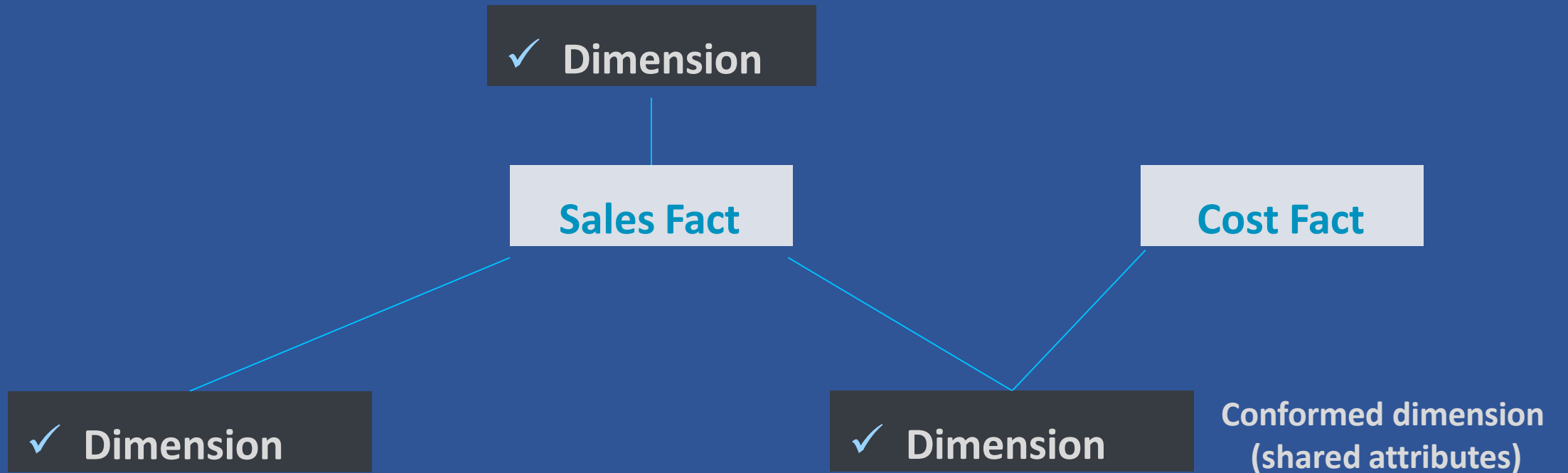
Conformed dimension is a dimension that is shared by multiple fact tables / stars.

Used to compare facts across different fact tables.

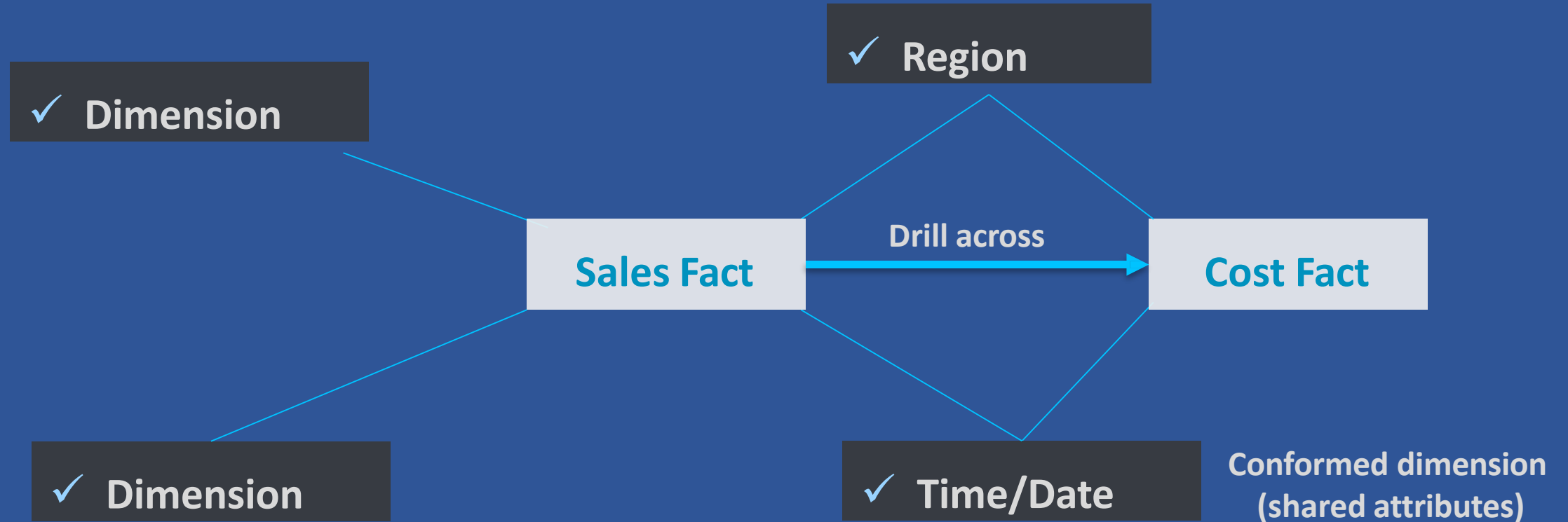
Conformed dimension



Conformed dimension



Conformed dimension



Conformed dimension

✓ Conformed Date dim

Month	Cost	Sales
January	\$50,300	\$67,300
February	\$55,300	\$71,400
March	\$65,100	\$79,400

✓ Conformed Region dim

Country	Cost	Sales
Spain	\$57,200	\$69,800
Belgium	\$15,300	\$21,900
wd	\$35,100	\$29,400

Conformed dimension

✓ Sales fact

Sales_PK	Sales	Date_FK
1	\$9,400	20220101
2	\$7,300	20220101
3	\$5,100	20220102

✓ Cost fact

Cost_PK	Cost	Date_FK
1	\$7,200	20220101
2	\$1,900	20220101
3	\$2,800	20220101

Conformed dimension

✓ Sales fact

Sales_PK	Sales	Date_FK
1	\$9,400	20220101
2	\$7,300	20220101
3	\$5,100	20220102

✓ Cost fact

Cost_PK	Cost	Date_FK
1	\$7,200	20220101
2	\$1,900	20220102
3	\$2,800	20220103

Same granularity not necessary!

Conformed dimension

✓ Sales fact

Sales_PK	Sales	Date_FK
1	\$9,400	20220101
2	\$7,300	20220101
3	\$5,100	20220102

✓ Cost fact

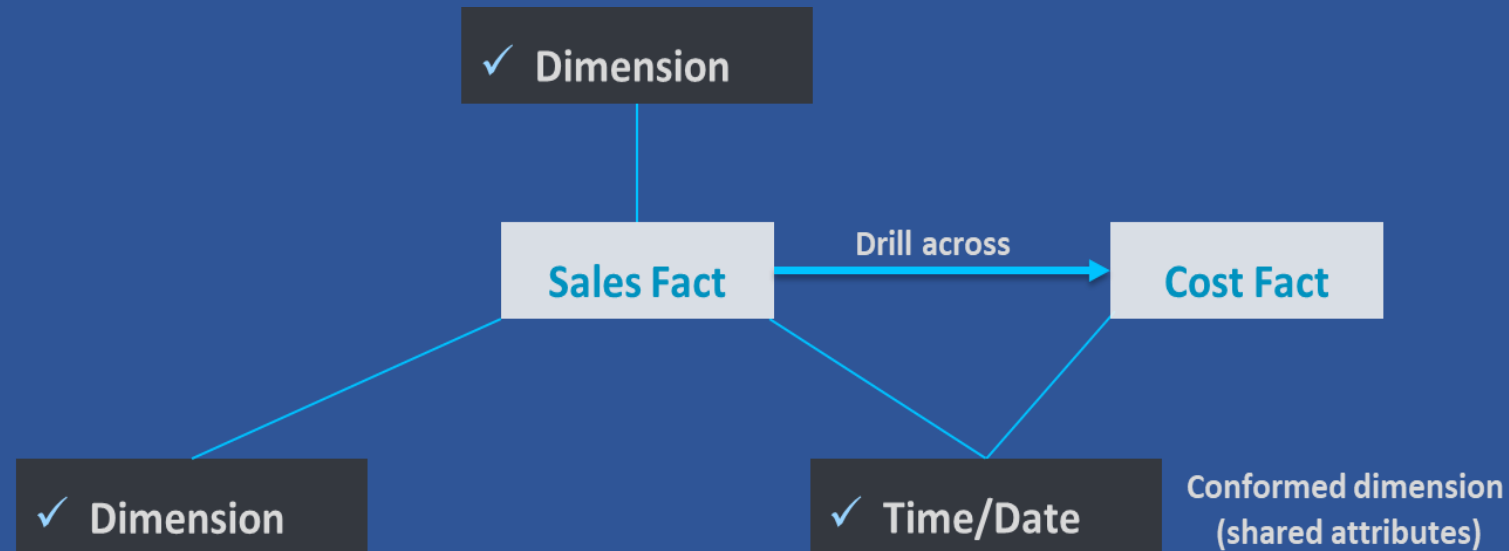
Cost_PK	Cost	DateMonth_FK
1	\$7,200	20220101
2	\$1,900	20220201
3	\$2,800	20220301

Different FK possible!

Conformed dimension

✓ Conformed Date dim

Month	Cost	Sales
January	\$50,300	\$67,300
February	\$55,300	\$71,400
March	\$65,100	\$79,400



Degenerate dimension

Degenerate dimension

✓ Transactional Sales fact

Transaction_PK	Amount	Payment_FK
1	\$530	234-032
2	\$553	234-032
3	\$654	234-033

Payment_PK	Header
234-032	Type A
234-033	Type A
234-034	Type B

Degenerate dimension

✓ Transactional Sales fact

Transaction_PK	Amount	Payment_FK
1	\$530	234-032
2	\$553	234-032
3	\$654	234-033

Payment_PK	Header
234-032	Type A
234-033	Type A
234-034	Type B

✓ All relevant information have already been extracted (to other dimensions)

Degenerate dimension

✓ Transactional Sales fact

Transaction_PK	Amount	Payment_FK	Payment_PK
1	\$530	234-032	234-032
2	\$553	234-032	234-033
3	\$654	234-033	234-034

- ✓ All relevant information have already been extracted (to other dimensions)
- ✓ Attribute can be still useful

Degenerate dimension

- ✓ Transactional Sales fact

Transaction_PK	Amount	Payment_DD
1	\$530	234-032
2	\$553	234-032
3	\$654	234-033

- ✓ All relevant information have already
been extracted (to other dimensions)
- ✓ Attribute can be still useful
- ✓ Indicate that it is a deg. dim. (e.g. _DD)

Degenerate dimension

- ✓ Degenerate dimension the dimension key without an associated dimension

Transaction_PK	Amount	Payment_DD
1	\$530	234-032
2	\$553	234-032
3	\$654	234-033

Occuring mostly in Transactional facts

Invoice no., billing no. or order_id
typically are degenerate dimensions

Junk dimensions

Junk dimensions

Transaction_PK	Amount	Payment_Type	Incoming / Outbound	Is_Bonus
1	\$530	Wired	Incoming	Yes
2	\$553	Credit Card	Outbound	No
3	\$654	Cash	Incoming	No

1. Eliminate them if they are not relevant
2. Leave them as they are in the fact
3. One Flag => One dimension

What if they are relevant?

Long text values? Table size?

Very wide fact table?

Alternative: Junk dimension

Junk dimensions

What is a junk dimension?

Dimension with various flags / indicators
with low cardinality

Junk dimensions

What is a junk dimension?

Like a box where we store items we need but have no separate storing location.

Junk dimensions

Note:

We call it "*junk dimension*" usually only internally.

Talking to business users we can refer to as

"*transactional indicator dimension*".

Junk dimensions

Transaction_PK	Amount	Payment_Type	Incoming / Outbound	Is_Bonus
1	\$530	Wired	Incoming	Yes
2	\$553	Credit Card	Outbound	No
3	\$654	Cash	Incoming	No

Transaction_PK	Amount	Transactional_Flag_FK
1	\$530	1
2	\$553	7
3	\$654	12

Flag_PK	Payment_Type	Incoming / Outbound	Is_Bonus
1	Wired	Incoming	Yes
2	Wired	Incoming	No
3	Wired	Outbound	Yes
4	Wired	Outbound	No

Junk dimensions

Payment_Type	Amount
Wired	\$5350
Credit Card	\$6553
Cash	\$6754

Is_Bonus	Amount
Yes	\$9350
No	\$11857

Junk dimensions

Number of combinations

$$3 \times 2 \times 2 = 12$$

Transaction_PK	Amount	Payment_Type	Incoming / Outbound	Is_Bonus
1	\$530	Wired	Incoming	Yes
2	\$553	Credit Card	Outbound	No
3	\$654	Cash	Incoming	No

Flag_PK	Payment_Type	Incoming / Outbound	Is_Bonus
1	Wired	Incoming	Yes
2	Credit Card	Outbound	No
3	Cash	Incoming	No
...			
12	Cash	Outbound	No

Junk dimensions

Many dimensions?

Many combinations!

9 indicators with 4 combinations

$$4^9 = 262144$$

1. Extract only available combinations of fact table
2. Two or more junk dimensions

$$4^5 = 1024$$

Role-playing dimension

Role-playing dimension

What is a role-playing dimension?

Dimension that is referenced multiple times
by a fact

Role-playing dimension

	Date FK	Measure		Date FK
	Order Date FK	No. Products	Product FK	Production Start FK
order_id				
1	20220102	100	32	20220103
2	20220103	100	32	20220104
3	20220103	100	32	20220103
4	20220104	100	32	20220106
5	20220104	100	32	20220108

Role 1

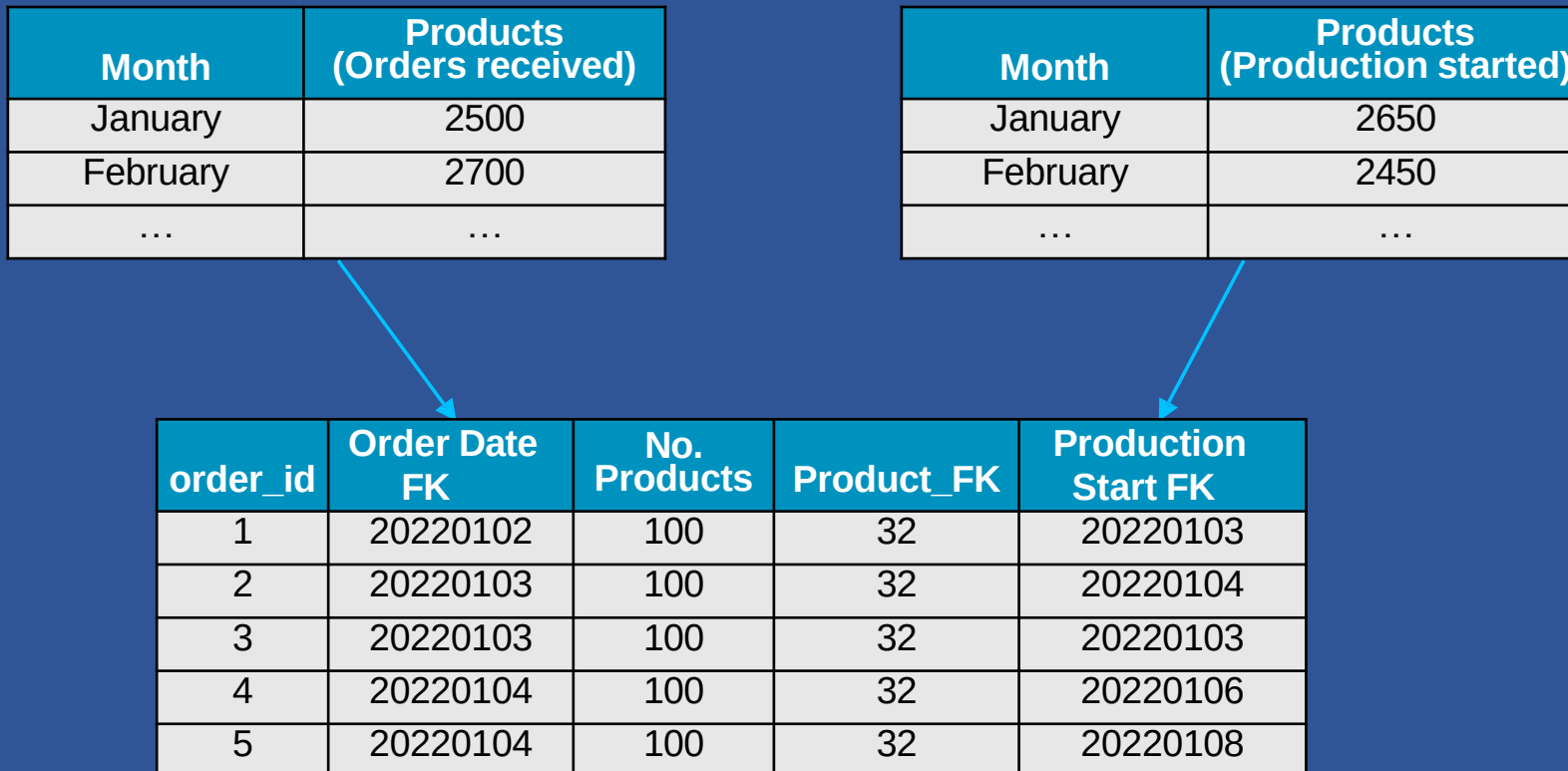
Role 2

Date_PK	Date	Month	Short Month	Year-Quarter	Year	Weekday	Is_Weekend
20220101	2022-01-01	January	Jan	2022-Q1	2022	Saturday	1
20220102	2022-01-02	January	Jan	2022-Q1	2022	Sunday	1
20220103	2022-01-03	January	Jan	2022-Q1	2022	Monday	0

Role-playing dimension

Month	Products (Orders received)
January	2500
February	2700
...	...

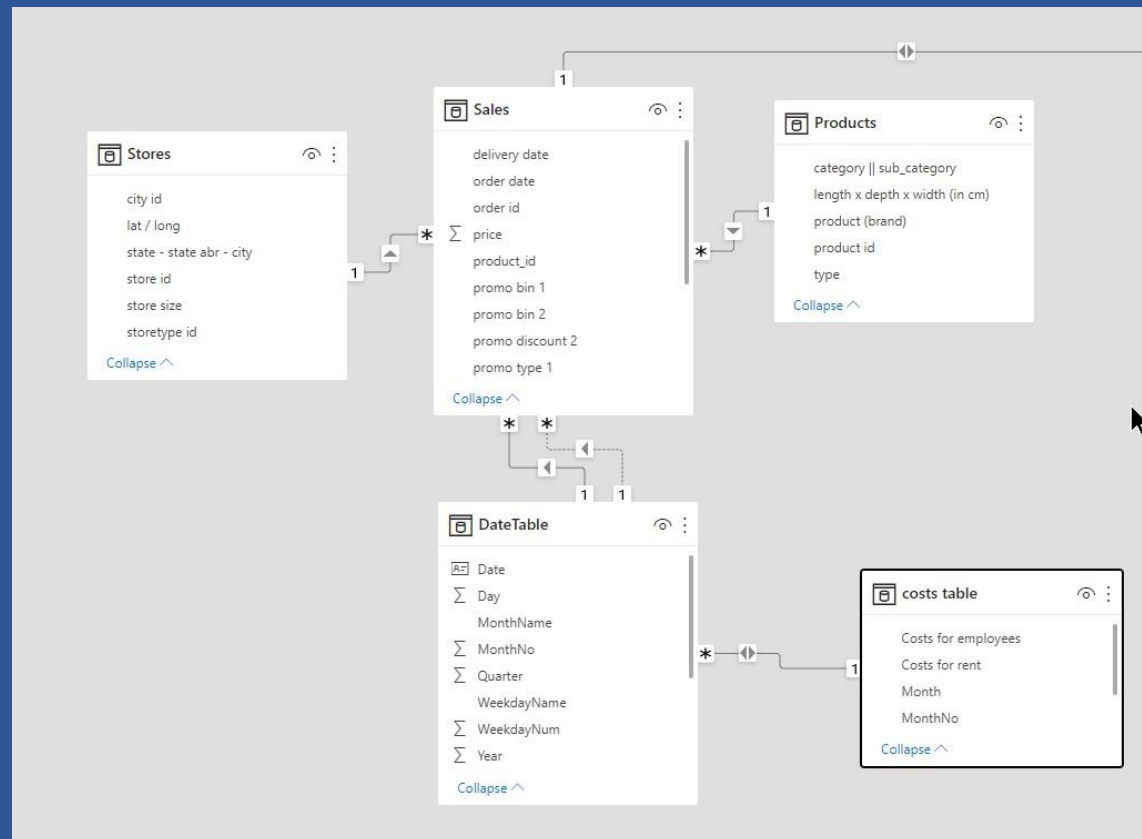
Month	Products (Production started)
January	2650
February	2450
...	...



order_id	Order Date FK	No. Products	Product_FK	Production Start FK
1	20220102	100	32	20220103
2	20220103	100	32	20220104
3	20220103	100	32	20220103
4	20220104	100	32	20220106
5	20220104	100	32	20220108

Role-playing dimension

- ✓ BI tools for example via active & inactive relationships



Role-playing dimension

- ✓ For analysis in SQL you can create *additional view for each role*
- ✓ No duplicated data but still we it appears like a separate dimension

Slowly changing dimensions

Slowly changing dimensions

Till now we have pretended dimensions never change...

... indeed they are rather static usually ...

... but surprise... they do change in the real world...

Develop a strategy to handle changes in dimensions...

Slowly changing dimensions

- 1. Be proactive: Ask about potential changes*
- 2. Business users + IT*
- 3. Strategy for each changing attribute*

Kimball introduced SCD in 1995 and distinguished between different types (1, 2, 3, ...).

Type 0: Retain Original

Type 0: Retain Original

- ✓ *There won't be any changes*
- ✓ *Date Table (expect for holidays etc.)*
- ✓ *"Original"*
- ✓ *Very simple and easy to maintain*

Type 1: Overwrite

Type 1: Overwrite

- ✓ *Old attributes are just overwritten*
- ✓ *Only current state is reflected*

Product_Key	Name	Category
1	Sunglases TR-7	Assecoirs
2	Chocolate bar 70% cacao	Sweets
3	Oat meal biscuits	Sweets



UPDATE

Product_Key	Name	Category
1	Sunglases TR-7	Assecoirs
2	Chocolate bar 70% cacao	Sweets
3	<i>Delicious</i> Oat meal biscuits	<i>Buscuits</i>

Type 1: Overwrite

✓ *Very simple*

✓ *No Fact table needs to be modified*

Product_Key	Name	Category
1	Sunglasses TR-7	Assecoirs
2	Chocolate bar 70% cacao	Sweets
3	Oat meal biscuits	Sweets



UPDATE

Product_Key	Name	Category
1	Sunglasses TR-7	Assecoirs
2	Chocolate bar 70% cacao	Sweets
3	<i>Delicious</i> Oat meal biscuits	<i>Buscuits</i>

Type 1: Overwrite

Problem

✓ *Very simple*

✓ *No Fact table needs to be modified*

❖ *History is lost!*

❖ *Insignificant changes*

❖ *Might affect / break existing queries*

Product_Key	Name	Category
1	Sunglases TR-7	Assecoirs
2	Chocolate bar 70% cacao	Sweets
3	Oat meal biscuits	Sweets

UPDATE

Product_Key	Name	Category
1	Sunglases TR-7	Assecoirs
2	Chocolate bar 70% cacao	Sweets
3	<i>Delicious</i> Oat meal biscuits	<i>Buscuits</i>

More significant

Not so significant

Type 2: New row

Type 2: New row

- ✓ Problem with Type 1: *No history of dimensions!*
- ✓ Only current state is reflected

Product_Key	Name	Category
1	Sunglasses TR-7	Assecoirs
2	Chocolate bar 70% cacao	Sweets
3	Oat meal biscuits	Sweets

UPDATE

Product_Key	Name	Category
1	Sunglasses TR-7	Assecoirs
2	Chocolate bar 70% cacao	Sweets
3	<i>Delicious</i> Oat meal biscuits	<i>Buscuits</i>

Type 2: New row

- ✓ Problem with Type 1: *No history of dimensions!*
- ✓ Only current state is reflected

Sales_Key	Name	Amount
1	Sunglasses TR-7	\$25
2	Chocolate bar 70% cacao	\$3
3	Oat meal biscuits	\$4
4	Chocolate bar 70% cacao	\$3
5	Oat meal biscuits	\$4

Before

Category	Amount
Assecoirs	\$25
Sweets	\$14

After

Category	Amount
Assecoirs	\$25
Sweets	\$6
Buscuits	\$8

Type 2: New row

- ✓ Problem with Type 1: *No history of dimensions!*
- ✓ Only current state is reflected

Sales_Key	Name	Amount
1	Sunglases TR-7	\$25
2	Chocolate bar 70% cacao	\$3
3	Oat meal biscuits	\$4
4	Chocolate bar 70% cacao	\$3
5	Oat meal biscuits	\$4

Before

Category	Amount
Assecoirs	\$25
Sweets	\$14

After

Category	Amount
Assecoirs	\$25
Sweets	\$6
Buscuits	\$8

Type 2: New row

- ✓ Problem with Type 1: *No history of dimensions!*
- ✓ Only current state is reflected

Sales_Key	Name	Amount
1	Sunglases TR-7	\$25
2	Chocolate bar 70% cacao	\$3
3	Oat meal biscuits	\$4
4	Chocolate bar 70% cacao	\$3
5	Oat meal biscuits	\$4

Before

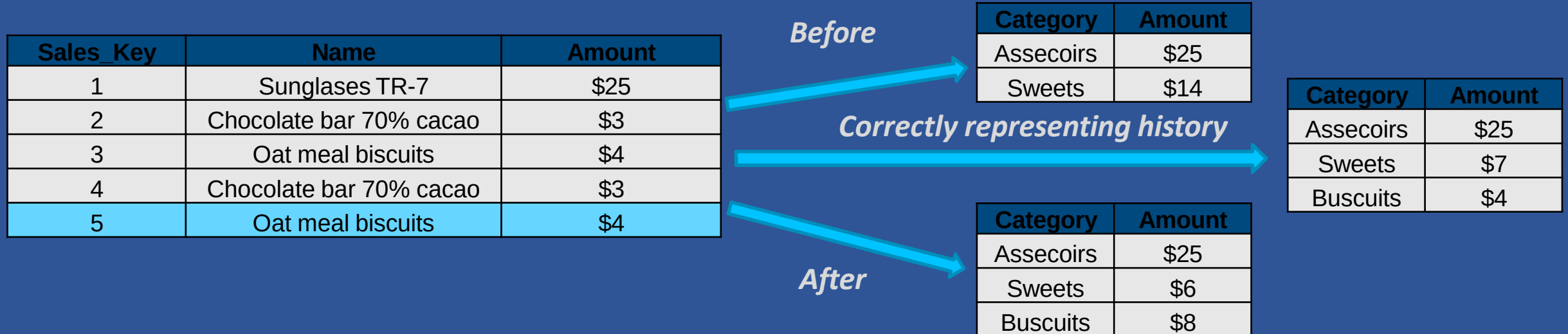
Category	Amount
Assecoirs	\$25
Sweets	\$14

After

Category	Amount
Assecoirs	\$25
Sweets	\$6
Buscuits	\$8

Type 2: New row

- ✓ Problem with Type 1: *No history of dimensions!*
- ✓ Only current state is reflected



Type 2: New row

- ✓ Type 2: Perfectly partitions history
- ✓ Changes are reflected with history

Default strategy

Sales_Key	Name	Amount
1	Sunglases TR-7	\$25
2	Chocolate bar 70% cacao	\$3
3	Oat meal biscuits	\$4
4	Chocolate bar 70% cacao	\$3
5	Oat meal biscuits	\$4

Before

Category	Amount
Assecoirs	\$25
Sweets	\$14

Correctly representing history

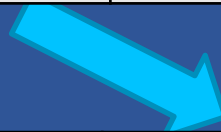
Category	Amount
Assecoirs	\$25
Sweets	\$7
Buscuits	\$4

After

Category	Amount
Assecoirs	\$25
Sweets	\$6
Buscuits	\$8

Type 2: New row

Product_Key	Name	Category
1	Sunglases TR-7	Assecoirs
2	Chocolate bar 70% cacao	Sweets
3	Oat meal biscuits	Sweets



UPDATE

Product_Key	Name	Category
1	Sunglases TR-7	Assecoirs
2	Chocolate bar 70% cacao	Sweets
3	<i>Delicious</i> Oat meal biscuits	<i>Buscuits</i>

Type 2: New row

Product_Key	Name	Category
1	Sunglasses TR-7	Assecoirs
2	Chocolate bar 70% cacao	Sweets
3	Oat meal biscuits	Sweets



UPDATE

Product_Key	Name	Category
1	Sunglasses TR-7	Assecoirs
2	Chocolate bar 70% cacao	Sweets
3	<i>Delicious</i> Oat meal biscuits	<i>Buscuits</i>

Product_Key	Name	Category
1	Sunglasses TR-7	Assecoirs
2	Chocolate bar 70% cacao	Sweets
3	Oat meal biscuits	Sweets



Add Row

Product_Key	Name	Category
1	Sunglasses TR-7	Assecoirs
2	Chocolate bar 70% cacao	Sweets
3	Oat meal biscuits	Sweets
4	<i>Delicious</i> Oat meal biscuits	<i>Buscuits</i>

Type 2: New row

Sales_Key	Name	Product_FK	Amount
1	Sunglases TR-7	1	\$25
2	Chocolate bar 70% cacao	2	\$3
3	Oat meal biscuits	3	\$4
4	Chocolate bar 70% cacao	2	\$3
5	Oat meal biscuits	4	\$4

Product_Key	Name	Category
1	Sunglases TR-7	Assecoirs
2	Chocolate bar 70% cacao	Sweets
3	Oat meal biscuits	Sweets
4	Delicious Oat meal biscuits	Buscuits

Respecting history

Category	Amount
Assecoirs	\$25
Sweets	\$3
Buscuits	\$4

- ☐ No updates in fact
- ☐ From that moment new FK

Type 2: New row

No . of products?

Product_PK	Product_ID	Name	Category
1	SG-TR7	Sunglasses TR-7	Assecoirs
2	CH-B70	Chocolate bar 70% cacao	Sweets
3	OT-BSC	Oat meal biscuits	Sweets
4	OT-BSC	Delicious Oat meal biscuits	Buscuits

Count distinct Product_ID

All current products?

Administrate Type 2 SCD

Administrate Type 2 SCD

Product_PK	Product_ID	Name	Category	Ef_Date	Ex_Date
1	SG-TR7	Sunglases TR-7	Assecoirs	2022-01-01	2100-01-01
2	CH-B70	Chocolate bar 70% cacao	Sweets	2022-01-01	2100-01-01
3	OT-BSC	Oat meal biscuits	Sweets	2022-01-01	2022-05-31
4	OT-BSC	Delicious Oat meal biscuits	Buscuits	2022-06-01	2100-01-01

*Period in which
values are valid*



*Instead of null better
date far in the future*



- ✓ Necessary also in ETL to use correct FK
- ✓ Requires Surrogate key instead of Natural key

Administrate Type 2 SCD

Correct FK?

Product_PK	Product_ID	Name	Category	Ef_Date	Ex_Date
1	SG-TR7	Sunglases TR-7	Assecoirs	2022-01-01	2100-01-01
2	CH-B70	Chocolate bar 70% cacao	Sweets	2022-01-01	2100-01-01
3	OT-BSC	Oat meal biscuits	Sweets	2022-01-01	2022-05-31
4	OT-BSC	Delicious Oat meal biscuits	Buscuits	2022-06-01	2100-01-01

- ✓ Add row in the Dimension first
- ✓ Lookup in the Dimension with Natural key + Ef_/Ex_Date

Administrate Type 2 SCD

Correct FK?

Product_PK	Product_ID	Name	Category	Ef_Date	Ex_Date	Is_Current
1	SG-TR7	Sunglases TR-7	Assecoirs	2022-01-01	2100-01-01	Yes
2	CH-B70	Chocolate bar 70% cacao	Sweets	2022-01-01	2100-01-01	Yes
3	OT-BSC	Oat meal biscuits	Sweets	2022-01-01	2022-05-31	No
4	OT-BSC	Delicious Oat meal biscuits	Buscuits	2022-06-01	2100-01-01	Yes

- ✓ Add row in the Dimension first
- ✓ Lookup in the Dimension with Natural key + Ef_/Ex_Date
- ✓ Additional for Is_Current

Mixing Type 1 + 2

Mixing Type 1 + 2

Product_PK	Product_ID	Name	Category	Ef_Date	Ex_Date
1	SG-TR7	Sunglases TR-7	Assecoirs	2022-01-01	2100-01-01
2	CH-B70	Chocolate bar 70% cacao	Sweets	2022-01-01	2100-01-01
3	OT-BSC	Oat meal biscuits	Sweets	2022-01-01	2100-01-01
4	OT-BSC	Delicious Oat meal biscuits	Buscuits	2022-06-01	2100-01-01

✓ Some attributes can be Type 1 and some Type 2

Mixing Type 1 + 2

Product_PK	Product_ID	Name	Category	Ef_Date	Ex_Date
1	SG-TR7	Sunglases TR-7	Assecoirs	2022-01-01	2100-01-01
2	CH-B70	Chocolate bar 70% cacao	Sweets	2022-01-01	2100-01-01
3	OT-BSC	Oat meal biscuits	Sweets	2022-01-01	2100-01-01
4	OT-BSC	Delicious Oat meal biscuits	Buscuits	2022-06-01	2100-01-01

- ✓ No set in stone rules but needs to be defined with business users
- ✓ *Not a technical decision*

Type 2: Implementation

Step 1: Customer Dimension Table Structure

```
Customer_Dimension ( Id INT PRIMARY KEY, -- Surrogate Key  
Customer_Id INT, -- Natural Key  
Customer_Name VARCHAR(100),  
City VARCHAR(100),  
Effective_Start_Date DATE,  
Effective_End_Date DATE,  
Is_Current BOOLEAN )
```

Step 2: Sample Historical Data for the Customer

Id	Customer_Id	Customer_Name	City	Effective_Start_Date	Effective_End_Date	Is_Current
1	101	Rajesh Kumar	Mumbai	2020-01-01	2021-11-30	FALSE
2	101	Rajesh Kumar	Bhubaneswar	2021-12-01	NULL	TRUE

Steps 3: ETL Logic for SCD Type 2

Cont...

-- Step 1: Check if Customer exists and city has changed

IF EXISTS (

SELECT 1

FROM Customer_Dimension

WHERE Customer_Id = 101

AND Is_Current = TRUE

AND City <> 'Bhubaneswar'

)

THEN

-- Step 2: Expire the existing record

UPDATE Customer_Dimension

SET Effective_End_Date = '2021-11-30',

Is_Current = FALSE

WHERE Customer_Id = 101

AND Is_Current = TRUE;

Steps 3: ETL Logic for SCD Type 2

```
-- Step 3: Insert new version of the record
INSERT INTO Customer_Dimension (
    Id, Customer_Id, Customer_Name, City,
    Effective_Start_Date, Effective_End_Date, Is_Current
)
VALUES (
    NEXTVAL('customer_dim_seq'), -- assuming a surrogate key sequence
    101, 'Rajesh Kumar', 'Bhubaneswar',
    '2021-12-01', NULL, TRUE
);
END IF;
```

Step 4: Query – Value of Goods Purchased in Each City

Assuming we have a **Fact Table**:

```
Sales_Fact (  
Sales_Id INT,  
Customer_Id INT,  
Sale_Amount DECIMAL(10,2),  
Sale_Date DATE  
)
```

Step 4: Query – Value of Goods Purchased in Each City

Query to Join with Customer_Dimension and Get Spend per City

```
SELECT
    CD.City,
    SUM(SF.Sale_Amount) AS Total_Spend
FROM
    Sales_Fact SF
JOIN
    Customer_Dimension CD
    ON SF.Customer_Id = CD.Customer_Id
    AND SF.Sale_Date BETWEEN CD.Effective_Start_Date AND
COALESCE(CD.Effective_End_Date, CURRENT_DATE)
WHERE
    CD.Customer_Id = 101
GROUP BY
    CD.City;
```

Results

City	Total_Spend
Mumbai	125000.00
Bhubaneswar	75000.00

Type 3: Additonal Attributes

Type 3: Additonal Attributes

Product_PK	Product_ID	Name	Category	Ef_Date	Ex_Date
1	SG-TR7	Sunglases TR-7	Assecoirs	2022-01-01	2100-01-01
2	CH-B70	Chocolate bar 70% cacao	Sweets	2022-01-01	2100-01-01
3	OT-BSC	Oat meal biscuits	Sweets	2022-01-01	2100-01-01
4	OT-BSC	Delicious Oat meal biscuits	Buscuits	2022-06-01	2100-01-01

- ✓ *Type 2 – Default strategy to maintain reflect history*
- ✓ *Type 1 – Static*
- ✓ *Type 3 – In-between: Switching back & forth between versions*

Type 3: Additional Attributes

Product_PK	Product_ID	Name	Category	Prev_Category
1	SG-TR7	Sunglasses TR-7	Assecoirs	Assecoirs
2	CH-B70	Chocolate bar 70% cacao	Sweets	Sweets
3	OT-BSC	Oat meal biscuits	Biscuit	Sweets

✓ *Instead of adding a row – we add a column*

Type 3: Additonal Attributes

Product_PK	Product_ID	Name	Category	Prev_Category
1	SG-TR7	Sunglases TR-7	Assecoirs	Assecoirs
2	CH-B70	Chocolate bar 70% cacao	Sweets	Sweets
3	OT-BSC	Oat meal biscuits	Biscuit	Sweets

✓ *Instead of adding a row – we add a column*

Category	Amount
Assecoirs	\$25
Sweets	\$6
Buscuits	\$8



Prev_Category	Amount
Assecoirs	\$25
Sweets	\$14

Type 3: Additonal Attributes

Product_PK	Product_ID	Name	Category	Prev_Category
1	SG-TR7	Sunglases TR-7	Assecoirs	Assecoirs
2	CH-B70	Chocolate bar 70% cacao	Sweets	Sweets
3	OT-BSC	Oat meal biscuits	Biscuit	Sweets

- ✓ *Instead of adding a row – we add a column*
- ✓ *Typically used for significant changes at a time
(e.g. restructurings in organizations)*

Type 3: Additonal Attributes

Sales_Key	Name	Region_FK	Amount
1	Sunglases TR-7	1	\$25
2	Chocolate bar 70% cacao	2	\$3
3	Oat meal biscuits	3	\$4
4	Chocolate bar 70% cacao	2	\$3
5	Oat meal biscuits	3	\$4

Reg_PK	Region	Prev_Region
1	North	North
2	West	West
3	South	West

- ✓ *Instead of adding a row – we add a column*
- ✓ *Typically used for significant changes at a time
(e.g. restructurings in organizations)*
- ✓ *Enables switching between historic / current view*

Type 3: Additional Attributes

Sales_Keyc	Name	Region_FK	Amount
1	Sunglases TR-7	1	\$25
2	Chocolate bar 70% cacao	2	\$3
3	Oat meal biscuits	3	\$4
4	Chocolate bar 70% cacao	2	\$3
5	Oat meal biscuits	3	\$4

Reg_PK	Region	Prev_Region
1	North	North
2	West	West
3	South	West
4	East	Not applicable

✓ *New attributes => New rows*

Limitations

✓ *It is possible to add multiple historic columns*

- ❖ Not suitable for frequent or unpredictable changes => better Type 2
- ❖ Minor changes => better Type 1

Least frequent type

Type 3: Additional Attributes

Sales_Keyc	Name	Region_FK	Amount
1	Sunglases TR-7	1	\$25
2	Chocolate bar 70% cacao	2	\$3
3	Oat meal biscuits	3	\$4
4	Chocolate bar 70% cacao	2	\$3
5	Oat meal biscuits	3	\$4

Reg_PK	Region	Prev_Region
1	North	North
2	West	West
3	South	West

✓ *Not suitable for frequent or unpredictable changes*

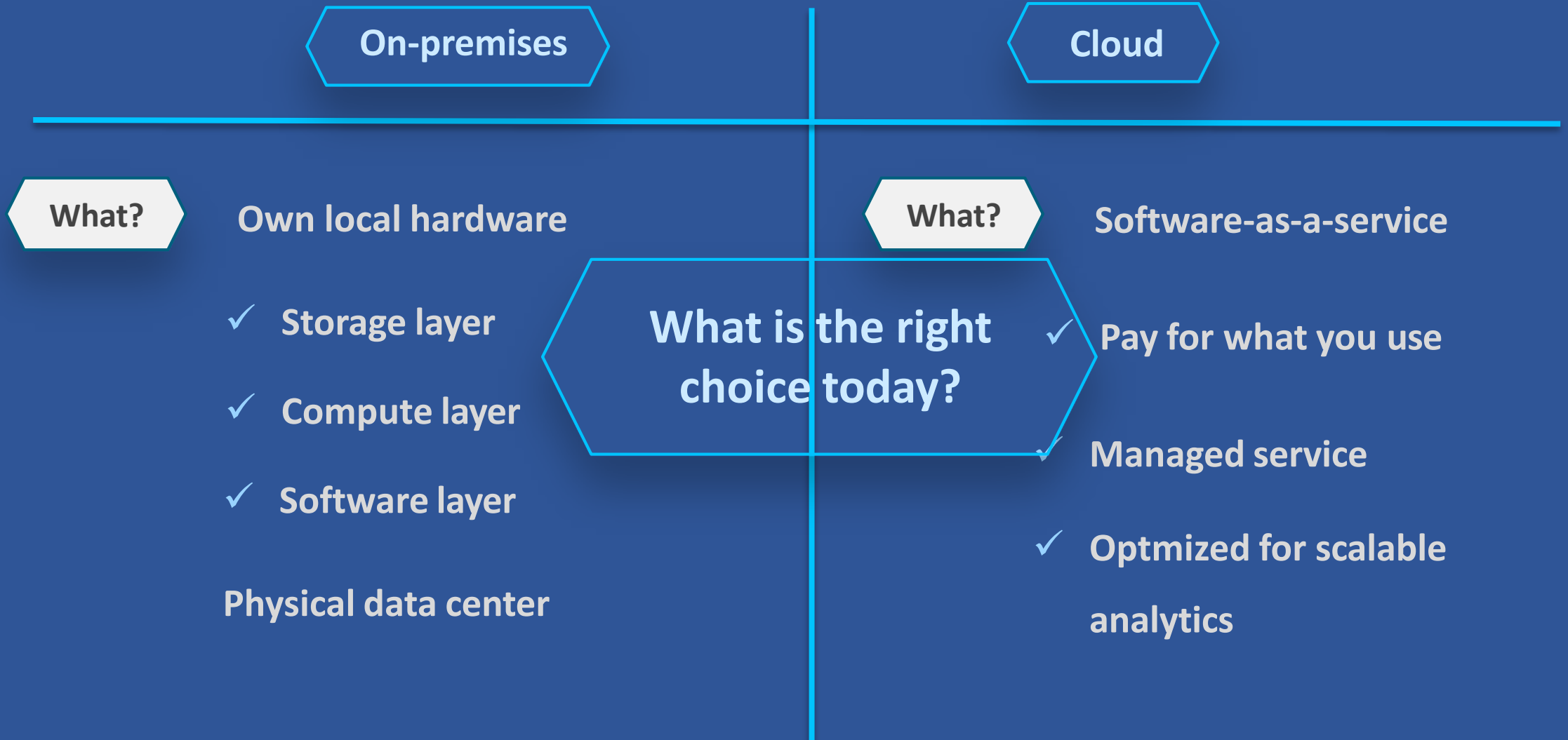
=> better Type 2

✓ *It is possible to add multiple historic columns*

✓ *Minor changes => better Type 1*

Least frequent type
(two versions
needed)

Cloud vs. On-premises



Cloud vs. On-premises

On-premises

Benefits

- ✓ Full control
- ✓ Data governance & compliance

Problems

- ❖ Full responsibility
- ❖ High costs
- ❖ More internal resources
- ❖ Less flexible

Cloud

Benefits

- ✓ Fully managed
- ✓ Scalable
- ✓ Cost-efficient
- ✓ Managed security
- ✓ Availability
- ✓ Time to market

Problems

- ❖ Regulations
- ❖ Different providers?

Conclusion?

Which one to
choose?

- ✓ Cloud data warehouses are on the rise
- ✓ Most companies opt for cloud data warehouse

In most cases cloud data warehouse is the better choice nowadays!

Conclusion?

What are the
options?

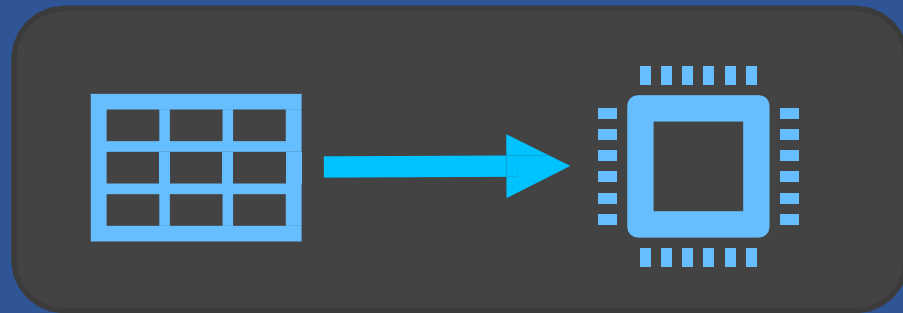
- ✓ **Snowflake**
- ✓ **Amazon Redshift**
- ✓ **Azure Synapse**
- ✓ **Google Big Query**

Conclusion?

What are the
options?

- ✓ **Snowflake**
- ✓ **Amazon Redshift**
- ✓ **Azure Synapse**
- ✓ **Google Big Query**

Massive parallel processing (MPP)



Traditional

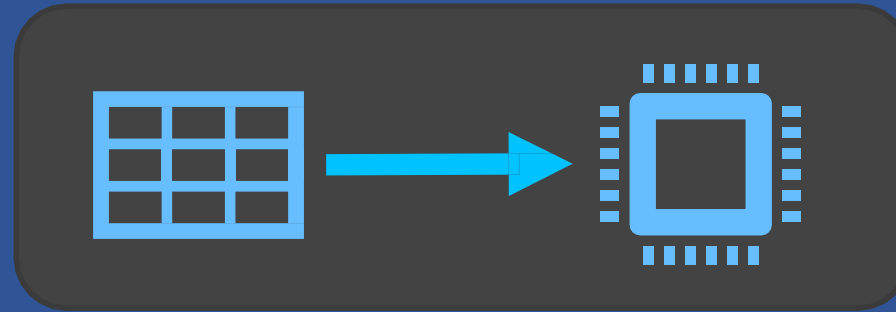
Massive parallel processing (MPP)

Example

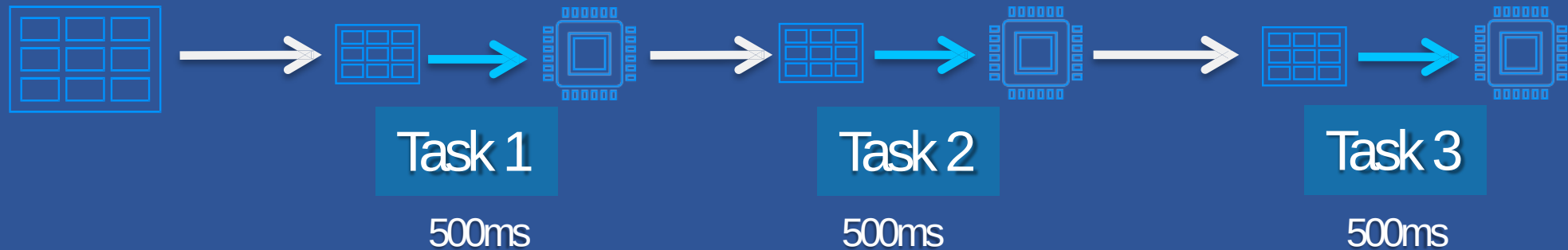
transaction_id [PK] integer	product_id character varying	customer_id integer	payment character varying	price numeric
1	P0494	4	visa	18.29
2	P0221	5	visa	1.49
3	P0625	5	visa	5.89
4	P0431	8	mastercard	11.59
5	P0058	5	mastercard	12.39

```
SELECT  
*  
FROM sales  
WHERE customer_id = 5
```

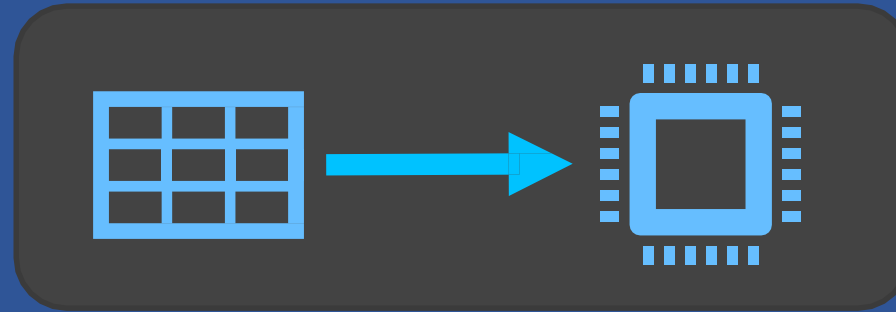
Massive parallel processing (MPP)



Traditional

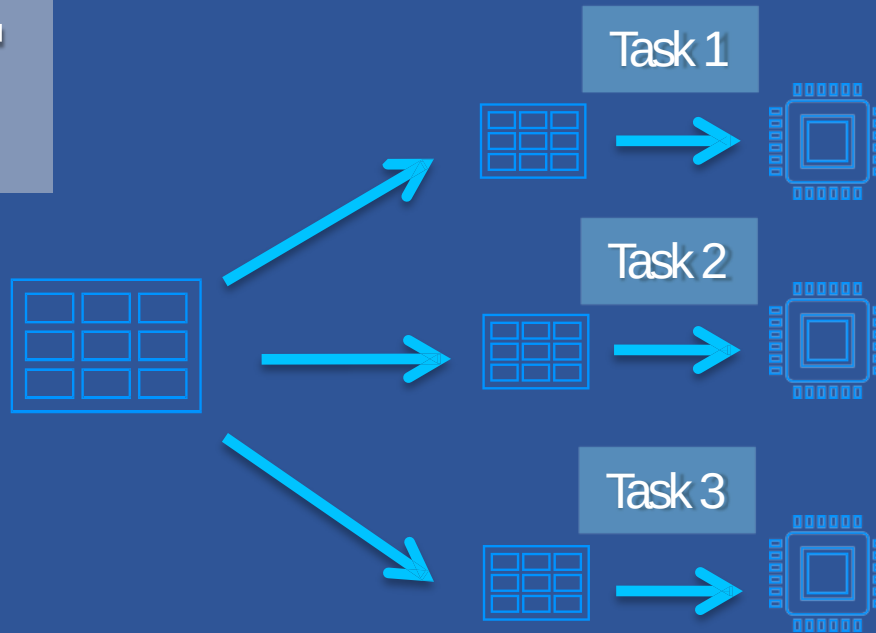


Massive parallel processing (MPP)



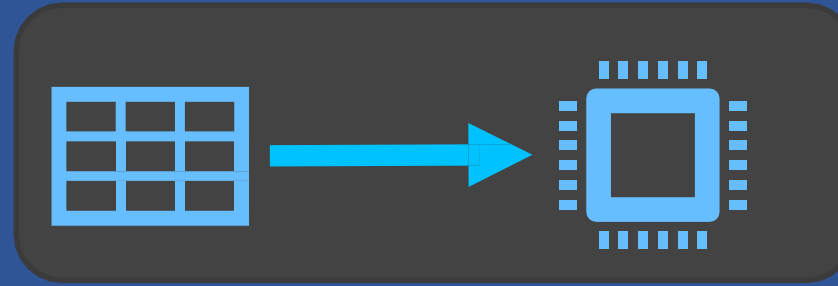
Traditional

"Shared disk"
architecture



MPP

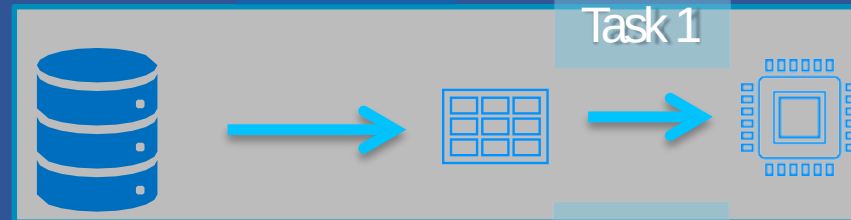
Massive parallel processing (MPP)



Traditional

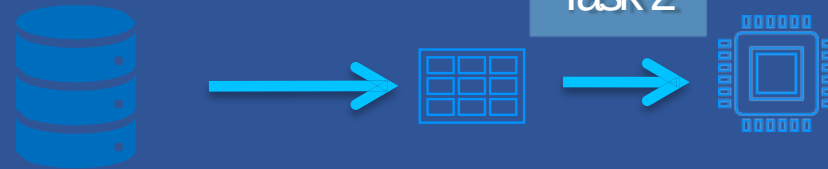
Node

"Shared nothing" architecture



Work load is split up & processed individually

Independent resources



MPP



Massive parallel processing (MPP)

- ✓ Modern way of solving performance issues
- ✓ Millions of rows can be processed faster
- ✓ Many people can run queries at the same time with good performance
- ✓ Helpful with centralizing massive amounts of data

Columnar databases

transaction_id [PK] integer	product_id character varying	customer_id integer	payment character varying	price numeric
1	P0494	4	visa	18.29
2	P0221	5	visa	1.49
3	P0625	5	visa	5.89
4	P0431	8	mastercard	11.59
5	P0058	5	mastercard	12.39

```
SELECT  
product_id  
FROM sales
```

```
3, P0625, 5, visa
```

```
4, P0432, 8, mastercard
```

```
1, P0494, 4, visa
```

```
6, P0058, 5, mastercard
```

```
2, P0221, 5, visa
```

Traditional
Relational DB

All rows have to be scanned

Good for transactional DB

Bad for fast data retrieval!

Columnar databases

transaction_id [PK] integer	product_id character varying	customer_id integer	payment character varying	price numeric
1	P0494	4	visa	18.29
2	P0221	5	visa	1.49
3	P0625	5	visa	5.89
4	P0431	8	mastercard	11.59
5	P0058	5	mastercard	12.39

1, 2, 3, 4, 5

P0494, P0221, P0625, P0431, P0058

4, 5, 5, 8, 5

visa, visa, visa, mastercard, mastercard

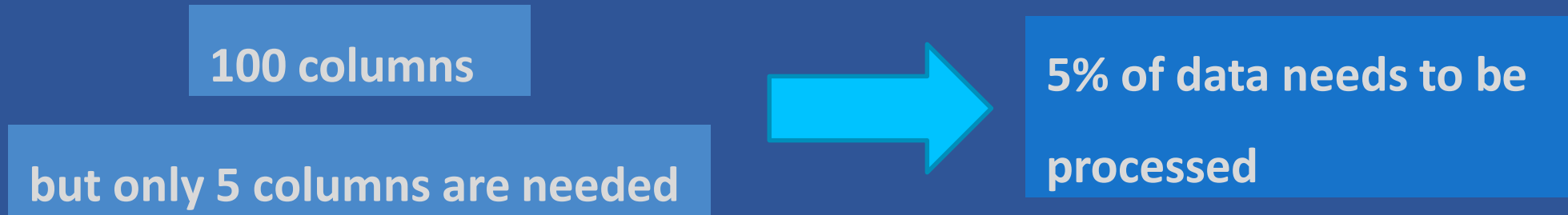
18.29, 1.49, 5.89, 11.59, 12.39

```
SELECT  
  product_id  
FROM sales
```

Less data needs to be processed!

Better compression, less storage

Columnar databases



- ✓ Important factor in improving analytical query performance

Guidelines

index if you frequently want to retrieve less than about 15% of the rows in a large table

Index columns used for joins to improve join performance

Small tables do not require indexes

PK automatically per default has an index

- There is a wide range of values (good for regular indexes).
- There is a small range of values (good for bitmap indexes).